Foundations: quantum-information preliminaries, number model, exact admissibility and emission

Contract, version 0.12.0

The foundations module states the arithmetic every other module relies on, and none of its mathematics is specific to a protocol; one emission rule names the data of a protocol declaration it binds. A declared value is a binary64 number and denotes exactly the dyadic rational it is; a sum is accumulated in a declared order and rounds nowhere when each of its binary64 additions is exact, which makes it the exact sum (Lemma 3.4). A declared operator family is exactly admissible for a sector map when it is exactly Hermitian, positive and complete, with a completeness sum that rounds nowhere (Definition 4.3). The requirements say who establishes this: declared operator data are exactly admissible at every module boundary (Requirement 9.1), every consumer decides its predicates exactly and refuses otherwise, and a producer verifies what it emits before emitting it (Requirement 11.3). One declared tolerance per instance bounds only how far an instance may move its own declaration, only a declarer snaps its data onto its lattice, and a snap of a measurement family keeps the blocks known exactly on the lattice and moves the others by one fidelity-first procedure that revives no zero (Requirement 10.5). A moved block is related to its model by no claim, so a run evaluates it off its snap, with the reference handed beside the snap in its place (Definition 5.7), or rests there on a named assumption (Assumption 8.1) unless it declares the strict option that refuses it; the module fixes the exact island, the bounds that carry a least eigenvalue, a rank and a matrix function across an enclosure, and the three tiers of a reported key rate. Beside the arithmetic it states the round model, with its attack sets and the round homogeneity the modules' round assumptions assert, and the quantum-information preliminaries the entropy optimisation is written in: pinching, the sandwiched Rényi divergence and its conditional entropies, the classical divergences and the quantum relative entropy, the extended divergence kernel (Lemma 1.4), the bound by which pinching lowers the trace power (Lemma 1.5), and two published results on pinched trace powers (Fact 1.6, Fact 1.7). The module's main claims are Lemma 3.4, Definition 4.3 and Requirement 9.1 on the side of the arithmetic, and Lemma 1.4 and Lemma 1.5 on the side of the entropy optimisation. It reads nothing from another module, and the strengthenings of exact admissibility that a module states for its own data, such as a declared lattice or conditions on Alice's data, stay with that module.

Grey text, here and throughout, is non-normative: it explains, and nothing in it is assumed or proved.

The design notes (non-normative) explain why the module is built as it is and draw

its processing flowchart. The dependencies between the sections are drawn in the section dependency matrix, and the graph index opens every section's dependency graph. The statements digest holds every definition, assumption, result and requirement without the proofs. How to read a contract explains this layout and its links.

Contents

Main results 3

Assumptions at a glance 3

Notation 4

Terms 4

I Mathematics 11

1 Quantum-information preliminaries 11

2 The round model 16

3 Binary64 numbers and exact sums 23

4 Declared data and exact admissibility 27

5 The declared tolerance and the snap 31

6 Domination and its limits 39

7 The exact island, enclosures and the tiers 50

8 The snapped-measurement assumption 62

II Engine requirements 65

9 Admission 65

10 The tolerance, norms and snapping 67

11 Emission 77

12 Arithmetic and tiers 80

Claim index 82

Index of defined terms 85

References 86

Main results

The module's most important results, those its overview names as what it establishes, with every assumption each rests on, directly or through the claims it uses. Each links to its statement and proof; the statements digest holds them in full.

Lemma 1.4 Extended divergence kernel
The divergence kernel $d(u, v) = u \log_2(u/v)$, extended to vanishing arguments, is jointly convex, lower semicontinuous and non-increasing in v. These properties keep the objective well defined at support boundaries and let it be composed with concave branch quantities.
Rests on no assumption

Lemma 1.5 Pinching lowers the trace power
For $\alpha > 1$, pinching the α-th root of a positive semidefinite operator and raising the result to the power α never increases the trace, and leaves it unchanged exactly when the operator is block diagonal for the pinching. This bounds the trace-power terms of the MEAT objective.
Rests on no assumption

Lemma 3.4 Exactness of a sum that rounds nowhere
A sequential sum rounds nowhere exactly when every exact partial sum is a binary64 number, and then the computed sum equals the exact sum at every step. A predicate stated on such a sum is a predicate on the declared values themselves.
Rests on no assumption

Definition 4.3 Exact admissibility of a declared operator family
A declared operator family is exactly admissible for a sector map with constant c when (T) its own sector map agrees, (H) every block is exactly Hermitian, (P) positive, (C) it sums to c times the identity and (S) its declared-order sum rounds nowhere. Each is a property of the declared numbers.
Rests on no assumption

Requirement 9.1 Exact admissibility at every module boundary
Declared operator data cross every module boundary exactly admissible, so every sum a predicate reads is the exact sum. The declarer makes them so, or, for data an instance passes on unchanged, the consumer's exact decision on entry does, and a consumer that finds otherwise refuses them.
Rests on no assumption

Assumptions at a glance

Every assumption this contract states or its claims assume, with its category and the claims of this contract that assume it directly. An assumption of another contract links into that contract's PDF.

Assumption	Category	Title	Stated as	Assumed directly by
A-SNAP	receiver	The device measures the snapped family where the run reads the snap	Assumption 8.1	Req. 10.5, Req. 11.3

Notation

The symbols of the repository's notation catalogue (spec/notation.md) that this contract uses, in the catalogue's order, with their meaning.

Symbol	Meaning
$\mathcal{H}(A)$	Hilbert space of the system label A
$\mathcal{L}(A)$	Bounded linear operators on $\mathcal{H}(A)$
$\mathrm{Pos}(A)$	Positive semidefinite operators
$S_=(A)$	Normalized states
$S_\leq(A)$	Subnormalized states
$\mathbb{1}$, $\mathbb{1}_C$	Identity operator (contextual / named)
id	Identity channel
$\mathfrak{L}$	Declared constraint list $\{(\hat{O}_k, \bowtie_k, \Theta_k)\}$
V_R, V_Z	Raw-key isometry; raw key $\to$ key register
$\mathcal{Z}$	The key pinching, on the raw-key register R
τ_{num}	The instance's one declared numerical tolerance: the ceiling on the pre-snap defect an instance may repair in what it declares — a declared block whose measured defect (Hermiticity, positivity, completeness, column-stochasticity, a probability sum) exceeds it is a modelling error and is refused (a probability vector prepared from numerical inputs is admitted by the input preparation's rule instead, Req. 10.1); one within it is snapped onto an exactly admissible family and the snap is recorded with the defect it corrected
λ_{EC}	Error-correction transcript length, total bits, fixed-length register (shorter transcripts padded)
D	Classical KL divergence, base 2; also the quantum relative entropy on operators
$\widetilde{D}_\alpha$	Sandwiched Rényi divergence
$\widetilde{H}^{\uparrow}_\alpha$	Sandwiched Rényi conditional entropy, up
$\widetilde{H}^{\downarrow}_\alpha$	Sandwiched Rényi conditional entropy, down
$\lambda_{\min}$	Minimum eigenvalue
T	Transpose, X^{T}

Terms

The phrases this contract defines, and the terms of other contracts it uses, each with a one-line gloss and the definition that fixes it. In the text, a linked term opens its definition.

Term	Meaning	Defined in
admissible attack sequence	an attack sequence whose every round lies in the attack set of a given constraint list	Definition 2.1
attack sequence	one channel per round whose output is the per-round system, with Alice's system where the adversary supplies it, and Eve's new memory	Definition 2.1
attack set	the channels whose adjoint maps every listed operator into the stated relation with its constant	Definition 2.1
binary64 numbers	the finite values of the IEEE 754 binary64 format, each a dyadic rational	Definition 3.1
binary64 sum	fl(a + b), the exact sum of a and b rounded to the nearest binary64 number, ties to even, or to an infinity on overflow	Definition 3.1
block list	one square complex matrix per block of a sector map, of that block's dimension, identified with their direct sum	Definition 4.2
certified	the tier of a lower bound on the minimum of the objective of a declaration, formed with every arithmetic step from that declaration's data exact or rigorously enclosed, so that none of its hypotheses concerns arithmetic	Definition 7.6
computed object	an object a module computes itself in the course of a computation, such as an iterate of an optimiser; it is not declared data	Definition 4.1
computed sequential sum	the left-to-right accumulation of a finite sequence from zero with every addition rounded by fl, entrywise, with no re-ordering and no re-association	Definition 3.3
consumer	a module other than a supplier that reads declared data	Definition 4.1

Term	Meaning	Defined in
declared data	the data an instance states and the data a producer emits as its own declaration, with the sets such data define	Definition 4.1
declared order	a total order on a finite alphabet, fixed together with the alphabet	Definition 3.3
declared tolerance	a binary64 number, at least zero and below one, that an instance states in its run configuration	Definition 5.1
declared value	a binary64 number stated as a datum, which denotes exactly that dyadic rational and approximates nothing	Definition 3.1
declarer	the party whose declaration declared data are: the instance that states them or the producer that emits them	Definition 4.1
defects	the entrywise maximum moduli of the differences that the predicates of data name: the Hermiticity defect and the completeness, column, trace and sum residues	Definition 5.1
entrywise enclosure	a Gaussian-rational Hermitian centre together with symmetric non-negative rational radii that bound, entry by entry, the modulus of the difference between a matrix and that centre	Definition 5.3
estimate	the tier of a key-rate number that is the value of the objective at the point an optimiser returned, claimed as no bound	Definition 7.6
evaluated off its snap	said of a block of a snap in a run: no number from which the run computes its key length reads the snapped block, save the points at which those numbers are evaluated and the values other than the output length that the run fixes before its first round	Definition 5.7

Term	Meaning	Defined in
exact island	the computations of the engine, its instances' declarations included, by which a consumer of data or the declarer about to declare them decides whether they satisfy the conditions their declaration states, a declarer computes and decides the reference of a measurement family, a consumer's replacement of an emitted scalar bound is decided to lie on the bound's weakening side, a producer constructs the operator data it emits, the acceptance side of the optimisation is constructed and certified, or a lower bound on a linear minimum over the feasible side of the optimisation is certified	Definition 7.1
exact reference	a reference all of whose radii are zero, so that each block is its Gaussian-rational centre	Definition 5.3
exactly admissible	said of a declared operator family whose sector map agrees with the given one, whose blocks are exactly Hermitian and positive semidefinite, and whose sum in the declared order equals the constant times the identity on every block and rounds nowhere; the constant is one where none is named	Definition 4.3
factor enclosure	a matrix box of columns together with an exact positive diagonal core, such that the matrix is the product of a matrix L the box contains, the core and the adjoint of L	Definition 5.4
first-order	the tier of a key-rate number that is a lower bound on the minimum of the objective, established by a claim under hypotheses that claim names	Definition 7.6
handed reference	a reference of a snapped measurement family that its declarer passes, with its centres and radii and unaltered, to every consumer of the snap	Definition 5.7
instance	the party that fixes one protocol's device data, protocol data and run configuration	Definition 4.1

Term	Meaning	Defined in
kept	a block of a snap of an operator family read as a measurement on which the snap is decided exactly to equal the data it is made from	Definition 5.5
lattice exponent	an integer k with $1 \leq k \leq 53$	Definition 3.1
lattice value	at a lattice exponent k, a real number that is an integer multiple of two to the power $-k$ and has modulus at most one; a complex number or a matrix is one when the real and the imaginary part of every entry are	Definition 3.1
matrix box	a matrix of Gaussian-rational centres together with non-negative rational radii that bound, entry by entry, the modulus of the difference between a rectangular complex matrix and that centre	Definition 5.4
module boundary	the passage of declared data from their declarer to a consumer	Definition 4.1
moved	a block of a snap of an operator family read as a measurement that is not kept: the snap is not decided exactly to equal on it the data it is made from	Definition 5.5
one-block map	the sector map of a system without a declared sector decomposition: one block of the system's dimension, with a block kind and an identifier common to every such system	Definition 4.2
per-round family	a finite list of objects that a model of the run declares for a round	Definition 2.2
positivity floor	the condition that the least eigenvalue of the Hermitian part of a square matrix is at least minus the declared tolerance	Definition 5.1
producer	a module other than a supplier that emits numerical data, constructed from declared data it read, to another module as its own declaration	Definition 4.1
reference	a family of Hermitian positive semidefinite block lists complete at the constant on every block, read together with an entrywise enclosure of every block of every element	Definition 5.3

Term	Meaning	Defined in
residual family	on a block with weight q, the family (G − (1−q)F)/q where q is positive, and G itself where q is zero	Definition 6.1
rigorously enclosed	said of a computation each of whose operations returns a set of numbers, such as an interval or a ball, containing the exact result of the operation at every value its inputs' sets contain	Definition 7.6
round model	n rounds of a per-round system S, with the attack sequences and attack sets below	Definition 2.1
round-homogeneous	one fixed list of objects describes every round, and every round behaves as they describe whatever the other rounds do	Definition 2.2
rounds nowhere	said of an accumulation every one of whose binary64 additions is exact	Definition 3.3
sector map	a block kind together with a non-empty, ordered list of block identifiers, each with a dimension	Definition 4.2
sequential sum	the exact sum of a finite sequence, accumulated left to right from zero	Definition 3.3
snap	at a lattice exponent k, data of the same shape as the given data, over the same alphabet in the same order, with every entry a lattice value at k, that satisfy exactly the conditions the declaration of the given data states	Definition 5.5
structural support	of a block of an element: the set of indices whose row or column holds an entry that is not structurally zero	Definition 5.5
structurally zero	said of an entry of data known to be exactly zero: for data given as numbers, an entry that is the number zero; for a reference, an entry whose centre and radius are both zero; and no other entry	Definition 5.5
sum in the declared order	the sequential sum of a family indexed by an alphabet, taken in the alphabet's declared order	Definition 3.3

Term	Meaning	Defined in
supplier	a module whose own claims decide their conditions on the data it reads or emits in floating point, at slacks an instance declares to it, rather than exactly	Definition 4.1
tier	one of three labels, estimate, first-order or certified, that a reported key-rate number carries	Definition 7.6
trusted-source mode	The source mode in which Alice's retained system never reaches the adversary, with a declared marginal and, above dimension one, a certificate.	transmitter/def: source-spec
untrusted-source mode	The source mode with no certificate and no marginal, in which the adversary may have prepared Alice's retained system.	transmitter/def: source-spec
within the declared tolerance	said of data whose defects are each at most the declared tolerance and whose blocks read as positive semidefinite each pass the positivity floor at it	Definition 5.1

Part I
Mathematics

1 Quantum-information preliminaries

These are the quantum-information objects the entropy optimisation is written in: pinching, the sandwiched Rényi divergence with its two conditional entropies, and the classical divergences with the quantum relative entropy. The section's main results, two lemmas, establish what the optimisation needs of them, that the extended divergence kernel is jointly convex, lower semicontinuous and non-increasing in its second argument (Lemma 1.4), and that pinching lowers the trace power (Lemma 1.5). Two published results close the section, the pinched trace-power form of the conditional Rényi entropy (Fact 1.6) and the concavity of the pinched trace-power functional (Fact 1.7).

Operator, state and channel conventions, and the pinching of a complete family of orthogonal projectors, which deletes the coherences between their ranges. A pinching is a self-adjoint, idempotent, unital mixture of unitary conjugations whose fixed points are the block-diagonal operators.

Definition 1.1 (Operator conventions and pinching)**.** All Hilbert spaces are finite-dimensional. For a system label A, $\mathcal{L}(A)$ is the space of linear operators on $\mathcal{H}(A)$, $\mathrm{Pos}(A)$ its positive semidefinite operators, and $S_=(A)$ and $S_\leq(A)$ the normalized and the subnormalized states, of trace equal to one and at most one. A channel is a CPTP map; an instrument branch is a CP trace-non-increasing (CPTNI) map. For $X \in \mathrm{Pos}(A)$ and $p > 0$, the power X^p is defined by functional calculus on the support of X and by $0^p = 0$ on its kernel; in particular X^p is block-diagonal for any orthogonal decomposition for which X is.

For a family of orthogonal projectors $\{\Pi_j\}_j$ on $\mathcal{H}(Q)$ with $\sum_j \Pi_j = \mathbb{1}_Q$, the associated *pinching* is the channel $\tau \mapsto \sum_j \Pi_j \tau \Pi_j$. A pinching is CPTP, self-adjoint as a superoperator ($\mathrm{tr}[Y\,\mathcal{E}(W)] = \mathrm{tr}[\mathcal{E}(Y)\,W]$), idempotent, and unital. An operator X is a fixed point iff it is block-diagonal, $X = \sum_j \Pi_j X \Pi_j$; functional calculus of a block-diagonal PSD operator acts blockwise, so a pinching commutes with taking powers of its fixed points. Each of these properties is a one-line computation from the form $\sum_j \Pi_j(\cdot)\Pi_j$ and orthogonality of the Π_j; self-adjointness is $\mathrm{tr}\left[Y \sum_j \Pi_j W \Pi_j\right] = \mathrm{tr}\left[\sum_j \Pi_j Y \Pi_j\, W\right]$ by cyclicity.

A pinching is a uniform mixture of unitary conjugations. Enumerate the projectors as $\Pi_1, \ldots, \Pi_m$, put $\zeta := e^{2\pi i/m}$ and $U_k := \sum_{j=1}^m \zeta^{jk} \Pi_j$ for $k \in \{0, \ldots, m-1\}$; each U_k is unitary, $U_k U_k^\dagger = \sum_j |\zeta^{jk}|^2 \Pi_j = \mathbb{1}_Q$ by

orthogonality, and

$$\sum_j \Pi_j X \Pi_j = \frac{1}{m} \sum_{k=0}^{m-1} U_k X U_k^\dagger \qquad \text{for every } X \in \mathcal{L}(Q), \tag{1.1}$$

since the right-hand side is $\sum_{j,j'} \big(\frac{1}{m}\sum_k \zeta^{(j-j')k}\big)\Pi_j X \Pi_{j'}$ and the inner average is 1 for $j = j'$ and 0 otherwise. In particular a pinching does not increase the operator norm, a mixture of unitary conjugates of X having norm at most $\|X\|$.

Used by Lem. 1.5, Fact 1.6, Def. 2.1, meat-core/lem:b-derivative, meat-core/lem:kraus-freedom, meat-core/prop:a-neutral · newly derived · implemented · *Code* certiqs_key.meat_core.objective.linalg:pinch

The sandwiched Rényi divergence of order $\alpha > 1$ and the two conditional entropies it induces, one at the fixed marginal and one optimised over the marginal, in bits. The fixed-marginal entropy is no larger, so substituting it inside a lower bound is conservative, and the reverse substitution is not.

The source [3, Def. 5.2] writes the two conditional entropies of the following definition, its Eqs. (5.18)–(5.19), with a tilde, as `spec/notation.md` does for the sandwiched family; the pair it writes with an overline, its Eqs. (5.16)–(5.17), is the Petz family, which is a different quantity.

Definition 1.2 (Sandwiched Rényi divergence and conditional entropies)**.** For $\alpha \in (1,\infty)$, $\rho \in S_{\le}(AB)$ and $\sigma \in \mathrm{Pos}(AB)$, on the same system,

$$\widetilde{D}_\alpha(\rho\,\|\,\sigma) \coloneqq \frac{1}{\alpha-1}\log_2 \mathrm{tr}\Big[\Big(\sigma^{\frac{1-\alpha}{2\alpha}}\,\rho\,\sigma^{\frac{1-\alpha}{2\alpha}}\Big)^\alpha\Big] \tag{1.2}$$

when $\mathrm{supp}\,\rho \subseteq \mathrm{supp}\,\sigma$, and $\widetilde{D}_\alpha(\rho\|\sigma) \coloneqq +\infty$ otherwise. The exponent $\frac{1-\alpha}{2\alpha}$ is negative, and the power of σ is taken on its support and is zero on its kernel, as for the generalised inverse; where $\mathrm{supp}\,\rho \subseteq \mathrm{supp}\,\sigma$, $\rho = P\rho P$ for the projector P onto that support, so the value reads the power on the support alone. For $\rho_{AB} \in S_=(AB)$,

$$\widetilde{H}^{\downarrow}_\alpha(A|B)_\rho \coloneqq -\widetilde{D}_\alpha(\rho_{AB}\,\|\,\mathbb{1}_A\otimes\rho_B), \qquad \widetilde{H}^{\uparrow}_\alpha(A|B)_\rho \coloneqq -\inf_{\sigma_B\in S_=(B)} \widetilde{D}_\alpha(\rho_{AB}\,\|\,\mathbb{1}_A\otimes\sigma_B). \tag{1.3}$$

Both are in bits. These are the definitions of [3, Def. 5.2, Eqs. (5.18)–(5.19)]. Since the infimum in (1.3) includes $\sigma_B = \rho_B$,

$$\widetilde{H}^{\downarrow}_\alpha(A|B)_\rho \;\le\; \widetilde{H}^{\uparrow}_\alpha(A|B)_\rho. \tag{1.4}$$

The substitution $\widetilde{H}^{\uparrow}_\alpha \to \widetilde{H}^{\downarrow}_\alpha$ inside a lower bound is therefore conservative, and the reverse is not.

Used by Fact 1.6, meat-core/fact:meat, meat-core/lem:purifier-indep, meat-core/prop:value-range, meat-core/rem:alpha-domain, meat-core/rem:losses, meat-core/thm:accumulated-entropy, meat-core/thm:keylength, meat-core/thm:reduction · newly derived · implemented · *Code* certiqs_key.meat_core.objective.linalg:psd_trace_power

The base-2 Kullback–Leibler and Rényi divergences of probability vectors and the quantum relative entropy, with their conventions at zero. The Kullback–Leibler divergence, the statistics penalty of the MEAT objective, is non-negative and vanishes only at equal vectors.

Definition 1.3 (Classical divergences and the quantum relative entropy)**.** For probability vectors $\mathbf{q}, \mathbf{p}$ on a finite alphabet, $D(\mathbf{q}\|\mathbf{p}) \coloneqq \sum_c q(c) \log_2 \frac{q(c)}{p(c)}$ with the conventions $0 \log_2 \frac{0}{p} = 0$ and $q \log_2 \frac{q}{0} = +\infty$ for $q > 0$. For $\alpha \in (1, \infty)$ the classical Rényi divergence is $D_\alpha(\mathbf{q}\|\mathbf{p}) \coloneqq \frac{1}{\alpha - 1} \log_2 \sum_c q(c)^\alpha p(c)^{1-\alpha}$ (with $+\infty$ if $\mathbf{q} \not\ll \mathbf{p}$), and $D_\infty(\mathbf{q}\|\mathbf{p}) \coloneqq \log_2 \max_c \frac{q(c)}{p(c)}$. For $X, Y \in \mathrm{Pos}()$ the quantum relative entropy is $D(X\|Y) = \mathrm{tr}[X(\log_2 X - \log_2 Y)]$ when $\mathrm{supp}\, X \subseteq \mathrm{supp}\, Y$, else $+\infty$.

The Kullback–Leibler divergence of probability vectors is non-negative, $D(\mathbf{q}\|\mathbf{p}) \geq 0$, with equality if and only if $\mathbf{q} = \mathbf{p}$ (Gibbs' inequality). If $q(c) > 0 = p(c)$ for some c, then $D(\mathbf{q}\|\mathbf{p}) = +\infty$. Otherwise, with $S \coloneqq \{c : q(c) > 0\}$, the terms outside S vanish by the first convention and $\log_2$ is strictly concave, so Jensen's inequality with the weights $q(c)$, $c \in S$, which sum to one, gives $-D(\mathbf{q}\|\mathbf{p}) = \sum_{c \in S} q(c) \log_2 \frac{p(c)}{q(c)} \leq \log_2 \sum_{c \in S} p(c) \leq \log_2 1 = 0$. Equality in the first step forces $p(c)/q(c)$ to be one constant γ on S, and in the second $\sum_{c \in S} p(c) = 1$; then $\gamma = \gamma \sum_{c \in S} q(c) = \sum_{c \in S} p(c) = 1$, so $\mathbf{p} = \mathbf{q}$ on S, and $\mathbf{p}$ vanishes off S, its mass on S being one. Conversely $D(\mathbf{q}\|\mathbf{q}) = 0$ term by term.

Used by meat-core/cor:drop-program, meat-core/def:feasible, meat-core/fact:meat, meat-core/frank-wolfe/cor:fo-dropped, meat-core/frank-wolfe/lem:certified-bracket, meat-core/frank-wolfe/lem:fo-khyp, meat-core/lem:objective-homogeneous, meat-core/prop:drop-bound, meat-core/prop:minimal-divergence, meat-core/prop:model-pass-bound, meat-core/prop:optol-nobound, meat-core/prop:value-range, meat-core/thm:accumulated-entropy, meat-core/thm:oracle · newly derived · implemented · *Code* certiqs_key.foundations.kernel:divergence_kernel

The divergence kernel $d(u, v) = u \log_2(u/v)$, extended to vanishing arguments, is jointly convex, lower semicontinuous and non-increasing in v. These properties keep the objective well defined at support boundaries and let it be composed with concave branch quantities.

Lemma 1.4 (Extended divergence kernel). Define $d : [0, \infty) \times [0, \infty) \to (-\infty, +\infty]$ by $d(u, v) \coloneqq u \log_2 \frac{u}{v}$ for $u, v > 0$, $d(0, v) \coloneqq 0$ for $v \geq 0$, and $d(u, 0) \coloneqq +\infty$ for $u > 0$. Then d is jointly convex and lower semicontinuous, and non-increasing in v.

Used by meat-core/cor:drop-program, meat-core/prop:a-neutral, meat-core/prop:drop-bound, meat-core/prop:model-pass-bound, meat-core/thm:convexity, meat-core/thm:oracle · newly derived · implemented · *Code* certiqs_key.foundations.kernel:divergence_kernel · *Oracles* registry.py

Proof. d is the closed perspective of $g(t) = t \log_2 t$ (with $g(0) = 0$), i.e. $d(u, v) = v\, g(u/v)$ for $v > 0$. *Convexity:* let (u_i, v_i), $i = 1, 2$, and $\lambda \in (0, 1)$,

and write (u_λ, v_λ) for the convex combination. If $v_1, v_2 > 0$, then with weights $\mu_i := \lambda_i v_i / v_\lambda$ (where $\lambda_1 = \lambda$, $\lambda_2 = 1-\lambda$; $\mu_1 + \mu_2 = 1$),

$$d(u_\lambda, v_\lambda) = v_\lambda\, g\Big(\mu_1 \tfrac{u_1}{v_1} + \mu_2 \tfrac{u_2}{v_2}\Big) \le v_\lambda \Big[\mu_1 g\big(\tfrac{u_1}{v_1}\big) + \mu_2 g\big(\tfrac{u_2}{v_2}\big)\Big] = \lambda d(u_1, v_1) + (1-\lambda) d(u_2, v_2)$$

by convexity of g. If, say, $v_2 = 0$ and $u_2 > 0$, the right-hand side is $+\infty$ and the inequality is trivial; if $v_2 = 0$ and $u_2 = 0$, then $(u_\lambda, v_\lambda) = \lambda(u_1, v_1)$ and $d(\lambda u_1, \lambda v_1) = \lambda d(u_1, v_1)$, again consistent. *Monotonicity in v:* for $u > 0$, $v \mapsto u \log_2(u/v)$ is decreasing on $(0, \infty)$ and jumps to $+\infty$ at $v = 0$; for $u = 0$ it is constant. *Lower semicontinuity:* at $(u, 0)$ with $u > 0$, any sequence $(u_n, v_n) \to (u, 0)$ with $v_n > 0$ has $u_n \log_2(u_n/v_n) \to +\infty$; at $(0, v)$ with $v > 0$, $|u_n \log_2(u_n/v_n)| \le u_n |\log_2 u_n| + u_n |\log_2 v_n| \to 0$; at $(0, 0)$, for $v_n \le \frac{1}{2}$ eventually, $u_n \log_2(u_n/v_n) \ge u_n \log_2 u_n \to 0$, so $\liminf \ge 0 = d(0, 0)$. □

For $\alpha > 1$, pinching the α-th root of a positive semidefinite operator and raising the result to the power α never increases the trace, and leaves it unchanged exactly when the operator is block diagonal for the pinching. This bounds the trace-power terms of the MEAT objective.

Lemma 1.5 (Pinching lowers the trace power). Let $\tau \in \mathrm{Pos}(Q)$, let $\mathcal{Z}$ be a pinching with projectors $\{Z_j\}_{j=1}^m$, and let $\alpha \in (1, \infty)$. Then

$$\mathrm{tr}\Big[\Big(\mathcal{Z}\big(\tau^{1/\alpha}\big)\Big)^\alpha\Big] \;\le\; \mathrm{tr}[\tau], \tag{1.5}$$

with equality if and only if $\mathcal{Z}(\tau^{1/\alpha}) = \tau^{1/\alpha}$, equivalently iff τ is block-diagonal for $\{Z_j\}$, equivalently iff $\mathcal{Z}(\tau) = \tau$.

Uses Def. 1.1 · *Used by* meat-core/cor:f-nonpos, meat-core/frank-wolfe/prop:relaxed-set-properties, meat-core/thm:branch-formula · newly derived · implemented · *Code* certiqs_key.meat_core.objective.branch: trace_power_pinched · *Oracles* registry.py

Proof. Step 1 (pinching as a unitary mixture). With $\zeta := e^{2\pi i/m}$ and the unitaries $U_k := \sum_{j=1}^m \zeta^{jk} Z_j$, $k \in \{0, \dots, m-1\}$, (1.1) of Definition 1.1 gives $\mathcal{Z}(X) = \frac{1}{m} \sum_{k=0}^{m-1} U_k X U_k^\dagger$ for every X.

Step 2 (strict convexity of $A \mapsto \mathrm{tr}[A^\alpha]$ on $\mathrm{Pos}(Q)$). Two standard facts, proved inline for self-containment. (a) *Peierls inequality with equality condition:* for convex $f : [0, \infty) \to \mathbb{R}$, any ONB $\{|i\rangle\}$ and $A \in \mathrm{Pos}(Q)$ with spectral decomposition $A = \sum_j \lambda_j |a_j\rangle\langle a_j|$,

$$\langle i|\, f(A)\, |i\rangle = \sum_j |\langle i|a_j\rangle|^2 f(\lambda_j) \;\ge\; f\Big(\sum_j |\langle i|a_j\rangle|^2 \lambda_j\Big) = f\big(\langle i|\, A\, |i\rangle\big)$$

by Jensen; summing over i gives $\mathrm{tr}\, f(A) \ge \sum_i f(\langle i|\, A\, |i\rangle)$. If f is strictly convex, equality for index i forces the distribution $|\langle i|a_j\rangle|^2$ to be supported on a single eigenvalue of A, i.e. $|i\rangle$ lies in one eigenspace of A; equality for all i forces $\{|i\rangle\}$ to be an eigenbasis of A. (b) *Strict convexity of the*

trace functional: let $f(t) = t^\alpha$, strictly convex on $[0,\infty)$ for $\alpha > 1$. For $A, B \in \mathrm{Pos}(Q)$ let $C = \frac{1}{2}(A+B)$ with eigenbasis $\{|i\rangle\}$. Then

$$\operatorname{tr} f(C) = \sum_i f\big(\langle i|\, C\, |i\rangle\big) = \sum_i f\Big(\tfrac{\langle i|A|i\rangle + \langle i|B|i\rangle}{2}\Big) \le \sum_i \tfrac{f(\langle i|A|i\rangle) + f(\langle i|B|i\rangle)}{2} \le \tfrac{\operatorname{tr} f(A) + \operatorname{tr} f(B)}{2},$$

using scalar convexity and then Peierls for A and B. By continuity, midpoint convexity gives convexity. If equality holds throughout, then (by strict scalar convexity) $\langle i|\, A\, |i\rangle = \langle i|\, B\, |i\rangle$ for all i, and (by the Peierls equality condition) $\{|i\rangle\}$ is an eigenbasis of both A and B; hence A and B are diagonal in a common basis with equal diagonals, so $A = B$. Thus $A \mapsto \operatorname{tr}[A^\alpha]$ is strictly convex.

Step 3 (conclusion). Set $X := \tau^{1/\alpha}$. By Steps 1–2 and unitary invariance of the trace functional,

$$\operatorname{tr}\big[\mathcal{Z}(X)^\alpha\big] = \operatorname{tr}\Big[\Big(\tfrac{1}{m}\sum_k U_k X U_k^\dagger\Big)^\alpha\Big] \le \frac{1}{m}\sum_k \operatorname{tr}\big[(U_k X U_k^\dagger)^\alpha\big] = \operatorname{tr}[X^\alpha] = \operatorname{tr}[\tau].$$

For the equality case: a strictly convex functional satisfies the m-point Jensen inequality with equality only if all arguments coincide (if two differed, replacing them by their midpoint would strictly decrease the right side while fixing the left, a contradiction). Hence equality in (1.5) forces $U_k X U_k^\dagger = U_0 X U_0^\dagger = X$ for all k, so $X = \frac{1}{m}\sum_k U_k X U_k^\dagger = \mathcal{Z}(X)$, i.e. $\tau^{1/\alpha}$ is block-diagonal. Since $\tau = (\tau^{1/\alpha})^\alpha$ and functional calculus preserves block-diagonality in both directions, this is equivalent to $\mathcal{Z}(\tau) = \tau$. Conversely, if $\mathcal{Z}(\tau^{1/\alpha}) = \tau^{1/\alpha}$ then (1.5) holds with equality by direct substitution. □

For a purified input and a projective key measurement, the fixed-marginal conditional Rényi entropy equals a trace-power expression involving only the measured system's marginal. This removes the explicit purifier from the computable branch objective.

Fact 1.6 (Pinched trace-power form of the conditional Rényi entropy)**.** Let $\rho_{QE} \in S_=(QE)$ be pure, let $\{Z_j\}_{j=1}^d$ be orthogonal projectors on Q with $\sum_j Z_j = \mathbb{1}_Q$, let $V_Z = \sum_j |j\rangle_S \otimes Z_j$ and $\sigma_{SQE} := V_Z\, \rho_{QE}\, V_Z^\dagger$, and let $\mathcal{Z}(\tau) := \sum_j Z_j \tau Z_j$. Then, for $\alpha \in (1,\infty)$,

$$\widetilde{H}_\alpha^\downarrow(S|E)_{\sigma_{SE}} = \frac{1}{1-\alpha}\log_2 \operatorname{tr}\Big[\Big(\mathcal{Z}\big(\rho_Q^{1/\alpha}\big)\Big)^\alpha\Big]. \tag{1.6}$$

Locator note. The source states no α restriction in the theorem; the domain $(1,\infty)$ used here lies inside the domain $(0,1)\cup(1,\infty)$ of its entropy definitions and is the only range any claim that uses this fact invokes.

Uses Def. 1.1, Def. 1.2 · *Used by* meat-core/thm:branch-formula · published in [1, Thm. 8, Eq. (59)] · accepted · *Oracles* registry.py

The pinched trace-power functional is concave on subnormalised states for every parameter in [0, 1]. This is the analytic ingredient behind the convexity of the engine's objective.

Fact 1.7 (Concavity of the pinched trace-power functional)**.** Let $\mathcal{Z}$ be a pinching built from a complete family of mutually orthogonal projectors (Def. 22 of the source) and for $\alpha' \in [0,2]$ let $Q_{\alpha'}(\rho) := \operatorname{tr}\left[(\mathcal{Z}(\rho^{\alpha'}))^{1/\alpha'}\right]$ (Eq. (F6) of the source). Then $\rho \mapsto Q_{\alpha'}(\rho)$ is concave on $S_{\leq}(Q)$ for $\alpha' \in [0,1]$.

Used by meat-core/cor:concavity, meat-core/lem:cone-concavity, meat-core/thm:gradient · published in [1, App. F.1, Thm. 24] · accepted · *Oracles* registry.py

2 The round model

A run of n rounds is attacked by a sequence of channels, one per round, each delivering the per-round system, with Alice's system too where the adversary supplies it, and carrying Eve's memory forward. A list of operator constraints restricts each round's channel by adjoint conditions, which cut out an attack set, with a stronger condition for an entry scoped to one factor of a product system. A register of any dimension, such as an adversary's memory, is approximated by compressing it onto a finite-dimensional subspace, at a cost in trace norm that the state's weight outside the subspace bounds and that adds along a sequence of channels (Lemma 2.3). The round model is stated once for a generic per-round system, and the modules that analyse a protocol instantiate it at their own systems. A family of objects describing a round is round-homogeneous when the same objects describe every round and every round behaves as they describe whatever the other rounds do, so no device has memory across rounds; the modules' round assumptions assert it, or its sameness clause alone where they say so, of their own families.

Over n rounds of a per-round system S, an attack sequence is one channel per round from Eve's register to S and her new memory, with Alice's system too when the adversary supplies it. A list of operator constraints cuts out an attack set by adjoint conditions, and so the admissible attack sequences.

Definition 2.1 (Round model, attack sequences and attack sets)**.** Fix a number $n \geq 1$ of rounds, a per-round system S, Alice's system A, and the instantiating model's source mode: in trusted-source mode Alice supplies A in every round, and in untrusted-source mode the adversary does. Systems, operators and channels follow the conventions of Definition 1.1, except that S may be separable of infinite dimension where the instantiating model declares it so; a channel whose input or output includes such a system is a normal CPTP map on trace-class operators. The dimensions are the

instantiating model's to declare. We write S_j and A_j for the copies of S and A in round j, and drop the round index where one round is in view. This is the *round model* with per-round system S.

(i) An *attack sequence* is a family $(\mathcal{A}_j)_{j=1}^{n}$ of channels whose output carries the systems the adversary supplies in the source mode:

$$\mathcal{A}_j \in \begin{cases} \mathrm{CPTP}\big(E_{j-1},\ S_j\,E'_j\big), & \text{trusted-source mode,} \\ \mathrm{CPTP}\big(E_{j-1},\ A_j\,S_j\,E'_j\big), & \text{untrusted-source mode,} \end{cases} \tag{2.1}$$

where E_0 is Eve's initial register, E'_j her outgoing memory, and E_j her register after round j, which holds E'_j together with what the instantiating model hands her in round j. In trusted-source mode A_j is *not* an output of $\mathcal{A}_j$: it is supplied by Alice and carried into the round as a separate input.

(ii) Let S be finite-dimensional, and let $\mathfrak{L} = \{(\hat{O}_k, \bowtie_k, c_k)\}_{k=1}^{m}$, $m \geq 0$, be a finite list of Hermitian operators $\hat{O}_k$ on S, relations $\bowtie_k \in \{\preceq, =, \succeq\}$ between operators, and real numbers c_k. For registers E', E'', the *attack set* of $\mathfrak{L}$ is

$$\mathfrak{A}_{\mathfrak{L}}(E',\, SE'') \coloneqq \Big\{\mathcal{A} \in \mathrm{CPTP}(E', SE'') :\ \mathcal{A}^{\dagger}\big(\hat{O}_k \otimes \mathbb{1}_{E''}\big) \bowtie_k c_k\, \mathbb{1}_{E'}\ \ \forall k\Big\} \qquad \text{(trusted-source mode} \tag{2.2}$$

and, with the constraint operator lifted to the larger supplied system,

$$\mathfrak{A}_{\mathfrak{L}}(E',\, ASE'') \coloneqq \Big\{\mathcal{A} \in \mathrm{CPTP}(E', ASE'') :\ \mathcal{A}^{\dagger}\big(\mathbb{1}_A \otimes \hat{O}_k \otimes \mathbb{1}_{E''}\big) \bowtie_k c_k\, \mathbb{1}_{E'}\ \ \forall k\Big\} \tag{2.3}$$

in untrusted-source mode; the two are distinguished by the output system named in the argument. For $\mathfrak{L} = \varnothing$ the condition is empty and $\mathfrak{A}_{\varnothing} = \mathrm{CPTP}$.

(iii) Where S is finite-dimensional and $S = \bigotimes_{f \in \mathbb{F}} S_f$ is a product of factors indexed by a finite ordered set $\mathbb{F}$ (with a round index as well, a factor is written $S_{f,j}$), an entry of $\mathfrak{L}$ may be *tagged* with a factor f; its operator $\hat{O}_k$ is then Hermitian on S_f, and the entry $(\hat{O}_k, \bowtie_k, c_k; f)$ contributes to the attack set, in place of the condition of (2.2) or (2.3), the condition

$$\big(\mathrm{id}_{R'} \otimes \mathcal{A}\big)^{\dagger}\Big(X \otimes \big(\hat{O}_k - c_k\, \mathbb{1}_{S_f}\big) \otimes \mathbb{1}_{E''}\Big)\ \bowtie_k\ 0 \qquad \text{for every finite-dimensional } R' \text{ and every } X \in \mathrm{P} \tag{2.4}$$

as an operator on $R'E'$, where S_{f^c} is the product of the factors other than f, the tensor factors of the argument are written in the order $R'R\,S_{f^c}$, S_f, E'', and R is A in untrusted-source mode and trivial in trusted-source mode. Untagged entries keep their conditions. The reference system R' is what carries Alice's retained system in trusted-source mode, where A is no output of $\mathcal{A}$. With R' trivial and $X = \mathbb{1}$,

(2.4) is the condition of (2.2) or (2.3) for the lifted operator $\mathbb{1}_{S_{fc}} \otimes \hat{O}_k$, because $\mathcal{A}^\dagger$ is unital; so the attack set with tagged entries is contained in the attack set with the same entries untagged and their operators lifted.

(iv) An attack sequence is an *admissible attack sequence* for $\mathfrak{L}$, or $\mathfrak{L}$-*admissible*, if $\mathcal{A}_j$ lies in the attack set of $\mathfrak{L}$ of the source mode, (2.2) or (2.3) with the conditions of (2.4) for tagged entries, with $E' = E_{j-1}$ and $E'' = E'_j$, for every j.

Uses Def. 1.1 · *Used by* Def. 2.2, meat-core/ass:eve-dim, meat-core/def:roundmodel, reductions/def:attset, reductions/def:roundmodel · newly derived · accepted

A per-round family is round-homogeneous when one fixed list of objects describes every round (sameness) and every round behaves as they describe whatever the other rounds do (no memory across rounds). An assumption may assert sameness alone, and must then say so.

Definition 2.2 (Round homogeneity of a per-round family)**.** Consider the n rounds of a round model (Definition 2.1), and a *per-round family*: a finite list of objects that a model of the run declares for a round, such as operators, channels or a declaration. The family is *round-homogeneous* if both of the following hold.

(i) *Sameness.* One list of objects, fixed before the run, describes every round: for every $j \in \{1, \ldots, n\}$ round j is described by the same objects. No object of the family depends on the round index or on the run.

(ii) *No memory.* Every round behaves physically as the objects describe it, whatever the inputs and the outputs of the other rounds are. This excludes memory across rounds, such as detector memory, afterpulsing or correlations between a source's emissions in different rounds. It is a condition on the rounds, not on the objects, and does not follow from (i).

An assumption may assert (i) alone of its family, and must then say so; an assumption that calls its family round-homogeneous without saying so asserts (i) and (ii). Rounds are not averaged: a family that describes the rounds only on average over the rounds, or up to a per-round variation, does not satisfy (i), and per-round variation is not modelled. Round homogeneity constrains the family and the rounds it describes, not the adversary: an attack sequence of the round model remains arbitrary and may correlate the rounds.

Uses Def. 2.1 · *Used by* protocol-compiler/ass:rhom, receiver/ass:round, reductions/ass:round · newly derived · accepted

Compressing one register of a state onto a finite-dimensional subspace moves the state by at most $2\sqrt{\delta} + \delta$ in trace norm, δ being its weight outside the subspace, whatever the other systems are. The moves add along a sequence of channels, and every channel extends by an identity of any dimension.

A register of any dimension is approximated by a finite one by compressing it: the part of a state on a finite-dimensional subspace is kept, and the weight outside it is moved onto a fixed state of the subspace. The lemma bounds what that costs in trace norm, with no condition on the other systems, and adds the costs along a sequence of channels; the compressed register can then be read as a finite-dimensional one, and channels acting on the other systems do not see the difference. Lemma "Approximating an attack by all-at-once attacks with finite memory" (meat-core/lem:memory-lift) applies it to the adversary's registers of a run. The route is that of [4, Def. A.1, Lemma A.1, Eqs. (A2)–(A5)], re-derived here with an explicit bound.

Lemma 2.3 (Compressing a register onto a finite-dimensional subspace)**.** *Setting.* In this lemma a Hilbert space may have any dimension, finite or infinite, separable or not, in departure from the convention of Definition 1.1, and every system named is of this kind. $\mathcal{T}(Z)$ is the space of trace-class operators on $\mathcal{H}(Z)$ with the trace norm $\|\cdot\|_1$, and RM is the Hilbert-space tensor product of R and M. A *channel* $\mathcal{T}(Z) \to \mathcal{T}(Z')$ is a trace-preserving linear map Φ that is completely positive: for every $k \geq 1$ the map that applies Φ to every entry of a $k \times k$ matrix of trace-class operators, which is $\mathcal{T}(\mathbb{C}^k Z)$, is positive. $\mathrm{tr}_R \colon \mathcal{T}(RM) \to \mathcal{T}(M)$ is the partial trace, fixed by $\mathrm{tr}[X \,\mathrm{tr}_R(\sigma)] = \mathrm{tr}[(\mathbb{1}_R \otimes X)\sigma]$ for every bounded operator X on $\mathcal{H}(M)$, and tr_M likewise.

(i) *(compression and inclusion)* Let M° be a non-zero finite-dimensional subspace of $\mathcal{H}(M)$, V its inclusion isometry into $\mathcal{H}(M)$, $\Pi \coloneqq VV^\dagger$ the orthogonal projection onto it, $\Pi^\perp \coloneqq \mathbb{1}_M - \Pi$, and ψ a state on M°. For every system R put, for $\sigma \in \mathcal{T}(RM)$ and $\tau \in \mathcal{T}(RM^\circ)$,

$$\mathcal{P}_R(\sigma) \coloneqq (\mathbb{1}_R \otimes V)^\dagger \sigma (\mathbb{1}_R \otimes V) + \mathrm{tr}_M\left[(\mathbb{1}_R \otimes \Pi^\perp)\,\sigma\,(\mathbb{1}_R \otimes \Pi^\perp)\right] \otimes \psi,$$
$$\iota_R(\tau) \coloneqq (\mathbb{1}_R \otimes V)\,\tau\,(\mathbb{1}_R \otimes V)^\dagger, \tag{2.5}$$

and write $\mathcal{P}$ and ι for the case of a one-dimensional R. Then $\mathcal{P}_R \colon \mathcal{T}(RM) \to \mathcal{T}(RM^\circ)$ and $\iota_R \colon \mathcal{T}(RM^\circ) \to \mathcal{T}(RM)$ are channels, $\mathcal{P}_R \circ \iota_R$ is the identity, $\|\iota_R(\tau)\|_1 = \|\tau\|_1$ for every τ, and $\mathcal{P}_R(x \otimes y) = x \otimes \mathcal{P}(y)$ and $\iota_R(x \otimes z) = x \otimes \iota(z)$ for $x \in \mathcal{T}(R)$, $y \in \mathcal{T}(M)$ and $z \in \mathcal{T}(M^\circ)$; more generally, for $R = R_1R_2$, $\mathcal{P}_{R_1R_2}(x \otimes y) = x \otimes \mathcal{P}_{R_2}(y)$ and $\iota_{R_1R_2}(x \otimes z) = x \otimes \iota_{R_2}(z)$ for $x \in \mathcal{T}(R_1)$, $y \in \mathcal{T}(R_2M)$ and $z \in \mathcal{T}(R_2M^\circ)$. The channel $\mathcal{C}_R \coloneqq \iota_R \circ \mathcal{P}_R$ on $\mathcal{T}(RM)$ is the *compression of M onto M°*.

(ii) *(one compression)* For every state σ on RM, with $\delta \coloneqq \mathrm{tr}\left[(\mathbb{1}_R \otimes \Pi^\perp)\sigma\right]$,

$$\left\|\mathcal{C}_R(\sigma) - \sigma\right\|_1 \;\leq\; 2\sqrt{\delta} + \delta. \tag{2.6}$$

For every state σ on RM and every $\eta > 0$ there is a non-zero finite-dimensional subspace M°, spanned by finitely many eigenvectors of $\mathrm{tr}_R(\sigma)$, at which $\delta \le \eta$.

(iii) *(a sequence of channels)* Let $Z_0, \dots, Z_m$ be systems, ω a state on Z_0, $\Phi_k \colon \mathcal{T}(Z_{k-1}) \to \mathcal{T}(Z_k)$ channels for $k = 1, \dots, m$, and $\mathcal{C}_k$ a channel on $\mathcal{T}(Z_k)$ for $k = 0, \dots, m$. Put $\sigma_0 \coloneqq \omega$ and $\sigma_k \coloneqq \Phi_k(\sigma_{k-1})$. Then

$$\big\|\mathcal{C}_m \Phi_m \mathcal{C}_{m-1} \cdots \Phi_1 \mathcal{C}_0(\omega) - \sigma_m\big\|_1 \;\le\; \sum_{k=0}^{m} \big\|\mathcal{C}_k(\sigma_k) - \sigma_k\big\|_1. \tag{2.7}$$

(iv) *(maps on different factors)* Let $G \colon \mathcal{T}(R) \to \mathcal{T}(R')$ and $H \colon \mathcal{T}(M) \to \mathcal{T}(M')$ be linear maps, and let $G_M \colon \mathcal{T}(RM) \to \mathcal{T}(R'M)$, $G_{M'} \colon \mathcal{T}(RM') \to \mathcal{T}(R'M')$, $H_R \colon \mathcal{T}(RM) \to \mathcal{T}(RM')$ and $H_{R'} \colon \mathcal{T}(R'M) \to \mathcal{T}(R'M')$ be linear maps, bounded in trace norm, with $G_M(x \otimes y) = G(x) \otimes y$, $G_{M'}(x \otimes y') = G(x) \otimes y'$, $H_R(x \otimes y) = x \otimes H(y)$ and $H_{R'}(x' \otimes y) = x' \otimes H(y)$ for all $x \in \mathcal{T}(R)$, $x' \in \mathcal{T}(R')$, $y \in \mathcal{T}(M)$, $y' \in \mathcal{T}(M')$. Then $H_{R'} \circ G_M = G_{M'} \circ H_R$. In particular, with $H = \mathcal{P}$ or $H = \iota$, whose extensions are $\mathcal{P}_R$, $\mathcal{P}_{R'}$, ι_R and $\iota_{R'}$ by (i),

$$\mathcal{P}_{R'} \circ G_M = G_{M^\circ} \circ \mathcal{P}_R, \qquad G_M \circ \iota_R = \iota_{R'} \circ G_{M^\circ}. \tag{2.8}$$

(v) *(extension by an identity)* For every channel $\Phi \colon \mathcal{T}(Z) \to \mathcal{T}(Z')$ and every system K there is exactly one linear map $\Phi_K \colon \mathcal{T}(ZK) \to \mathcal{T}(Z'K)$, bounded in trace norm, with $\Phi_K(x \otimes y) = \Phi(x) \otimes y$ for all $x \in \mathcal{T}(Z)$ and $y \in \mathcal{T}(K)$; it is a channel, written $\Phi \otimes \mathrm{id}_K$.

Used by meat-core/lem:honest-attacks, meat-core/lem:memory-lift · newly derived · accepted

Proof. Idea. Each map of (i) is built from conjugations, a partial trace and the appending of a state. For (ii), the projected state differs from the state by terms that the weight outside the subspace controls, vector by vector in a spectral decomposition. Part (iii) is a telescoping sum, every channel contracting the trace norm of a Hermitian operator, and (iv) holds on product operators, which span a dense subspace. Part (v) builds the extension from finite-dimensional subspaces of K, as the limit along which part (ii) lets the state be compressed.

Step 1 (two facts on channels). A positive trace-preserving linear map Φ satisfies $\|\Phi(Z)\|_1 \le \|Z\|_1$ for every Hermitian trace-class Z, and a positive linear map Φ with $\mathrm{tr}\,\Phi(Z) \le \mathrm{tr}\,Z$ for $Z \succeq 0$ does too: write $Z = Z^+ - Z^-$ by the spectral decomposition, with positive $Z^\pm$ and $\|Z\|_1 = \mathrm{tr}\,Z^+ + \mathrm{tr}\,Z^-$; then $\|\Phi(Z)\|_1 \le \|\Phi(Z^+)\|_1 + \|\Phi(Z^-)\|_1 = \mathrm{tr}\,\Phi(Z^+) + \mathrm{tr}\,\Phi(Z^-) \le \|Z\|_1$, a positive trace-class operator having trace norm equal to its trace. Such a map is bounded in trace norm: every τ is $H_1 + iH_2$ with the Hermitian

$H_1 = \frac{1}{2}(\tau + \tau^\dagger)$ and $H_2 = \frac{1}{2i}(\tau - \tau^\dagger)$, each of trace norm at most $\|\tau\|_1$, so $\|\Phi(\tau)\|_1 \le 2\|\tau\|_1$.

Step 2 (part (i)). A conjugation $\sigma \mapsto B\sigma B^\dagger$ by a bounded operator is completely positive: applied entrywise to a $k \times k$ matrix it is the conjugation by $\mathbb{1}_{\mathbb{C}^k} \otimes B$. The partial trace is completely positive, applied entrywise being the partial trace on $\mathbb{C}^k RM$; appending the state ψ, $Y \mapsto Y \otimes \psi$, is completely positive; and sums and compositions of completely positive maps are. So $\mathcal{P}_R$ and ι_R are completely positive. Trace: for trace-class σ and bounded B, $\operatorname{tr}\big[B^\dagger \sigma B\big] = \operatorname{tr}\big[\sigma B B^\dagger\big]$, so $\operatorname{tr}\mathcal{P}_R(\sigma) = \operatorname{tr}[(\mathbb{1}_R \otimes \Pi)\sigma] + \operatorname{tr}\big[(\mathbb{1}_R \otimes \Pi^\perp)\sigma\big] = \operatorname{tr}\sigma$, and $\operatorname{tr}\iota_R(\tau) = \operatorname{tr}\big[\tau(\mathbb{1}_R \otimes V^\dagger V)\big] = \operatorname{tr}\tau$ since V is an isometry. $\mathcal{P}_R \circ \iota_R$ is the identity because $(\mathbb{1}_R \otimes V)^\dagger(\mathbb{1}_R \otimes V) = \mathbb{1}_{RM}$ and $\Pi^\perp V = 0$, which makes the second term of $\mathcal{P}_R$ vanish on $\iota_R(\tau)$. For an isometry W, $|W\tau W^\dagger| = W|\tau|W^\dagger$, since $W|\tau|W^\dagger$ is positive with square $W\tau^\dagger\tau W^\dagger = (W\tau W^\dagger)^\dagger(W\tau W^\dagger)$; taking traces, $\|W\tau W^\dagger\|_1 = \|\tau\|_1$, which is the claim for $W = \mathbb{1}_R \otimes V$. On products, $(\mathbb{1}_R \otimes V)^\dagger(x \otimes y)(\mathbb{1}_R \otimes V) = x \otimes V^\dagger y V$ and $\operatorname{tr}_M[(\mathbb{1}_R \otimes \Pi^\perp)(x \otimes y)(\mathbb{1}_R \otimes \Pi^\perp)] = \operatorname{tr}\big[\Pi^\perp y \Pi^\perp\big]\, x$, which together give $x \otimes \mathcal{P}(y)$; the product form of ι_R is the same computation. For $R = R_1R_2$ and $x \in \mathcal{T}(R_1)$, $y \in \mathcal{T}(R_2M)$, the operators $\mathbb{1}_{R_1R_2} \otimes V$ and $\mathbb{1}_{R_1R_2} \otimes \Pi^\perp$ are $\mathbb{1}_{R_1} \otimes (\mathbb{1}_{R_2} \otimes V)$ and $\mathbb{1}_{R_1} \otimes (\mathbb{1}_{R_2} \otimes \Pi^\perp)$, so the conjugations act on $x \otimes y$ as x tensored with the conjugations of y, and the partial trace over M of $x \otimes Y$ is $x \otimes \operatorname{tr}_M Y$ for Y on R_2M, by its defining property tested on products; together they give $x \otimes \mathcal{P}_{R_2}(y)$, and likewise for ι.

Step 3 (part (ii)). Write $P := \mathbb{1}_R \otimes \Pi$ and $Q := \mathbb{1}_R \otimes \Pi^\perp$, so that $\delta = \operatorname{tr}[Q\sigma]$ and, by (2.5),

$$\mathcal{C}_R(\sigma) - \sigma = \big(P\sigma P - \sigma\big) + \operatorname{tr}_M\big[Q\sigma Q\big] \otimes V\psi V^\dagger.$$

The second term is positive, being the partial trace of the positive $Q\sigma Q$ tensored with a state, so its trace norm is its trace, $\operatorname{tr}[Q\sigma Q] = \operatorname{tr}[Q\sigma] = \delta$. For the first, let $\sigma = \sum_l \mu_l\, |w_l\rangle\langle w_l|$ be a spectral decomposition, with $\mu_l \ge 0$, $\sum_l \mu_l = 1$ and orthonormal w_l, a series converging in trace norm; the conjugation by P is bounded in trace norm, so $P\sigma P - \sigma = \sum_l \mu_l \big(\, |Pw_l\rangle\langle Pw_l| - |w_l\rangle\langle w_l|\,\big)$. Since $w = Pw + Qw$, $|Pw\rangle\langle Pw| - |w\rangle\langle w| = -\,|Qw\rangle\langle w| - |Pw\rangle\langle Qw|$, and a rank-one operator $|a\rangle\langle b|$ has trace norm $\|a\|\,\|b\|$, so for a unit vector w the trace norm of this difference is at most $2\|Qw\|$. Hence, by the Cauchy–Schwarz inequality for the weights μ_l,

$$\|P\sigma P - \sigma\|_1 \le 2\sum_l \mu_l \|Qw_l\| \le 2\Big(\sum_l \mu_l \|Qw_l\|^2\Big)^{1/2} = 2\Big(\sum_l \mu_l\, \langle w_l|\, Q\, |w_l\rangle\Big)^{1/2} = 2\sqrt{\operatorname{tr}[Q\sigma]} = 2\sqrt{\delta},$$

using $\|Qw\|^2 = \langle w|\, Q\, |w\rangle$ for the projection Q. The triangle inequality gives (2.6). For the second sentence, the partial trace $\sigma_M := \operatorname{tr}_R(\sigma)$ is a positive trace-class operator of trace one, with a spectral decomposition $\sigma_M = \sum_l \lambda_l\, |e_l\rangle\langle e_l|$, finite or countable, with $\lambda_l > 0$, $\sum_l \lambda_l = 1$ and orthonormal e_l, and it has at least one term. By the defining property of the partial

trace, $\delta = \operatorname{tr}\big[(\mathbb{1}_R \otimes \Pi^\perp)\sigma\big] = \operatorname{tr}\big[\Pi^\perp \sigma_M\big]$. At $M^\circ = \operatorname{span}\{e_1, \dots, e_d\}$, with d at most the number of terms, $\Pi^\perp e_l = 0$ for $l \le d$ and $\Pi^\perp e_l = e_l$ for $l > d$, so $\delta = \sum_{l>d} \lambda_l$, the tail of a convergent series of positive terms; it is at most η for d large enough, and zero once d reaches a finite number of terms.

Step 4 (part (iii)). For $k = 0, \dots, m$ let $\mathcal{W}_k \coloneqq \mathcal{C}_m \Phi_m \cdots \mathcal{C}_{k+1}\Phi_{k+1}$, the identity for $k = m$, and put $\chi_k \coloneqq \mathcal{W}_k \mathcal{C}_k(\sigma_k)$ and $\chi_{m+1} \coloneqq \sigma_m$. Then χ_0 is the first state in (2.7), and for $k < m$, $\chi_{k+1} = \mathcal{W}_{k+1}\mathcal{C}_{k+1}\Phi_{k+1}(\sigma_k) = \mathcal{W}_k(\sigma_k)$, so that $\chi_k - \chi_{k+1} = \mathcal{W}_k\big(\mathcal{C}_k(\sigma_k) - \sigma_k\big)$ for every $k \le m$, the case $k = m$ reading $\mathcal{C}_m(\sigma_m) - \sigma_m$. Each $\mathcal{W}_k$ is a channel and $\mathcal{C}_k(\sigma_k) - \sigma_k$ is Hermitian, so by Step 1 its image has trace norm at most $\|\mathcal{C}_k(\sigma_k) - \sigma_k\|_1$. Summing $\chi_0 - \chi_{m+1} = \sum_{k=0}^{m}(\chi_k - \chi_{k+1})$ and applying the triangle inequality gives (2.7).

Step 5 (part (iv)). The two composites $H_{R'} \circ G_M$ and $G_{M'} \circ H_R$ are linear and bounded in trace norm, as compositions of bounded maps, a channel being bounded by Step 1. On a product $x \otimes y$, $H_{R'}G_M(x \otimes y) = H_{R'}(G(x) \otimes y) = G(x) \otimes H(y) = G_{M'}(x \otimes H(y)) = G_{M'}H_R(x \otimes y)$. Products span a trace-norm dense subspace of $\mathcal{T}(RM)$: finite-rank operators are dense, by truncating the singular value decomposition $\tau = \sum_l s_l\, |u_l\rangle\langle v_l|$, whose tail beyond L terms has trace norm at most $\sum_{l>L} s_l$, with $\sum_l s_l = \|\tau\|_1$; a rank-one $|a\rangle\langle b|$ is the trace-norm limit of $|a_t\rangle\langle b_t|$ for finite sums $a_t \to a$ and $b_t \to b$ of product vectors, which are dense in the tensor product, since $\| \, |a\rangle\langle b| - |a_t\rangle\langle b_t| \, \|_1 \le \|a - a_t\| \, \|b\| + \|a_t\| \, \|b - b_t\|$; and $|\sum_s \alpha_s \otimes \beta_s\rangle\langle\sum_t \gamma_t \otimes \delta_t| = \sum_{s,t} |\alpha_s\rangle\langle\gamma_t| \otimes |\beta_s\rangle\langle\delta_t|$ is a finite sum of products. Two bounded linear maps that agree on a set spanning a dense subspace agree everywhere. For $H = \mathcal{P}$ and $H = \iota$ the maps $\mathcal{P}_R$, $\mathcal{P}_{R'}$, ι_R and $\iota_{R'}$ are channels with the product forms of (i), so (2.8) is the general identity at these maps.

Step 6 (part (v)). For a finite-dimensional subspace K_0 of $\mathcal{H}(K)$, with inclusion isometry W_0 and $P_0 \coloneqq \mathbb{1}_Z \otimes W_0 W_0^\dagger$, let $\Phi \otimes \mathrm{id}_{K_0}$ be the map that applies Φ to every entry of an operator on ZK_0 written as a matrix in a basis of K_0; it is a channel by complete positivity, and it acts on products as $\Phi(x) \otimes y$, which determines it, since products span $\mathcal{T}(ZK_0)$. Put $F_0(\tau) \coloneqq (\mathbb{1}_{Z'} \otimes W_0)\,(\Phi \otimes \mathrm{id}_{K_0})\big((\mathbb{1}_Z \otimes W_0)^\dagger \tau (\mathbb{1}_Z \otimes W_0)\big)\,(\mathbb{1}_{Z'} \otimes W_0)^\dagger$, a composite of channels and a conjugation, so $\|F_0(\tau)\|_1 \le 2\|\tau\|_1$ by Step 1. For $K_0 \subseteq K_1$, writing the matrices in a basis of K_1 that extends one of K_0 shows $F_0(\tau) = F_1(P_0 \tau P_0)$, and $F_1(\tau) = F_1(P_1 \tau P_1)$, so

$$\|F_1(\tau) - F_0(\tau)\|_1 \le 2\|P_1\tau P_1 - P_0 \tau P_0\|_1 \le 2\|P_1\tau P_1 - \tau\|_1 + 2\|\tau - P_0\tau P_0\|_1.$$

For a positive τ, the estimate of Step 3, read for $\tau / \operatorname{tr}\tau$, gives $\|\tau - P_0\tau P_0\|_1 \le 2\big(\operatorname{tr}\tau \cdot \operatorname{tr}\Big[(\mathbb{1}_Z \otimes (\mathbb{1}_K - W_0 W_0^\dagger))\tau\Big]\big)^{1/2}$, and the last trace is $\operatorname{tr}\Big[(\mathbb{1}_K - W_0 W_0^\dagger)\operatorname{tr}_Z \tau\Big]$, which does not increase as K_0 grows and is below any $\eta > 0$ once K_0 contains finitely many eigenvectors of $\operatorname{tr}_Z \tau$ (Step 3). A general τ is a combination of four positive ones (Step 1). So, along the finite-dimensional subspaces of $\mathcal{H}(K)$ ordered by inclusion, $F_0(\tau)$ is a Cauchy net in the complete space

$\mathcal{T}(Z'K)$; let $\Phi_K(\tau)$ be its limit. Φ_K is linear, with $\|\Phi_K(\tau)\|_1 \le 2\|\tau\|_1$. For y of finite rank with range and support in K_0, $F_1(x \otimes y) = \Phi(x) \otimes y$ for every $K_1 \supseteq K_0$, so $\Phi_K(x \otimes y) = \Phi(x) \otimes y$; both sides are continuous in y, and finite-rank operators are dense (Step 5), so this holds for every y. It is unique because products span a dense subspace (Step 5). Trace: $\operatorname{tr} F_0(\tau) = \operatorname{tr}[P_0\tau] \to \operatorname{tr}\tau$. Complete positivity: for $k \ge 1$, applying F_0 to every entry of a positive $k \times k$ matrix ω of trace-class operators conjugates ω by $\mathbb{1}_{\mathbb{C}^k} \otimes \mathbb{1}_Z \otimes W_0^\dagger$, applies Φ to every entry of a matrix of size $k \dim K_0$, which is positive, and conjugates back, so the result is positive; entrywise, these converge in trace norm to Φ_K applied to every entry, and a trace-norm limit of positive operators is positive. □

3 Binary64 numbers and exact sums

Declared values are binary64 numbers, read as the dyadic rationals they are. A sum of them is accumulated left to right in a declared order, and it rounds nowhere when every binary64 addition is exact. The main result, Lemma 3.4, shows that such a sum is the exact sum, so a predicate on it is a predicate on the declared values. An example shows that the order of accumulation changes the computed sum.

A declared value is a binary64 number and denotes exactly the dyadic rational it is, with no exact object behind it. A binary64 addition is exact precisely when the true sum is a binary64 number, and a lattice value at k is an integer multiple of two to the power $-k$ of modulus at most one.

Definition 3.1 (Binary64 numbers, declared values and dyadic lattices). (i) Let $\mathbb{B}$ be the set of real numbers $m\,2^q$ with $m, q \in \mathbb{Z}$, $|m| < 2^{53}$ and $-1074 \le q \le 971$, and let $\mathbb{B}_{\mathbb{C}} := \{x+iy : x, y \in \mathbb{B}\}$. The elements of $\mathbb{B}$ are the *binary64 numbers*, the finite values of the IEEE 754 binary64 format. Every binary64 number is a dyadic rational, an element of $\mathbb{Z}[\frac{1}{2}]$.

(ii) For a real number x with $|x| < 2^{1024} - 2^{970}$, $\mathrm{fl}(x)$ is x rounded to the nearest element of $\mathbb{B}$, a tie going to the element whose m is even in its representation of (i) with the least q. For $|x| \ge 2^{1024} - 2^{970}$, $\mathrm{fl}(x) := +\infty$ if $x > 0$ and $-\infty$ if $x < 0$; the threshold is the midpoint between the largest element $(2^{53}-1)\,2^{971}$ of $\mathbb{B}$ and 2^{1024}, which is where IEEE 754 rounding to nearest overflows. Further, $\mathrm{fl}(\pm\infty + y) := \pm\infty$ for $y \in \mathbb{B}$. The *binary64 sum* of a and b is $\mathrm{fl}(a + b)$. On $\mathbb{B}_{\mathbb{C}}$, and on matrices over it, fl acts on the real and on the imaginary part of every entry separately. Two properties of fl are all that is used of it: on real input its value lies in $\mathbb{B} \cup \{\pm\infty\}$, and $\mathrm{fl}(x) = x$ for $x \in \mathbb{B}$. The binary64 sum of a and b is *exact* when it equals $a + b$; by those two properties

it is exact if and only if the real and the imaginary part of every entry of $a + b$ lie in $\mathbb{B}$.

(iii) A *declared value* is a binary64 number stated as a datum. It denotes that dyadic rational and nothing else: there is no exact object behind it of which it is an approximation, and every predicate stated of declared values is a predicate of those rationals. A declared complex value is an element of $\mathbb{B}_{\mathbb{C}}$, and a declared matrix is a matrix of declared complex values.

(iv) A *lattice exponent* is an integer k with $1 \le k \le 53$. A real number x is a *lattice value* at k when $2^k x \in \mathbb{Z}$ and $|x| \le 1$; a complex number, or a matrix, is one when the real and the imaginary part of every entry are.

Used by Lem. 3.2, Def. 3.3, Lem. 3.4, Rem. 3.5, Def. 4.1, Def. 4.3, Rem. 4.4, Def. 5.1, Def. 5.3, Def. 5.5, Req. 10.5, Req. 11.1, Req. 11.2, Req. 12.1, protocol-compiler/def:exact, protocol-compiler/lem:weighted-factor, receiver/def:complex-enclosure, receiver/def:exact-route, receiver/detector-imperfections/cor:exact-dark-composed, receiver/req:exact-route-decision, receiver/thm:exact-and-evaluated, reductions/def:exactadm · newly derived · implemented · *Code* certiqs_key.foundations.exact:scalar_is_binary64

Every lattice value at an exponent $k \le 53$ is a binary64 number, and between two consecutive powers of two the binary64 numbers are the multiples of one power of two, in [1/2, 1) those of two to the power -53. Representability of a declared entry is therefore a divisibility condition.

Lemma 3.2 (Binary64 numbers on dyadic lattices)**.** (i) For every lattice exponent k, every real lattice value at k is a binary64 number, and every complex lattice value at k lies in $\mathbb{B}_{\mathbb{C}}$.

(ii) For every integer e with $-1022 \le e \le 1023$,

$$\mathbb{B} \cap \left[2^e, 2^{e+1}\right) \;=\; 2^{e-52}\,\mathbb{Z} \cap \left[2^e, 2^{e+1}\right). \tag{3.1}$$

In particular the binary64 numbers in $[\frac{1}{2}, 1)$ are the multiples of 2^{-53} in that interval, and those in $[1, 2)$ are the multiples of 2^{-52} in it.

Uses Def. 3.1 · *Used by* Rem. 3.5, Rem. 4.4, Req. 11.1, reductions/def:exactadm · newly derived · accepted

Proof. (i) Let x be a real lattice value at k (Definition 3.1 (iv)) and write $x = n\,2^{-k}$ with $n \in \mathbb{Z}$; then $|n| = 2^k|x| \le 2^k \le 2^{53}$. If $|n| < 2^{53}$, then $x = n\,2^q$ with $q = -k \in [-53, -1] \subseteq [-1074, 971]$, so $x \in \mathbb{B}$ by Definition 3.1 (i). Otherwise $|n| = 2^{53}$, which forces $k = 53$ and $|x| = 1$, and $x = (\pm 1)\,2^0 \in \mathbb{B}$. A complex lattice value has its real and imaginary parts on the lattice, hence in $\mathbb{B}$.

(ii) Let $x = n\,2^{e-52}$ with $n \in \mathbb{Z}$ and $2^e \le x < 2^{e+1}$. Then $2^{52} \le n < 2^{53}$, and $q := e - 52$ satisfies $-1074 \le q \le 971$ because $-1022 \le e \le 1023$;

so $x \in \mathbb{B}$. Conversely, let $x = m\,2^q \in \mathbb{B}$, as in Definition 3.1 (i), with $2^e \le x < 2^{e+1}$. Then $m > 0$, and $m < 2^{53}$ gives $2^q = x/m > 2^e\,2^{-53}$, so the integer q satisfies $q > e-53$, that is $q \ge e-52$. Hence $x = \left(m\,2^{q-e+52}\right)2^{e-52}$ with $m\,2^{q-e+52} \in \mathbb{Z}$, which proves (3.1). The two particular cases are $e = -1$ and $e = 0$. □

A sum of declared values is accumulated left to right in a declared order, and it rounds nowhere when every one of its binary64 additions is exact. Binary64 addition is not associative, so the order is part of the datum.

In exact arithmetic the order in which a finite sum is accumulated is immaterial. Binary64 addition is not associative, so a sum that a predicate reads comes with its order, and the order is part of the datum (Remark 3.5).

Definition 3.3 (Sums in a declared order and sums that round nowhere)**.** Let $(x_1, \dots, x_n)$, $n \ge 1$, be a finite sequence of elements of $\mathbb{B}_{\mathbb{C}}$, or of matrices of one shape over $\mathbb{B}_{\mathbb{C}}$ (Definition 3.1 (i)): declared values and declared matrices (Definition 3.1 (iii)), or the computed inner sums of a sum of sums below. Its *sequential sum* is

$$S_0 \coloneqq 0, \qquad S_i \coloneqq S_{i-1} + x_i \quad (1 \le i \le n), \tag{3.2}$$

and its *computed sequential sum* is $\hat{S}_0 \coloneqq 0$, $\hat{S}_i \coloneqq \mathrm{fl}(\hat{S}_{i-1} + x_i)$ for $1 \le i \le n$ (Definition 3.1 (ii)), formed left to right, entrywise, with no re-ordering and no re-association. The accumulation *rounds nowhere* when every one of its n binary64 additions is exact: for every $1 \le i \le n$, every entry of $\hat{S}_{i-1}$ is finite and $\hat{S}_i = \hat{S}_{i-1} + x_i$.

A *declared order* on a finite alphabet J is a total order $j_1 \prec j_2 \prec \cdots \prec j_{|J|}$ on J, fixed together with J. For a family $\{a_j\}_{j \in J}$ of declared values, or of declared matrices of one shape, the *sum in the declared order* is the sequential sum of $(a_{j_1}, \dots, a_{j_{|J|}})$, and the family's sum rounds nowhere when that accumulation does. The trace of a declared $d \times d$ matrix X is accumulated as the sequential sum of $(X_{11}, X_{22}, \dots, X_{dd})$, in increasing row index. A sum of such sums over an alphabet with a declared order rounds nowhere when every inner accumulation rounds nowhere and the sequential sum, in that order, of the computed inner sums rounds nowhere; that sequential sum is then the computed sum of sums. Each computed inner sum is then over $\mathbb{B}_{\mathbb{C}}$: its last addition is exact, so it equals a sum of finite numbers, and it is a value of fl, which lies in $\mathbb{B} \cup \{\pm\infty\}$ entry by entry and part by part (Definition 3.1 (ii)). The outer sequential sum is therefore one of the sequences above.

Uses Def. 3.1 · *Used by* Lem. 3.4, Rem. 3.5, Def. 4.3, Def. 5.5, Req. 9.1, Req. 9.3, Req. 11.3, meat-core/def:declaration · newly derived · implemented · *Code* certiqs_key.foundations.exact:sum_in_declared_order

A sequential sum rounds nowhere exactly when every exact partial sum is a binary64 number, and then the computed sum equals the exact sum at every step. A predicate stated on such a sum is a predicate on the declared values themselves.

Lemma 3.4 (Exactness of a sum that rounds nowhere). Let $(x_1, \dots, x_n)$ be a finite sequence of elements of $\mathbb{B}_{\mathbb{C}}$, or of matrices of one shape over $\mathbb{B}_{\mathbb{C}}$, such as declared values or the computed inner sums of a sum of sums, with sequential sum $(S_i)_{i=0}^n$ and computed sequential sum $(\hat{S}_i)_{i=0}^n$ (Definition 3.3). The following are equivalent:

(a) the accumulation rounds nowhere;

(b) $\hat{S}_i = S_i$ for every $0 \le i \le n$;

(c) for every $1 \le i \le n$, the real and the imaginary part of every entry of S_i are binary64 numbers.

If they hold, the computed sum $\hat{S}_n$ is the exact sum $\sum_{i=1}^n x_i$.

Uses Def. 3.1, Def. 3.3 · *Used by* Rem. 3.5, Def. 4.3, Rem. 4.4, Req. 9.1, protocol-compiler/def:exact · newly derived · accepted

Proof. The map fl acts on the real and the imaginary part of every entry separately (Definition 3.1 (ii)), and S_i and $\hat{S}_i$ are formed entry by entry, so it suffices to prove the lemma for a sequence of real numbers $x_i \in \mathbb{B}$.

(a)⇒(b). By induction on i: $\hat{S}_0 = 0 = S_0$, and if $\hat{S}_{i-1} = S_{i-1}$, then (a) gives $\hat{S}_i = \hat{S}_{i-1} + x_i = S_{i-1} + x_i = S_i$.

(b)⇒(c). Let $1 \le i \le n$. The value $\hat{S}_i = \mathrm{fl}(\hat{S}_{i-1} + x_i)$ lies in $\mathbb{B} \cup \{\pm\infty\}$, by the first property of fl where $\hat{S}_{i-1}$ is finite and by its definition on $\pm\infty$ otherwise. By (b) it equals the real number S_i, so $S_i \in \mathbb{B}$.

(c)⇒(a). We show by induction on i that $\hat{S}_i = S_i$ and that the i-th addition is exact. For $i = 0$, $\hat{S}_0 = S_0 = 0$. Let $1 \le i \le n$ and $\hat{S}_{i-1} = S_{i-1}$. Then $\hat{S}_{i-1}$ is finite, and $\hat{S}_{i-1} + x_i = S_i \in \mathbb{B}$ by (c), so the second property of fl gives $\hat{S}_i = \mathrm{fl}(S_i) = S_i = \hat{S}_{i-1} + x_i$: the i-th addition is exact, and $\hat{S}_i = S_i$.

Finally, if (b) holds then $\hat{S}_n = S_n = \sum_{i=1}^n x_i$ by (3.2). □

Two orderings of the same three binary64 numbers have the same exact sum but different computed sums, and only one of them rounds nowhere. A computed sum is therefore a determinate number only once its order is fixed.

Remark 3.5 (Dependence of a computed sum on its order). Binary64 addition is not associative, so the computed sequential sum of Definition 3.3 depends on the order of its terms. By Lemma 3.2 (ii) the binary64 numbers in $[1, 2)$ are the multiples of 2^{-52}, so $1 + 2^{-60}$ lies strictly between the adjacent binary64 numbers 1 and $1 + 2^{-52}$ and is nearer to 1; hence $\mathrm{fl}(1 + 2^{-60}) = 1$ (Definition 3.1 (ii)). The sequence $(1, -1, 2^{-60})$ therefore has computed sequential sum $0, 1, 0, 2^{-60}$, every addition being exact, while $(1, 2^{-60}, -1)$ has computed sequential sum $0, 1, 1, 0$, its second addition rounding. The exact sum is 2^{-60} in both orders. Only the first accumulation rounds nowhere, and its computed sum is the exact sum, as Lemma 3.4 guarantees; the second computed sum is 0.

Uses Def. 3.1, Lem. 3.2, Def. 3.3, Lem. 3.4 · newly derived · accepted

4 Declared data and exact admissibility

Declared data are what an instance states and what a producer emits as its own declaration, and a consumer reads them across a module boundary. An operator is a list of blocks against a sector map and is block diagonal by construction. The section's main definition, Definition 4.3, states the five conditions every module's admission shares, and a counterexample shows why a completeness sum must round nowhere.

Declared data are what an instance states, a supplier's output included, and what a producer emits as its own declaration. A consumer reads declared data across a module boundary; a point a module computes itself is never declared.

Definition 4.1 (Declared data and the parties to a module boundary)**.** (i) An *instance* is the party that fixes one protocol's device data, protocol data and run configuration. The data it states are *declared data*, and it is their *declarer*.

(ii) A module is a *supplier* when its own claims decide their conditions on the data it reads or emits in floating point, at slacks an instance declares to it, rather than exactly. A floating-point evaluation of a device model is an example. Its numerical output is not declared data until an instance states it as its own declared data, and the instance is then its declarer. Numbers of a supplier's output that reach another module in a run are declared by the instance that fixed the run: those it rounds and states, and those it passes on unchanged, whether in a declaration of its own or by handing the supplier's output on untouched.

(iii) A module other than a supplier that emits numerical data to another module as its own declaration, constructed from declared data it read, is a *producer* of those data; they are declared data, and the producer is their declarer. Declared data that a module passes on unchanged keep their declarer, and the module is not their producer. A scalar bound that a producer emits and that a consumer has replaced by its rounding to a binary64 number, in a direction that the producer's own claims license as weakening it, is read, at its replaced value, as its producer's declaration of that bound, and keeps its producer as declarer; a set defined by such bounds is then declared data by (v).

(iv) A module other than a supplier that reads declared data is a *consumer* of them. The passage of declared data from their declarer to a consumer is a *module boundary*.

(v) A set defined by declared data, such as a feasible set or an acceptance set, is declared data. An object that a module computes itself in the course of a computation, such as an iterate of an optimiser or the return of an oracle, is a *computed object* and is not declared data.

(vi) Every entry of declared operator data is a declared value (Definition 3.1 (iii)).

Uses Def. 3.1 · *Used by* Def. 5.1, Def. 5.5, Def. 5.7, Def. 7.1, Ass. 8.1, Req. 9.1, Req. 9.2, Req. 9.3, Req. 10.1, Req. 10.2, Req. 10.3, Req. 10.4, Req. 10.5, Req. 11.1, Req. 11.2, Req. 11.3, Req. 12.1 · newly derived · accepted

A sector map names the blocks of an operator by a block kind and an ordered list of identifiers with dimensions. An operator given as one array per block is block diagonal by construction, and its Hermiticity, positivity and completeness hold block by block.

Definition 4.2 (Sector maps and block-list operators)**.** A *sector map* $\mathfrak{m}$ consists of two components: a block kind, which is a label naming what its blocks are, and a finite non-empty set S of block identifiers listed in a fixed order, each identifier $s \in \mathsf{S}$ with a dimension $d_s \geq 1$. Two sector maps *agree* when they have the same block kind and the same identifiers, in the same order, with the same dimensions. A system of dimension d without a declared sector decomposition carries the *one-block map* of dimension d: one identifier, with dimension d, and a block kind and an identifier that are the same for every such system, so that two one-block maps agree if and only if their dimensions are equal.

A *block list* against $\mathfrak{m}$ is a list $X = (X^{(s)})_{s\in\mathsf{S}}$ of complex $d_s \times d_s$ matrices. It denotes the operator $\bigoplus_{s\in\mathsf{S}} X^{(s)}$ on $\bigoplus_{s\in\mathsf{S}} \mathbb{C}^{d_s}$, and we identify the two: the list is the operator, and no entry outside the blocks exists to be written, so a block list is block diagonal by construction. A family of block lists against $\mathfrak{m}$ is a family $\{X_j\}_{j\in J}$ of such lists, $X_j^{(s)}$ being the block s of X_j.

Because the direct sum acts block by block, $X = X^\dagger$ if and only if $X^{(s)} = X^{(s)\dagger}$ for every s; $X \succeq 0$ if and only if $X^{(s)} \succeq 0$ for every s, since $\langle v, Xv\rangle = \sum_s \langle v_s, X^{(s)} v_s\rangle$ for $v = (v_s)_s$; and $\sum_{j\in J} X_j = c\,\mathbb{1}$ for a scalar c if and only if $\sum_{j\in J} X_j^{(s)} = c\,\mathbb{1}_{d_s}$ for every s.

Used by Def. 4.3, Def. 5.1, Def. 5.3, Def. 5.5, Def. 5.7, Def. 6.1, Lem. 6.2, Ass. 8.1, Req. 10.6, Req. 11.3, meat-core/thm:choi, receiver/detector-imperfections/cor:exact-dark-composed, receiver/detector-imperfections/req:dark-composed-reference, receiver/thm:enclosed-blocks · newly derived · implemented · *Code* certiqs_key.foundations.blocks:SectorStructure

A declared operator family is exactly admissible for a sector map with constant c when (T) its own sector map agrees, (H) every block is exactly Hermitian, (P) positive, (C) it sums to c times the identity and (S) its declared-order sum rounds nowhere. Each is a property of the declared numbers.

The five conditions (T), (H), (P), (C) and (S) of the definition are the part that every module's admission of a declared operator family shares. A module may impose further conditions of its own, such as a declared lattice or conditions on other declared objects. A module may also carry the dimension clause of (T) by the type of its family: where each block is typed as a $d_s \times d_s$ matrix for the dimension d_s of the consumer's block s, comparing block kind, identifiers and order is the whole of (T).

Definition 4.3 (Exact admissibility of a declared operator family). Let $\mathfrak{m}$ be a sector map with blocks $s \in \mathsf{S}$ of dimensions d_s (Definition 4.2), let J be a finite non-empty alphabet with a declared order, let c be a declared value, and let $\{a_j\}_{j\in J}$ be a family of block lists against a sector map $\mathfrak{m}'$ whose entries are declared values (Definition 3.1 (iii)). The family is *exactly admissible* for $\mathfrak{m}$ with constant c when

(T) *(sector map)* $\mathfrak{m}'$ agrees with $\mathfrak{m}$;

(H) *(Hermiticity)* $a_j^{(s)} = a_j^{(s)\dagger}$ for every $j \in J$ and $s \in \mathsf{S}$;

(P) *(positivity)* $a_j^{(s)} \succeq 0$ for every $j \in J$ and $s \in \mathsf{S}$;

(C) *(completeness)* $\sum_{j\in J} a_j^{(s)} = c\,\mathbb{1}_{d_s}$ for every $s \in \mathsf{S}$, as an identity of the declared numbers;

(S) *(the sum rounds nowhere)* for every $s \in \mathsf{S}$, the sum of $\{a_j^{(s)}\}_{j\in J}$ in the declared order of J rounds nowhere (Definition 3.3).

The family is exactly admissible for $\mathfrak{m}$ when it is so with constant $c = 1$; it is then a positive operator-valued measure on $\bigoplus_s \mathbb{C}^{d_s}$, by the block-by-block reading of Definition 4.2. Since the block s of a block list against $\mathfrak{m}'$ is a square matrix of the dimension $\mathfrak{m}'$ gives s, (T) holds if and only if $\mathfrak{m}'$ has the block kind and the identifiers of $\mathfrak{m}$, in the same order, and every block $a_j^{(s)}$ is a $d_s \times d_s$ matrix, J being non-empty. A family on a system without a declared sector decomposition is read against the one-block map of that system's dimension, and (T) then reduces to the agreement of that dimension with the size of the family's matrices.

Each condition is a property of the declared values themselves and involves no threshold and no norm: (T) compares declared labels and dimensions, (H) and (C) are identities between exactly represented numbers, (P) is a property of matrices with dyadic entries, and (S) is a property of the binary64 additions that form the declared-order sum. Under (S) the computed sum in the declared order is the exact sum (Lemma 3.4), so (C) holds if and only if that computed sum equals $c\,\mathbb{1}_{d_s}$ on every block.

Uses Def. 3.1, Def. 3.3, Lem. 3.4, Def. 4.2 · *Used by* Rem. 4.4, Def. 5.5, Def. 7.1, Req. 9.1, Req. 10.5, Req. 10.6, Req. 11.3, protocol-compiler/def:exact, reductions/def:exactadm · newly derived · implemented (effective: accepted) · *Code* certiqs_key.foundations.exact_admission:admit_family_exact

Three positive binary64 numbers can have computed sum exactly one while their true sum exceeds one. A completeness test on the computed sum alone would admit them, which is why the rounds-nowhere condition belongs to exact admissibility.

Remark 4.4 (A complete computed sum from an incomplete family). Conditions (H) and (P) of Definition 4.3, together with the identity of the *computed* declared-order sum with the identity, do not imply (C). Take the one-block map of dimension one and the family $(1, 2^{-54}, 2^{-54})$ in this declared order. Its entries are binary64

numbers, Hermitian and positive. By Lemma 3.2 (ii) the binary64 numbers in $[1, 2)$ are the multiples of 2^{-52}, so $1 + 2^{-54}$ lies strictly between the adjacent binary64 numbers 1 and $1+2^{-52}$ and is nearer to 1; hence $\mathrm{fl}(1+2^{-54}) = 1$ (Definition 3.1 (ii)), and the computed sequential sum is $0, 1, 1, 1$. The exact sum is $1 + 2^{-53} \neq 1$, so (C) fails and the family is not a positive operator-valued measure. Condition (S) fails at the second addition; it is the condition under which the computed sum is the exact sum at every step (Lemma 3.4).

Uses Def. 3.1, Lem. 3.2, Lem. 3.4, Def. 4.3 · newly derived · accepted

5 The declared tolerance and the snap

An instance declares one tolerance, which bounds the defects it may repair in its own data, measured as entrywise defects of differences and as a floor on the least eigenvalue. Lemma 5.2 shows that the floor is the stricter of the two, by a factor of the dimension that is attained. A snap replaces a declared family by an exactly admissible one on the declarer's lattice and records what it changed. A measurement family has a reference, the model it is declared from, known exactly or through rational entrywise enclosures, and on some blocks also through an enclosed factor with an exact core (Definition 5.4). Entries known to be zero fix the structural support of each element, which no snap may enlarge, save the flagged last resort of Requirement 10.6 (iv); a block is kept where the snap is decided equal to the data it is made from, and moved otherwise. An enclosure entry whose disc contains zero may be recentred at zero and widened, and the reference is unchanged (Lemma 5.6). A declarer may hand the reference beside the snap, and a block whose snapped data nothing the key length rests on reads, save the points and the acceptance data, is evaluated off its snap (Definition 5.7).

The declared tolerance is a binary64 number in [0, 1) that bounds the defects of an instance's own data before they are snapped. A defect of a difference is an entrywise maximum modulus, and positivity is measured by a floor on the least eigenvalue.

Definition 5.1 (The declared tolerance and the defects it bounds)**.** The *declared tolerance* is a binary64 number τ_{num} with $0 \leq \tau_{\mathrm{num}} < 1$, stated by an instance in its run configuration (Definition 4.1). For a complex matrix Y we write $\|Y\|_{\max} := \max_{i,l} |Y_{il}|$ for its *entrywise maximum modulus*. The *defects* of data are the entrywise maximum moduli of the differences their predicates name:

(i) the *Hermiticity defect* $\left\|X - X^{\dagger}\right\|_{\max}$ of a square matrix X;

(ii) the *completeness residue* $\max_{s \in \mathsf{S}} \left\|\sum_{j \in J} a_j^{(s)} - c\,\mathbb{1}_{d_s}\right\|_{\max}$ of a family of block lists $\{a_j\}_{j \in J}$ against a sector map with blocks $s \in \mathsf{S}$ (Definition 4.2), read with a declared constant c;

(iii) the *column residue* $\max_l \left|\sum_i P_{il} - 1\right|$ of a real matrix P read as column-stochastic;

(iv) the *trace residue* $|\operatorname{tr} X - 1|$ of a square matrix X read as having unit trace, and the *sum residue* $\left|\sum_x p_x - 1\right|$ of a vector p read as a probability vector.

The sums here are exact sums. A square matrix X passes the *positivity floor* at τ_{num} when $\lambda_{\min}\left(\frac{1}{2}(X + X^\dagger)\right) \geq -\tau_{\text{num}}$. Data lie *within the declared tolerance* when each of their defects is at most τ_{num} and each of their blocks read as positive semidefinite passes the positivity floor at τ_{num}.

Uses Def. 3.1, Def. 4.1, Def. 4.2 · *Used by* Def. 5.5, Def. 7.1, Req. 10.1, Req. 10.2, Req. 10.3, Req. 10.4 · newly derived · accepted

A Hermitian matrix with least eigenvalue at least $-\delta$ lies within entrywise distance δ of the positive cone, but entrywise distance δ only guarantees a least eigenvalue of $-d\delta$ in dimension d, and that factor is attained. The two tests are different quantities and never interchangeable.

The declared tolerance governs two kinds of test, entrywise defects of differences and a floor on the least eigenvalue (Definition 5.1). The lemma compares them.

Lemma 5.2 (The positivity floor against entrywise closeness to the cone)**.** Let $d \geq 1$ and $\delta \geq 0$, and write $\|\cdot\|$ for the operator norm and $\|\cdot\|_{\max}$ for the entrywise maximum modulus of complex $d \times d$ matrices.

(i) $\|Y\|_{\max} \leq \|Y\| \leq d\, \|Y\|_{\max}$ for every complex $d \times d$ matrix Y.

(ii) If X is Hermitian and $\lambda_{\min}(X) \geq -\delta$, then $X + \delta\, \mathbb{1}_d \succeq 0$ and $\|(X + \delta\, \mathbb{1}_d) - X\|_{\max} = \delta$.

(iii) If X is Hermitian and $\|X - P\|_{\max} \leq \delta$ for some $P \succeq 0$, then $\lambda_{\min}(X) \geq -d\delta$. The bound is attained: for the $d \times d$ matrix E with every entry equal to 1, the matrices $X := -\delta E$ and $P := 0$ satisfy $\|X - P\|_{\max} = \delta$ and $\lambda_{\min}(X) = -d\delta$.

Used by Req. 10.3 · newly derived · accepted

Proof. (i) For standard basis vectors e_i, e_l, $|Y_{il}| = |\langle e_i, Y e_l\rangle| \leq \|Y\|$ by the Cauchy–Schwarz inequality, which is the first inequality. For a unit vector v, the Cauchy–Schwarz inequality applied to each row gives

$$\|Yv\|^2 = \sum_i \left|\sum_l Y_{il} v_l\right|^2 \leq \sum_i \left(\sum_l |Y_{il}|^2\right)\left(\sum_l |v_l|^2\right) = \sum_{i,l} |Y_{il}|^2 \leq d^2\, \|Y\|_{\max}^2,$$

so $\|Y\| \leq d\, \|Y\|_{\max}$.

(ii) The Hermitian matrix $X + \delta\, \mathbb{1}_d$ has the eigenvalues of X shifted by δ, which are non-negative, so it is positive semidefinite. The difference $\delta\, \mathbb{1}_d$

has diagonal entries δ and off-diagonal entries 0, so its entrywise maximum modulus is δ.

(iii) $X - P$ is Hermitian, X and P being so. For a unit vector v,

$$\langle v, Xv\rangle = \langle v, Pv\rangle + \langle v, (X-P)v\rangle \geq 0 - \|X-P\| \geq -d\,\|X-P\|_{\max} \geq -d\delta,$$

by $P \succeq 0$, the Cauchy–Schwarz inequality and (i). Since $\lambda_{\min}(X) = \min_{\|v\|=1}\langle v, Xv\rangle$ for Hermitian X, this gives $\lambda_{\min}(X) \geq -d\delta$. For the example, let u be the unit vector with every entry $d^{-1/2}$; then $E = d\,uu^{\dagger}$, whose eigenvalues are d, on u, and 0, on the orthogonal complement of u. So the least eigenvalue of $-\delta E$ is $-d\delta$, and every entry of $-\delta E - 0$ has modulus δ. □

A reference is the model a measurement family is declared from: Hermitian, positive and complete blocks, each read through an entrywise enclosure, a Gaussian-rational centre with rational entrywise radii. It is exact when every radius is zero, and every decision about it is then on rationals.

A family declared on a lattice is a family of dyadic rationals; the model it is declared from need not be one. Photon-number blocks written in the orthonormal occupation basis carry factors $1/\sqrt{n}$ even when every parameter of the model is rational. A polarisation analyser whose analysed state is $\cos\alpha\,|H\rangle + \sin\alpha\,|V\rangle$ has on the one-photon block the entries $\cos^2\alpha$, $\cos\alpha\sin\alpha$ and $\sin^2\alpha$ in the H, V basis, which are all rational exactly when $\cos\alpha = 0$ or $\tan\alpha$ is rational; at $\alpha = \pi/4$ the cosine is irrational and every entry is $1/2$, while at $\alpha = \pi/8$ the entries are irrational. A decision about such a model is still a decision on rationals when each of its entries is known to lie in a rational box around a Gaussian-rational centre, and the definition reads the model through such boxes. A block of the form $D^{-1}SD^{-1}$, with S Gaussian-rational and D the diagonal matrix of square roots of positive rationals, receives one from rational bounds on those square roots (Lemma 6.5 (ii)), and an entry known exactly is enclosed with radius zero. A model whose entries are all Gaussian rationals is the case in which every radius is zero.

Definition 5.3 (Reference of a measurement family, exact or enclosed)**.** Let $\mathfrak{m}$ be a sector map with blocks $s \in \mathsf{S}$ of dimensions d_s (Definition 4.2), let J be a finite non-empty alphabet with a declared order, and let $c > 0$ be a declared value (Definition 3.1 (iii)). A *entrywise enclosure* of a complex $d \times d$ matrix X is a pair (X_c, r) of a Hermitian $d \times d$ matrix X_c with entries in $\mathbb{Q}(i) = \{x + iy : x, y \in \mathbb{Q}\}$, its *centre*, and rationals $r_{il} = r_{li} \geq 0$, its *radii*, with

$$\left|X_{il} - (X_c)_{il}\right| \;\leq\; r_{il} \qquad \text{for all } i, l. \tag{5.1}$$

A *reference* against $\mathfrak{m}$ over J with constant c is a family $G = \{G_j\}_{j\in J}$ of block lists against $\mathfrak{m}$ whose blocks are Hermitian and positive semidefinite, with

$$\sum_{j\in J} G_j^{(s)} \;=\; c\,\mathbb{1}_{d_s} \qquad \text{for every } s \in \mathsf{S}, \tag{5.2}$$

read together with an entrywise enclosure $\left(C_j^{(s)}, r_j^{(s)}\right)$ of every block $G_j^{(s)}$. It is an *exact reference* when every radius is 0; then $G_j^{(s)} = C_j^{(s)}$ has entries in $\mathbb{Q}(i)$, and its Hermiticity, positive semidefiniteness and completeness are properties of finitely many Gaussian rationals.

For a Hermitian $d_s \times d_s$ matrix Y with entries in $\mathbb{Q}(i)$, the pair $\left(C_j^{(s)} - Y, r_j^{(s)}\right)$ is an entrywise enclosure of $G_j^{(s)} - Y$, since the two differences in (5.1) are equal. A reference is an object of its own beside a family declared from it: the entries of the declared family are declared values, and each denotes exactly the dyadic rational it is (Definition 3.1 (iii)).

Uses Def. 3.1, Def. 4.2 · *Used by* Def. 5.4, Def. 5.5, Lem. 5.6, Def. 5.7, Rem. 6.6, Def. 7.1, Cor. 7.3, Ass. 8.1, Req. 10.1, Req. 10.4, Req. 10.5, Req. 10.6, meat-core/frank-wolfe/lem:certified-bracket-enclosed, meat-core/req:companion-admission, receiver/detector-imperfections/cor:exact-dark-composed, receiver/detector-imperfections/req:dark-composed-reference, receiver/detector-imperfections/req:snap-fix, receiver/lem:enclosure-arithmetic, receiver/req:rigorous-weight, receiver/thm:enclosed-blocks, receiver/thm:exact-and-evaluated, receiver/thm:exact-blocks, reductions/cor:reference-companion, reductions/def:emitnorm, reductions/wpfss/prop:precomposed-blocks · newly derived · implemented · *Code* certiqs_key.foundations.reference:Reference

A factor enclosure knows a matrix as L W L, with L* the adjoint of some L in a box of complex matrices around Gaussian-rational centres and W an exact positive diagonal core; its members are those products. A reference may carry one on a block beside the block's entrywise enclosure.*

An entrywise enclosure with positive radii of an element whose rank is below its dimension, and whose support is spanned by no set of coordinate vectors, contains the element, whose block on the rows the enclosure does not fix at zero is singular, so no lower bound on a least eigenvalue over it is positive and it fixes no rank (Corollary 7.3 (iv)). The projector of an ideal analyser at an angle whose projector has irrational entries is such an element. It is often known in another form: as a sum $\sum_m w_m c_m c_m^\dagger$ over columns c_m computed in an enclosing arithmetic, with exact positive weights w_m, as an element is that compresses a projector through an isometry. Read through its columns, the element keeps the rank the columns fix (Corollary 7.4). The definition names that reading; it is an addition to a reference, never a replacement of its entrywise enclosures.

Definition 5.4 (Factor enclosure of a block)**.** Let $d \geq 1$ and $k \geq 0$ be integers. A *matrix box* of complex $d \times k$ matrices is a pair (L_c, t) of a $d \times k$ matrix L_c with entries in $\mathbb{Q}(i)$, its *centre*, and rationals $t_{im} \geq 0$, its *radii*; it *contains* a complex $d \times k$ matrix L when $|L_{im} - (L_c)_{im}| \leq t_{im}$ for all i, m. For $k = 0$ it contains the one $d \times 0$ matrix.

A *factor enclosure* of a complex $d \times d$ matrix X is a triple (L_c, t, W) of a matrix box (L_c, t) of complex $d \times k$ matrices and a diagonal $k \times k$ matrix

W with positive rational diagonal entries, its *core*, such that

$$X \;=\; LWL^{\dagger} \qquad \text{for some } L \text{ that } (L_c, t) \text{ contains.} \tag{5.3}$$

The number k is its *number of columns*, a matrix L that the box contains is a *factor* of the triple, and the matrices $LWL^{\dagger}$ with L a factor are its *members*, X among them; for $k = 0$ the one member is the zero $d \times d$ matrix. Every member is Hermitian and positive semidefinite, since $x^{\dagger}LWL^{\dagger}x = \|W^{1/2}L^{\dagger}x\|^2$ for every $x \in \mathbb{C}^d$.

A reference G (Definition 5.3) *carries* a factor enclosure on a block $G_j^{(s)}$ when it is read, besides the entrywise enclosure $\left(C_j^{(s)}, r_j^{(s)}\right)$ of that block, with a factor enclosure of $G_j^{(s)}$; the block then satisfies both (5.1) and (5.3).

Uses Def. 5.3 · *Used by* Cor. 7.4, Req. 10.5, meat-core/cor:reference-companion-factors, meat-core/frank-wolfe/lem:certified-bracket-enclosed, meat-core/frank-wolfe/req:certified-evaluation, protocol-compiler/lem:weighted-factor, protocol-compiler/req:reference-companion-compiled, receiver/detector-imperfections/req:dark-composed-reference, receiver/thm:factor-blocks, reductions/cor:companion-factor-enclosure, reductions/req:reference-companion · newly derived · implemented · *Code* certiqs_key.foundations.reference:FactorEnclosure

A snap replaces data by lattice data of the same shape that meet their conditions exactly, entries known to be zero fixing each element's structural support. A block of a measurement snap is kept where it is decided equal to the data it is made from, and moved otherwise.

One documented procedure produces the snaps an instance applies to what it declares; a producer that snaps its own emission does so by the procedure its own claims state. For an operator family read as a measurement the instance keeps a block whose reference (Definition 5.3) is known exactly and already lies on the lattice, and snaps every other block by a procedure that stays as close to what it starts from as the rule allows that every element, row and column the data state to be zero stays zero, which is what the structural support below records (Requirement 10.6); only where that procedure cannot restore the conditions of the snap does a flagged last resort mix a uniform family into the elements. The steps are Requirement 10.5 and the shared numerical code (`spec/architecture.md`). A snap relates nothing to the model it replaced beyond its displacement; a run that holds a reference can read the snapped family only to find points, evaluating the numbers its key length rests on with the reference in its place (Definition 5.7), and where it does not, it rests on the snap (Requirement 10.5 (iii)).

Definition 5.5 (Snapping declared data onto a lattice)**.** Let D be data a declarer is about to declare, a producer's exact construction of what it emits (Definition 4.1), or a reference of an operator family (Definition 5.3) — a family of block lists against a sector map (Definition 4.2) over an alphabet with a declared order, a matrix or a vector — read with the conditions its

declaration states of it, and let k be a lattice exponent (Definition 3.1 (iv)). For a family read as a measurement, those conditions are exact admissibility with its declared constant (Definition 4.3); every sum they read is read in its declared order (Definition 3.3). A *snap* of D at k is data D' of the same shape, over the same alphabet in the same order, every entry of which is a lattice value at k, and which satisfy those conditions exactly. Its *displacement* is the largest entrywise modulus of $D' - D$, taken over every block of every element (Definition 5.1), with D replaced by its centres where D is a reference.

An entry of D is *structurally zero* when it is known to be exactly zero, which is the following and nothing else: where D is given as numbers, an entry that is the number 0; where D is a reference, an entry whose centre and radius are both 0. An entry of a reference whose centre is 0 and whose radius is not is not structurally zero. The *structural support* of a block $D_j^{(s)}$ is the set of indices i such that row i or column i of $D_j^{(s)}$ holds an entry that is not structurally zero. A row, a column or an element all of whose entries are structurally zero is structurally zero, and an index lies outside the structural support exactly when both its row and its column are. For a reference, whose centres are Hermitian and whose radii are symmetric (Definition 5.3), row i of a block is structurally zero exactly when column i is.

Where D is an operator family read as a measurement against a sector map with blocks $s \in \mathsf{S}$, over an alphabet J, a block s of a snap $F = D'$ of D is *kept* when $F_j^{(s)} = D_j^{(s)}$ is decided exactly for every $j \in J$, and *moved* otherwise. Here D on block s is the data the snap is made from: the centres of an exact reference of the block where the snap runs on them, a producer's exact construction, or the data as they stand where the declarer states them with no model behind them. Where the declarer computes the data of the block from a model of it and the snap runs on that evaluation, the data the snap is made from are the model, of which the evaluation is no exact presentation, so no exact decision equates the snap with them and the block is moved, even where the snap leaves the evaluation unchanged. A snap with no moved block changes nothing.

The definition fixes what a snap produces and not how. The displacement is a record of the move, and it is the whole of what relates D' to D here.

Uses Def. 3.1, Def. 3.3, Def. 4.1, Def. 4.2, Def. 4.3, Def. 5.1, Def. 5.3 · *Used by* Lem. 5.6, Def. 5.7, Def. 7.1, Ass. 8.1, Req. 10.1, Req. 10.4, Req. 10.5, Req. 10.6, Req. 11.1, Req. 11.3, meat-core/rem:declared-measurement, receiver/detector-imperfections/req:dark-composed-reference, receiver/detector-imperfections/req:snap-fix, reductions/cor:reference-companion, reductions/def:emitnorm, reductions/thm:emit, reductions/wpfss/prop:precomposed-blocks · newly derived · implemented (effective: accepted) · *Code* certiqs_key.foundations.snap:snap_measurement_family

Zeroing every centre entry whose enclosure contains zero, and adding its modulus bound to the radius, gives another entrywise enclosure of the same matrix; a reference so recentred is the same reference. Any enclosure of a Hermitian matrix becomes an entrywise one by averaging.

An enclosure whose disc contains zero does not say whether the entry it encloses is zero. A family evaluated at its centres would then carry, at such an entry, a number of the order of the radius that the model may not have, and a support decision taken on the evaluation could see support there that the model lacks. Setting such a centre to zero, and widening the radius by what was removed, keeps the enclosure valid; the lemma says so and fixes the rational bound the widening uses. An enclosure formed by arithmetic on enclosures, of a matrix known to be Hermitian, need not come with a Hermitian centre and symmetric radii; part (iv) gives it them, so that it can be recentred in turn.

Lemma 5.6 (Zeroing the centres an enclosure cannot tell from zero). Let (C, r) be an entrywise enclosure of a complex $d \times d$ matrix X (Definition 5.3), and let

$$Z \;\coloneqq\; \big\{(i,l) : (\operatorname{Re} C_{il})^2 + (\operatorname{Im} C_{il})^2 \le r_{il}^2\big\},$$

the entries whose enclosure contains 0. Put $C'_{il} \coloneqq 0$ and $r'_{il} \coloneqq r_{il} + |\operatorname{Re} C_{il}| + |\operatorname{Im} C_{il}|$ for $(i,l) \in Z$, and $C'_{il} \coloneqq C_{il}$, $r'_{il} \coloneqq r_{il}$ otherwise. Then:

(i) Z is decided by comparisons of rationals, and (C', r') is an entrywise enclosure of X: C' is Hermitian with entries in $\mathbb{Q}(i)$, and $r'_{il} = r'_{li} \ge 0$ are rationals;

(ii) $r'_{il} \le (1 + \sqrt{2})\, r_{il}$ for every (i,l), and $r'_{il} = r_{il}$ wherever $C_{il} = 0$; in particular an entry that is structurally zero for (C, r) (Definition 5.5) is so for (C', r'), and every entry of Z has centre 0 in (C', r');

(iii) if G is a reference with constant c read with enclosures $\big(C_j^{(s)}, r_j^{(s)}\big)$, then G read with the enclosures $\big(C_j'^{(s)}, r_j'^{(s)}\big)$ formed by (i) from each is a reference with the same constant; it is the same family G, and it is an exact reference exactly when the first is;

(iv) if X is Hermitian and $(\tilde{C}, \tilde{r})$ is any pair of a complex $d \times d$ matrix $\tilde{C}$ with entries in $\mathbb{Q}(i)$ and rationals $\tilde{r}_{il} \ge 0$ with $|X_{il} - \tilde{C}_{il}| \le \tilde{r}_{il}$ for all i, l, Hermitian or symmetric or not, then $\big(\frac{1}{2}(\tilde{C} + \tilde{C}^{\dagger}), \frac{1}{2}(\tilde{r} + \tilde{r}^{\mathsf{T}})\big)$ is an entrywise enclosure of X, equal to $(\tilde{C}, \tilde{r})$ where $\tilde{C}$ is Hermitian and $\tilde{r}$ symmetric, so (i)–(ii) apply to it.

Uses Def. 5.3, Def. 5.5 · *Used by* meat-core/frank-wolfe/req:certified-evaluation, reductions/req:reference-companion · newly derived · implemented (effective: accepted) · *Code* certiqs_key.foundations.recentring:recentred_enclosure

Proof. (i) The real and imaginary parts of an entry of $\mathbb{Q}(i)$ are rationals and r_{il} is rational, so the inequality defining Z is a comparison of rationals, and r'_{il} is a sum of rationals. For $(i,l) \in Z$,

$$|X_{il} - C'_{il}| = |X_{il}| \le |X_{il} - C_{il}| + |C_{il}| \le r_{il} + |\operatorname{Re} C_{il}| + |\operatorname{Im} C_{il}| = r'_{il},$$

by the triangle inequality, (5.1) and $|a+ib| \le |a|+|b|$; off Z nothing changed. C is Hermitian, so $C_{li} = \overline{C_{il}}$, which has the same modulus and the same moduli of real and imaginary part; with $r_{li} = r_{il}$ this gives $(l,i) \in Z \iff (i,l) \in Z$ and $r'_{li} = r'_{il}$. On Z both C'_{il} and C'_{li} are $0 = \overline{0}$, and off Z they are C_{il} and $\overline{C_{il}}$, so C' is Hermitian; a diagonal entry of C is real and stays real or becomes 0.

(ii) For $(i,l) \in Z$, $|\operatorname{Re} C_{il}| + |\operatorname{Im} C_{il}| \le \sqrt{2}\,|C_{il}| \le \sqrt{2}\,r_{il}$ by the Cauchy–Schwarz inequality in $\mathbb{R}^2$ and the definition of Z; off Z, $r' = r$. Where $C_{il} = 0$ the added term is 0. An entry is structurally zero for an enclosure exactly when its centre and its radius are both 0 (Definition 5.5); such an entry lies in Z, keeps centre 0 and radius 0, and an entry with centre or radius not zero keeps a radius not zero or a centre not zero, since $r'_{il} \ge r_{il}$ and $C'_{il} \ne 0$ only off Z, where $C'_{il} = C_{il}$. Every entry of Z has centre 0 by construction.

(iii) Definition 5.3 asks of a reference that its blocks be Hermitian, positive semidefinite and complete at c, properties of G alone, and that each block be read with an entrywise enclosure with a Hermitian centre in $\mathbb{Q}(i)$ and symmetric non-negative rational radii, which (i) gives. The family G is unchanged. Every radius of (C,r) is 0 exactly when every radius of (C',r') is: if every $r_{il} = 0$, an entry lies in Z only where $C_{il} = 0$, so $r' = r$ by (ii); and $r' \ge r$ always.

(iv) The entries of $\frac{1}{2}(\tilde{C} + \tilde{C}^{\dagger})$ are $\frac{1}{2}(\tilde{C}_{il} + \overline{\tilde{C}_{li}}) \in \mathbb{Q}(i)$, the matrix is Hermitian, and $\frac{1}{2}(\tilde{r}_{il} + \tilde{r}_{li})$ is a non-negative rational, symmetric in (i,l). Since $X_{il} = \overline{X_{li}}$,

$$\left| X_{il} - \tfrac{1}{2}\big(\tilde{C}_{il} + \overline{\tilde{C}_{li}}\big) \right| \le \tfrac{1}{2}|X_{il} - \tilde{C}_{il}| + \tfrac{1}{2}\overline{|X_{li} - \tilde{C}_{li}|} \le \tfrac{1}{2}(\tilde{r}_{il} + \tilde{r}_{li}).$$

Where $\tilde{C}$ is Hermitian and $\tilde{r}$ symmetric, both averages return them unchanged. □

A declarer hands a reference beside a snap when it passes on the reference's centres and radii with it, unaltered. A block is evaluated off its snap when nothing its key length is computed from reads the snapped block, save the points and the values fixed before the first round.

A snap serves a run in two ways. A search reads the snapped family to find points, and the numbers from which the run computes its key length read it again at those points. The second reading is the one a premise about the snap is needed for. A run that holds a reference of the family can do the second reading on a family that carries the reference in place of the snap,

and then the snap only steers the search: the points it chose are points like any other, and a lower bound that holds at any admissible point holds at them. The definition names the two things a record states when a run does this: what the declarer passes on beside the snap, and on which blocks the snapped data are read by nothing the key length rests on. Which values a run of the engine fixes before its first round is listed in Assumption "Protocol parameters fixed before the run" (meat-core/ass:fixed); the definition names the values by that property and does not depend on the list.

Definition 5.7 (A reference handed beside a snap; a block evaluated off its snap)**.** Let F be a snap (Definition 5.5) of data D against a sector map with blocks $s \in \mathsf{S}$ (Definition 4.2), over an alphabet J, where D is an operator family read as a measurement, the data an instance states of it or a reference of it, or a producer's exact construction of what it emits (Definition 4.1).

(i) Where D is an operator family read as a measurement and the declarer holds a reference G of it (Definition 5.3), the declarer *hands the reference beside the snap* on a block s when it passes to every consumer of F, together with F and altered by no step, the centre $C_j^{(s)}$, with entries in $\mathbb{Q}(i)$, and the radii $r_j^{(s)}$, rationals, of an entrywise enclosure of $G_j^{(s)}$ for every $j \in J$, with the identity of the reference and the claims that establish it. The reference so passed is the *handed reference*, and s is a *handed block*.

(ii) A block s of F is *evaluated off its snap* in a run when the record of the run shows that no number from which the run computes its key length reads $F_j^{(s)}$ for any $j \in J$, save in two places: the points at which those numbers are evaluated, which any computation may have chosen, a computation on F among them; and every value the run fixes before its first round other than the output length, the acceptance set and the transcript length λ_{EC} among them, whatever they were computed from.

Uses Def. 4.1, Def. 4.2, Def. 5.3, Def. 5.5 · *Used by* Ass. 8.1, Req. 10.4, Req. 10.5, meat-core/frank-wolfe/req:bracket-on-reference, meat-core/rem:declared-measurement, receiver/detector-imperfections/rem:noise-floor-receiver, receiver/detector-imperfections/req:dark-composed-reference, reductions/cor:reference-companion, reductions/req:reference-companion · newly derived · implemented (effective: accepted) · *Code* certiqs_key.runs.direct_route:direct_route

6 Domination and its limits

A family G dominates a family F at a block weight q when G − (1−q)F is positive semidefinite for every outcome; G is then a mixture of F and a complete residual family, which is what lets a noise channel reduce G

to F plus flags of weight q. A dominated family adds no support, though it may lose some, and domination composes and survives trusted classical post-processing (Lemma 6.2); for a family that absorbs its residue in one element only that element constrains the weight (Lemma 6.3). A reference can dominate a lattice family at a weight below one only along rationally spanned directions (Lemma 6.4), and an exact decision on a rational enclosure gives a sufficient condition for the positivity of matrices with irrational entries (Lemma 6.5). No requirement relates a snap to its model by domination; the section is kept for a declared device weight. The closing remark explains what a weight does not control, why an idealised model with an element whose support has no rational direction needs a noise floor before it dominates a lattice family at a small weight, and that reductions other than domination are not bound by that obstruction.

A family G dominates a family F at block weights q when G − (1−q)F is positive semidefinite for every outcome on every block, one weight per block. The residual family is what G holds beyond the weighted F, rescaled by the weight.

Two exactly admissible measurements that are entrywise close can have key-rate bounds far apart, by an amount the distance between them does not control (Proposition "Instability of the rate under measurement perturbations" (meat-core/prop:optol-nobound)). A small change can alter which outcomes a measurement treats as impossible on some direction, and a bound on its size says nothing about that. Domination is a relation between two families under which the dominated family gains no support; it bounds the dominated family from above and not from below. It is the hypothesis under which a measurement G is a noise channel followed by F together with classical flags of weight at most q_s on block s [2, Thm. 3, Eq. (20)].

Definition 6.1 (Domination of one operator family by another at block weights)**.** Let $\mathfrak{m}$ be a sector map with blocks $s \in \mathsf{S}$ of dimensions d_s (Definition 4.2), let J be a finite non-empty alphabet, and let $G = \{G_j\}_{j\in J}$ and $F = \{F_j\}_{j\in J}$ be families of block lists against $\mathfrak{m}$ whose blocks are Hermitian and positive semidefinite; their entries are arbitrary complex numbers. A family is *complete at* a real number c on block s when $\sum_{j\in J} G_j^{(s)} = c\,\mathbb{1}_{d_s}$. For weights $q = (q_s)_{s\in\mathsf{S}}$ with $q_s \in [0, 1)$, the family G *dominates* F at q when

$$G_j^{(s)} - (1 - q_s)\, F_j^{(s)} \;\succeq\; 0 \qquad \text{for every } j \in J \text{ and } s \in \mathsf{S}. \tag{6.1}$$

The weight q_s belongs to the block s and is the same for every outcome. On a block s, the *residual family* of the domination is $R_j^{(s)} := q_s^{-1}\big(G_j^{(s)} - (1 - q_s)F_j^{(s)}\big)$ for $j \in J$ where $q_s > 0$, and $R_j^{(s)} := G_j^{(s)}$ where $q_s = 0$. For a Hermitian matrix X we write $\operatorname{supp} X$ for its range, the orthogonal complement of its kernel.

Uses Def. 4.2 · *Used by* Lem. 6.2, Lem. 6.3, Lem. 6.4, Rem. 6.6, reductions/domination/cor:dom-chain, reductions/domination/def:domination-link, reductions/domination/rem:rotated-projective, reductions/domination/thm:noise-channel · newly derived · accepted

Domination is upward closed in the weight, holds below one exactly when each support of F lies in that of G, is equality at weight zero, composes multiplicatively and survives classical post-processing. G is then a mixture of F and a complete residual family.

Lemma 6.2 (Elementary properties of domination)**.** Let $\mathfrak{m}$, J and the families $G = \{G_j\}_{j\in J}$, $F = \{F_j\}_{j\in J}$ and $E = \{E_j\}_{j\in J}$ of block lists against $\mathfrak{m}$ with Hermitian positive semidefinite blocks be as in Definition 6.1, and let $s \in \mathsf{S}$ be a block. Say that G dominates F at q_s on block s when (6.1) holds at that block.

(i) *(Upward closure.)* If G dominates F at q_s on block s and $q_s \le q_s' < 1$, then G dominates F at q_s' on block s.

(ii) *(Support.)* If G dominates F at some $q_s \in [0,1)$ on block s, then $\operatorname{supp} F_j^{(s)} \subseteq \operatorname{supp} G_j^{(s)}$ for every $j \in J$. Conversely, if $\operatorname{supp} F_j^{(s)} \subseteq \operatorname{supp} G_j^{(s)}$ for every $j \in J$, then G dominates F on block s at some weight in $[0,1)$.

(iii) *(Weight zero.)* If G and F are both complete at the same c on block s, then G dominates F at 0 on block s if and only if $F_j^{(s)} = G_j^{(s)}$ for every $j \in J$.

(iv) *(Composition.)* If G dominates F at q_s and F dominates E at q_s' on block s, then G dominates E on block s at $1-(1-q_s)(1-q_s')$, which lies in $[0,1)$ and is at most $q_s + q_s'$.

(v) *(Classical post-processing.)* Let W be a finite non-empty alphabet and let $P_{w|j} \ge 0$ be real numbers for $w \in W$ and $j \in J$. Put $G'_w \coloneqq \sum_{j\in J} P_{w|j} G_j$ and $F'_w \coloneqq \sum_{j\in J} P_{w|j} F_j$, block by block. If G dominates F at q_s on block s, then G' dominates F' at the same q_s on block s. If moreover $\sum_{w\in W} P_{w|j} = 1$ for every j and G and F are complete at c on block s, then so are G' and F'. In particular, multiplying each G_j and F_j by the same number $p_j \ge 0$ preserves domination at the same weight.

(vi) *(Residual family.)* If G and F are both complete at c on block s and G dominates F at q_s there, then the residual family $R^{(s)}$ of Definition 6.1 has positive semidefinite elements, is complete at c, and satisfies

$$G_j^{(s)} = (1-q_s)\, F_j^{(s)} + q_s\, R_j^{(s)} \qquad \text{for every } j \in J. \tag{6.2}$$

Uses Def. 4.2, Def. 6.1 · *Used by* Lem. 6.3, Lem. 6.4, Rem. 6.6, reductions/domination/rem:rotated-projective, reductions/domination/thm:noise-channel · newly derived · accepted

Proof. Throughout, fix the block s and drop the superscript (s).

(i) $G_j - (1-q'_s)F_j = \big(G_j - (1-q_s)F_j\big) + (q'_s - q_s)F_j$ is a sum of two positive semidefinite matrices, the second because $q'_s - q_s \geq 0$ and $F_j \succeq 0$.

(ii) Let $x \in \ker G_j$. Then $0 \leq \langle x, (G_j - (1-q_s)F_j)x\rangle = -(1-q_s)\langle x, F_j x\rangle \leq 0$, since $1-q_s > 0$ and $F_j \succeq 0$; so $\langle x, F_j x\rangle = 0$. For a positive semidefinite F_j this gives $\|F_j^{1/2}x\|^2 = 0$, hence $F_j x = F_j^{1/2}F_j^{1/2}x = 0$. So $\ker G_j \subseteq \ker F_j$, and taking orthogonal complements, $\operatorname{supp} F_j \subseteq \operatorname{supp} G_j$, both matrices being Hermitian. Conversely, suppose $\operatorname{supp} F_j \subseteq \operatorname{supp} G_j$ for every j. For each j with $F_j \neq 0$, $G_j \neq 0$ too; let Π_j be the orthogonal projector onto $\operatorname{supp} G_j$ and $\lambda_j > 0$ the least non-zero eigenvalue of G_j, so that $G_j \succeq \lambda_j \Pi_j$ by the spectral decomposition. Since $F_j \preceq \|F_j\|\,\mathbb{1}$ and F_j vanishes on $(\operatorname{supp} G_j)^\perp \subseteq \ker F_j$, we have $F_j = \Pi_j F_j \Pi_j \preceq \|F_j\|\,\Pi_j \preceq t_j G_j$ with $t_j \coloneqq \|F_j\|/\lambda_j > 0$. Put $q_s \coloneqq \max\big(\{0\} \cup \{1 - t_j^{-1} : F_j \neq 0\}\big)$, which lies in $[0,1)$. For $F_j = 0$, $G_j - (1-q_s)F_j = G_j \succeq 0$; for $F_j \neq 0$, $(1-q_s)t_j \leq 1$, so $(1-q_s)F_j \preceq (1-q_s)t_j G_j \preceq G_j$.

(iii) If $F_j = G_j$ for every j, (6.1) holds at $q_s = 0$ with equality. Conversely, if $G_j - F_j \succeq 0$ for every j, then $\sum_j (G_j - F_j) = c\,\mathbb{1} - c\,\mathbb{1} = 0$ is a sum of positive semidefinite matrices with trace 0; each summand has non-negative trace, so each has trace 0, and a positive semidefinite matrix of trace 0 is 0. So $F_j = G_j$ for every j.

(iv) Multiply $F_j - (1-q'_s)E_j \succeq 0$ by $1-q_s \geq 0$ and add $G_j - (1-q_s)F_j \succeq 0$: this gives $G_j - (1-q_s)(1-q'_s)E_j \succeq 0$, which is (6.1) for G and E at $1-(1-q_s)(1-q'_s)$. That weight equals $q_s + q'_s - q_s q'_s$, which is at most $q_s + q'_s$ and at least 0; and $(1-q_s)(1-q'_s) > 0$, so it is below 1.

(v) Every G'_w and F'_w is Hermitian and positive semidefinite, being a non-negative combination of such matrices, and $G'_w - (1-q_s)F'_w = \sum_j P_{w|j}\big(G_j - (1-q_s)F_j\big) \succeq 0$ for the same reason. If every $\sum_w P_{w|j} = 1$, then $\sum_w G'_w = \sum_j \big(\sum_w P_{w|j}\big)G_j = \sum_j G_j = c\,\mathbb{1}$, and the same for F'. The case p_j is $W = J$ with $P_{w|j} = p_j$ for $w = j$ and 0 otherwise.

(vi) If $q_s > 0$, then $R_j = q_s^{-1}(G_j - (1-q_s)F_j) \succeq 0$ by (6.1), $\sum_j R_j = q_s^{-1}\big(c\,\mathbb{1} - (1-q_s)c\,\mathbb{1}\big) = c\,\mathbb{1}$, and $(1-q_s)F_j + q_s R_j = G_j$ by the definition of R_j. If $q_s = 0$, then $R_j = G_j \succeq 0$ is complete at c, and $F_j = G_j$ by (iii), so $(1-0)F_j + 0 \cdot R_j = G_j$. □

When every element but one is rounded down below the reference element and the remaining one takes the rest, only that designated element constrains the weight. A weight below one exists exactly when the absorbed residue lies in its support, and it is at most residue norm over least eigenvalue.

A family that keeps every element but one below its reference element and lets one designated element take the rest is the simplest family a model

can dominate at a small weight. The lemma shows that the weight of the domination is then decided by the designated element alone, and how large it is.

Lemma 6.3 (Domination of a family that absorbs its residue in one element)**.** Let s be a block of dimension d_s, J a finite non-empty alphabet, $c > 0$, and $j^\star \in J$. Let $\{G_j\}_{j\in J}$ be Hermitian positive semidefinite $d_s \times d_s$ matrices with $\sum_{j\in J} G_j = c\,\mathbb{1}_{d_s}$, and let $\{F_j\}_{j\in J}$ be Hermitian matrices with

$$0 \preceq F_j \preceq G_j \quad (j \neq j^\star), \qquad F_{j^\star} = c\,\mathbb{1}_{d_s} - \sum_{j\neq j^\star} F_j. \tag{6.3}$$

Put $\Delta \coloneqq \sum_{j\neq j^\star}(G_j - F_j)$, the *absorbed residue*.

(i) $\Delta \succeq 0$, $F_{j^\star} = G_{j^\star} + \Delta \succeq G_{j^\star}$, every F_j is positive semidefinite, and $\sum_j F_j = c\,\mathbb{1}_{d_s}$.

(ii) For $q \in [0,1)$, $\{G_j\}$ dominates $\{F_j\}$ at q on this block (Definition 6.1) if and only if

$$q\,G_{j^\star} - (1-q)\,\Delta \;\succeq\; 0. \tag{6.4}$$

For every $j \neq j^\star$ and every $q \in [0,1)$, $G_j - (1-q)F_j \succeq 0$ holds.

(iii) Some $q \in [0,1)$ satisfies (6.4) if and only if $\operatorname{supp}\Delta \subseteq \operatorname{supp} G_{j^\star}$; this holds in particular when $G_{j^\star}$ is positive definite.

(iv) Suppose $\operatorname{supp}\Delta \subseteq \operatorname{supp} G_{j^\star}$, let $\lambda > 0$ be the least non-zero eigenvalue of $G_{j^\star}$ (its least eigenvalue when it is positive definite) and let $\delta \geq \|\Delta\|$. If $G_{j^\star} \neq 0$, then (6.4) holds for every q with $\delta/(\lambda+\delta) \leq q < 1$. If $\Delta = 0$, it holds at $q = 0$.

Uses Def. 6.1, Lem. 6.2 · *Used by* Rem. 6.6, reductions/domination/rem:rotated-projective · newly derived · accepted

Proof. (i) Each $G_j - F_j$ with $j \neq j^\star$ is positive semidefinite by (6.3), so $\Delta \succeq 0$. By completeness of $\{G_j\}$, $G_{j^\star} = c\,\mathbb{1}_{d_s} - \sum_{j\neq j^\star} G_j$, and subtracting this from the definition of $F_{j^\star}$ gives $F_{j^\star} - G_{j^\star} = \sum_{j\neq j^\star}(G_j - F_j) = \Delta$. So $F_{j^\star} = G_{j^\star} + \Delta \succeq G_{j^\star} \succeq 0$; the other F_j are positive semidefinite by (6.3), and $\sum_j F_j = c\,\mathbb{1}_{d_s}$ by the definition of $F_{j^\star}$.

(ii) For $j \neq j^\star$, $G_j - (1-q)F_j = (G_j - F_j) + qF_j$ is a sum of two positive semidefinite matrices, by (6.3) and $q \geq 0$. For $j^\star$, by (i), $G_{j^\star} - (1-q)F_{j^\star} = G_{j^\star} - (1-q)(G_{j^\star} + \Delta) = qG_{j^\star} - (1-q)\Delta$. So (6.1) holds at every outcome if and only if it holds at $j^\star$, which is (6.4).

(iii) If (6.4) holds for some $q < 1$, then, by (ii), $\{G_j\}$ dominates $\{F_j\}$ at q, and Lemma 6.2 (ii) gives $\operatorname{supp} F_{j^\star} \subseteq \operatorname{supp} G_{j^\star}$. Since $0 \preceq \Delta \preceq \Delta + G_{j^\star} = F_{j^\star}$, every $x \in \ker F_{j^\star}$ satisfies $0 \leq \langle x, \Delta x\rangle \leq \langle x, F_{j^\star} x\rangle = 0$, so $\Delta x = 0$ as in the proof of Lemma 6.2 (ii); hence $\operatorname{supp}\Delta \subseteq \operatorname{supp} F_{j^\star} \subseteq \operatorname{supp} G_{j^\star}$. The converse

is (iv) when $G_{j^\star} \neq 0$ and $q = 0$ when $\Delta = 0$; if $G_{j^\star} = 0$, the inclusion forces $\Delta = 0$. A positive definite $G_{j^\star}$ has support the whole space.

(iv) If $\Delta = 0$, (6.4) at $q = 0$ reads $0 \succeq 0$. Now let $G_{j^\star} \neq 0$, and let Π be the orthogonal projector onto $\operatorname{supp} G_{j^\star}$. By the spectral decomposition $G_{j^\star} \succeq \lambda\Pi$. Since $\operatorname{supp}\Delta \subseteq \operatorname{supp} G_{j^\star}$ and Δ is Hermitian, $\Delta = \Pi\Delta\Pi \preceq \|\Delta\|\,\Pi \preceq \delta\,\Pi$. For $q \geq \delta/(\lambda+\delta)$ we have $q\lambda \geq (1-q)\delta$, so

$$qG_{j^\star} - (1-q)\Delta \;\succeq\; q\lambda\,\Pi - (1-q)\delta\,\Pi \;=\; \big(q\lambda - (1-q)\delta\big)\Pi \;\succeq\; 0.$$

Since $\lambda > 0$, $\delta/(\lambda+\delta) < 1$, so such q exist in $[0,1)$. □

A family with Gaussian-rational entries that G dominates at a weight below one has every support inside the largest rational subspace of the matching support of G. With completeness the weight is at least the largest value of G on the complement of that subspace, over the constant.

A lattice family has entries in the Gaussian rationals $\mathbb{Q}(i) = \{x + iy : x, y \in \mathbb{Q}\}$. The lemma shows that such a family can be dominated by a reference element, at a weight below one, only along directions that rational vectors span, whatever its lattice exponent. Domination itself does not depend on the basis, since a unitary applied to both families preserves it; what depends on the basis is the lattice family, whose entries are Gaussian rationals in the basis in which it is declared. The lemma bounds the domination of such a family and nothing else. A reduction that first applies a channel to the reference, such as a unitary, changes the family that is compared with a lattice family: the lemma then applies to the image of the reference, whose supports may have rational directions that those of the reference lacked (Remark 6.6).

Lemma 6.4 (Rationality obstruction to the domination of a lattice family)**.** Let $d \geq 1$, and read every vector of $\mathbb{C}^d$ and every $d \times d$ matrix in the standard basis. Call a subspace U of $\mathbb{C}^d$ $\mathbb{Q}(i)$*-rational* when it is spanned over $\mathbb{C}$ by vectors of $\mathbb{Q}(i)^d$ (the subspace $\{0\}$ is spanned by the empty set), and for a subspace V of $\mathbb{C}^d$ put

$$\mathcal{R}(V) \;\coloneqq\; \operatorname{span}_{\mathbb{C}}\big(V \cap \mathbb{Q}(i)^d\big), \tag{6.5}$$

the *largest* $\mathbb{Q}(i)$*-rational subspace of* V.

(i) $\mathcal{R}(V)$ is a $\mathbb{Q}(i)$-rational subspace contained in V, and it contains every $\mathbb{Q}(i)$-rational subspace contained in V.

(ii) For every $d \times d'$ matrix A with entries in $\mathbb{Q}(i)$, the range of A is $\mathbb{Q}(i)$-rational. In particular the support of a Hermitian $d \times d$ matrix with entries in $\mathbb{Q}(i)$ is.

(iii) Let $\{G_j\}_{j\in J}$ and $\{F_j\}_{j\in J}$ be Hermitian positive semidefinite $d \times d$ matrices over a finite non-empty alphabet J, every F_j with entries in $\mathbb{Q}(i)$, such that $G_j - (1-q)F_j \succeq 0$ for every j, for some $q \in [0,1)$. Then $\operatorname{supp} F_j \subseteq \mathcal{R}_j \coloneqq \mathcal{R}(\operatorname{supp} G_j)$ for every j.

(iv) If, in (iii), moreover $\sum_j G_j = \sum_j F_j = c\,\mathbb{1}_d$ with $c > 0$, then for every $j \in J$

$$q \;\geq\; \frac{\lambda_{\max}\big(P_j G_j P_j\big)}{c}, \tag{6.6}$$

where P_j is the orthogonal projector onto $\mathcal{R}_j^{\perp}$.

(v) In particular, if $\operatorname{supp} G_j$ contains no non-zero vector of $\mathbb{Q}(i)^d$, then $q \geq \lambda_{\max}(G_j)/c$, which equals $\operatorname{tr} G_j / c$ when G_j has rank one.

Uses Def. 6.1, Lem. 6.2 · *Used by* Rem. 6.6, receiver/detector-imperfections/rem:noise-floor-receiver, reductions/domination/rem:rotated-projective · newly derived · accepted

Proof. (i) $V \cap \mathbb{Q}(i)^d \subseteq V$ and V is a subspace, so the span lies in V; it is spanned by vectors of $\mathbb{Q}(i)^d$ by its definition. If $U \subseteq V$ is spanned by vectors $u_1, \ldots, u_m \in \mathbb{Q}(i)^d$, each u_l lies in $U \cap \mathbb{Q}(i)^d \subseteq V \cap \mathbb{Q}(i)^d$, so $U \subseteq \mathcal{R}(V)$.

(ii) The range of A is $\{Ax : x \in \mathbb{C}^{d'}\}$, and $Ax = \sum_l x_l A e_l$, so it is the span of the columns $Ae_1, \ldots, Ae_{d'}$ of A, each a vector of $\mathbb{Q}(i)^d$. For a Hermitian matrix the support is the range (Definition 6.1).

(iii) By Lemma 6.2 (ii), applied to the one-block families $\{G_j\}$ and $\{F_j\}$ at weight $q < 1$, $\operatorname{supp} F_j \subseteq \operatorname{supp} G_j$. By (ii) $\operatorname{supp} F_j$ is $\mathbb{Q}(i)$-rational, so by (i) it lies in $\mathcal{R}(\operatorname{supp} G_j)$.

(iv) Fix j. Summing $G_l - (1-q)F_l \succeq 0$ over $l \neq j$ and using both completeness identities,

$$0 \;\preceq\; \big(c\,\mathbb{1}_d - G_j\big) - (1-q)\big(c\,\mathbb{1}_d - F_j\big) \;=\; qc\,\mathbb{1}_d - G_j + (1-q)F_j.$$

Let $\varphi \in \mathcal{R}_j^{\perp}$ be a unit vector. By (iii), $\operatorname{supp} F_j \subseteq \mathcal{R}_j$, so φ is orthogonal to the range of the Hermitian matrix F_j and therefore lies in its kernel; hence $\langle \varphi, F_j \varphi \rangle = 0$, and the display gives $\langle \varphi, G_j \varphi \rangle \leq qc$. Now let $x \in \mathbb{C}^d$ be any unit vector. If $P_j x = 0$, then $\langle x, P_j G_j P_j x \rangle = 0 \leq qc$. Otherwise $\varphi := P_j x / \|P_j x\|$ is a unit vector of $\mathcal{R}_j^{\perp}$ and $\|P_j x\| \leq 1$, so $\langle x, P_j G_j P_j x \rangle = \|P_j x\|^2 \langle \varphi, G_j \varphi \rangle \leq qc$. Since $P_j G_j P_j$ is Hermitian, its largest eigenvalue is the maximum of $\langle x, P_j G_j P_j x \rangle$ over unit vectors x, which is therefore at most qc.

(v) Under the hypothesis, $\operatorname{supp} G_j \cap \mathbb{Q}(i)^d = \{0\}$, so $\mathcal{R}_j = \{0\}$ and $P_j = \mathbb{1}_d$, and (6.6) reads $q \geq \lambda_{\max}(G_j)/c$. A positive semidefinite matrix of rank one has one non-zero eigenvalue, which is its trace. □

A Hermitian matrix is at least μ times the identity when a Gaussian-rational centre within entrywise radii r is at least $\rho + \mu$ times it, ρ the largest row sum of r. Every hypothesis is a comparison of rationals, so matrices with irrational entries are decided in exact arithmetic.

A matrix with irrational entries, such as one with entries $S_{il}/\sqrt{n_i n_l}$, is not a matrix of rationals, and an exact decision of its positive semidefiniteness cannot read its entries directly. The lemma replaces the decision by one

on a rational centre, at the price of a margin. It is a sufficient condition only: a matrix whose least eigenvalue is smaller than the uncertainty of its enclosure is in general not certified, and a finer enclosure is the remedy.

Lemma 6.5 (Positive semidefiniteness from an entrywise enclosure)**.** (i) Let X be a Hermitian complex $d\times d$ matrix, X_c a Hermitian $d\times d$ matrix with entries in $\mathbb{Q}(i)$, and $r_{il}\ge 0$ rationals with $|X_{il}-(X_c)_{il}|\le r_{il}$ for all i,l. Let ρ be a rational with

$$\rho \;\ge\; \max_i \sum_l r_{il} \qquad \text{or} \qquad \rho\ge 0 \;\text{ and }\; \rho^2 \;\ge\; \sum_{i,l} r_{il}^2, \tag{6.7}$$

and $\mu\ge 0$ a rational. If $X_c-(\rho+\mu)\,\mathbb{1}_d\succeq 0$, then $X\succeq \mu\,\mathbb{1}_d$; in particular $X\succeq 0$, and X is positive definite when $\mu>0$. If $X_c-\rho\,\mathbb{1}_d$ is positive definite, then X is positive definite. Every hypothesis on X_c, r, ρ and μ is a property of finitely many rationals and Gaussian rationals.

(ii) Let $n_1,\dots,n_d$ be positive rationals and $D\coloneqq \operatorname{diag}(\sqrt{n_1},\dots,\sqrt{n_d})$, and let a_i,e_i be rationals with $0\le e_i\le a_i$ and

$$(a_i-e_i)^2\,n_i \;\le\; 1 \;\le\; (a_i+e_i)^2\,n_i. \tag{6.8}$$

Then $|a_i-n_i^{-1/2}|\le e_i$. Let S and Y be Hermitian $d\times d$ matrices with entries in $\mathbb{Q}(i)$, put $X\coloneqq D^{-1}SD^{-1}-Y$, $A\coloneqq\operatorname{diag}(a_1,\dots,a_d)$, $X_c\coloneqq ASA-Y$ and

$$r_{il} \;\coloneqq\; \sigma_{il}\,\big(a_ie_l+e_ia_l+e_ie_l\big), \qquad \sigma_{il}\coloneqq|\operatorname{Re}S_{il}|+|\operatorname{Im}S_{il}|. \tag{6.9}$$

Then X, X_c and r satisfy the hypotheses of (i) on X, X_c and r, and the radii do not depend on Y.

(iii) Let X, X_c and r be as in (i), let F be a Hermitian positive semidefinite $d\times d$ matrix with entries in $\mathbb{Q}(i)$, and let $q\in[0,1]$ be rational. Then $X-(1-q)F$, $X_c-(1-q)F$ and r satisfy the hypotheses of (i) on X, X_c and r. If, for some ρ and μ as in (i), $X_c-(1-q)F-(\rho+\mu)\mathbb{1}_d\succeq 0$, then $X_c-(1-q')F-(\rho+\mu)\mathbb{1}_d\succeq 0$ for every rational $q'\in[q,1]$.

Used by receiver/thm:exact-and-evaluated · newly derived · implemented · *Code* certiqs_key.foundations.reference:enclosure_psd

Proof. (i) Put $Z\coloneqq X-X_c$, a Hermitian matrix, so $|Z_{il}|=|Z_{li}|$. Then $|Z_{il}|\le r'_{il}\coloneqq\min(r_{il},r_{li})$, and r' is symmetric with $\sum_l r'_{il}\le\sum_l r_{il}$ for every i. Let $x\in\mathbb{C}^d$ be a unit vector. If $\rho\ge\max_i\sum_l r_{il}$, then, using $|x_i||x_l|\le\frac12(|x_i|^2+|x_l|^2)$ and the symmetry of r',

$$|\langle x,Zx\rangle|\le\sum_{i,l}|x_i|\,r'_{il}\,|x_l|\le\tfrac12\sum_{i,l}r'_{il}\big(|x_i|^2+|x_l|^2\big)=\sum_i|x_i|^2\sum_l r'_{il}\le\max_i\sum_l r_{il}\le\rho.$$

If instead $\rho \geq 0$ and $\rho^2 \geq \sum_{i,l} r_{il}^2$, the Cauchy–Schwarz inequality on the d^2 numbers r_{il} and $|x_i||x_l|$ gives

$$|\langle x, Zx\rangle| \leq \sum_{i,l} r_{il}\,|x_i||x_l| \leq \Big(\sum_{i,l} r_{il}^2\Big)^{1/2}\Big(\sum_{i,l} |x_i|^2|x_l|^2\Big)^{1/2} = \Big(\sum_{i,l} r_{il}^2\Big)^{1/2} \leq \rho,$$

since $\sum_{i,l} |x_i|^2|x_l|^2 = \big(\sum_i |x_i|^2\big)^2 = 1$. In both cases $\langle x, Xx\rangle = \langle x, X_c x\rangle + \langle x, Zx\rangle \geq (\rho+\mu) - \rho = \mu$, because $X_c - (\rho+\mu)\mathbb{1}_d \succeq 0$. As X is Hermitian and x was an arbitrary unit vector, $X \succeq \mu\,\mathbb{1}_d$. If instead $X_c - \rho\,\mathbb{1}_d$ is positive definite, the same bounds on $|\langle x, Zx\rangle|$ give $\langle x, Xx\rangle \geq \langle x, X_c x\rangle - \rho = \langle x, (X_c - \rho\,\mathbb{1}_d)x\rangle > 0$ for every unit vector x, so X is positive definite. The hypotheses are a finite set of comparisons of rationals and the positive semidefiniteness of the matrix $X_c - (\rho+\mu)\mathbb{1}_d$ with entries in $\mathbb{Q}(i)$.

(ii) The map $t \mapsto t^2 n_i$ is strictly increasing on $[0,\infty)$, and $a_i - e_i \geq 0$, $n_i^{-1/2} > 0$, $(n_i^{-1/2})^2 n_i = 1$; so (6.8) gives $a_i - e_i \leq n_i^{-1/2} \leq a_i + e_i$. Write $n_i^{-1/2} = a_i + \varepsilon_i$ with $|\varepsilon_i| \leq e_i$. The entry (i,l) of $D^{-1}SD^{-1}$ is $S_{il}(a_i+\varepsilon_i)(a_l+\varepsilon_l)$, and that of ASA is $S_{il}a_i a_l$, so

$$|X_{il} - (X_c)_{il}| = |S_{il}|\,\big|a_i\varepsilon_l + \varepsilon_i a_l + \varepsilon_i\varepsilon_l\big| \leq \sigma_{il}\,\big(a_i e_l + e_i a_l + e_i e_l\big) = r_{il},$$

using $a_i, a_l \geq 0$ and $|z| \leq |\mathrm{Re}\,z| + |\mathrm{Im}\,z|$. The matrix X is Hermitian, D^{-1} being real diagonal and S, Y Hermitian; X_c is Hermitian with entries in $\mathbb{Q}(i)$ for the same reasons; the r_{il} are non-negative rationals, and Y does not enter them.

(iii) $X-(1-q)F$ is Hermitian, and $X_c-(1-q)F$ is Hermitian with entries in $\mathbb{Q}(i)$, q being rational and F having entries in $\mathbb{Q}(i)$. Since $\big(X-(1-q)F\big)_{il} - \big(X_c-(1-q)F\big)_{il} = X_{il}-(X_c)_{il}$, the radii r still bound the differences entry by entry. Finally, $X_c-(1-q')F-(\rho+\mu)\mathbb{1}_d = \big(X_c-(1-q)F-(\rho+\mu)\mathbb{1}_d\big)+(q'-q)F$ is a sum of two positive semidefinite matrices, since $q'-q \geq 0$ and $F \succeq 0$. □

A weight at which a model dominates a lattice family shows that the family adds no support, not how far it lowers an element. An element with no rational direction in its support forces a weight of at least its top eigenvalue over the constant; noise in the model removes that bound.

No requirement of the engine relates a snapped family to its model by domination: a run evaluates the numbers its key length rests on with the model in place of the snap where it holds one, and rests on a named premise elsewhere. The remark says what domination would control if a weight were declared or established, and why an idealised model could not be related to a lattice family that way at a small weight. Reductions other than domination that could reduce an idealised model exactly, and a declared device weight, are future scope (`spec/future-scope.md` §13).

Remark 6.6 (What domination controls, and why it needs a noise floor). Let G be a reference (Definition 5.3) and F a family of the same shape whose entries

are lattice values, both complete at c, with G dominating F at a weight $q < 1$ (Definition 6.1). The relation bounds every element of F from above by its reference element, $(1-q)F_j \preceq G_j$. What that bound controls, and what it does not, comes first. Three facts then decide when the weight can be small, and together they explain when an idealised model whose elements are rank-deficient dominates no lattice family at a small weight, what a declaration should do instead, and why that obstruction belongs to domination and not to every reduction.

What domination controls. The relation of Definition 6.1 is the hypothesis under which the reference is a noise channel followed by F, together with classical flags of weight at most q_s on block s [2, Thm. 3, Eq. (20)]. Domination bounds each element of F from above and not from below: F may lower the probability of an outcome, and may remove support, making impossible an outcome that the model makes possible, at a weight below the lattice step. For one block of dimension one with $c = 1$, $G_1 = 2^{-k}/3$ and $G_2 = 1 - 2^{-k}/3$, the only lattice value at k in $[0, 2^{-k}/3]$ is 0, and the family $F_1 = 0$, $F_2 = 1$ that rounds G_1 down and lets G_2 take the rest is dominated at every $q \geq 2^{-k}/3$, since $G_1 - (1-q)F_1 = G_1 \succeq 0$ and $G_2 - (1-q)F_2 = q - 2^{-k}/3$, although outcome 1 has become impossible.

Support. At a weight below one, an element of F has no support outside that of its reference element (Lemma 6.2 (ii)): for x in the kernel of G_j, $0 \leq \langle x, (G_j - (1-q)F_j)x\rangle = -(1-q)\langle x, F_j x\rangle$, so $F_j x = 0$. An outcome that the model makes impossible on some direction stays exactly impossible there, so an element of F must fit inside the support of its reference element, and that is what a lattice family can fail to do at a small weight.

Rationality. A lattice family has Gaussian-rational entries in the basis in which it is declared, so the support of each of its elements is spanned by Gaussian-rational vectors. A dominated element therefore lies in the largest rational subspace $\mathcal{R}_j$ of the support of G_j (Lemma 6.4 (iii)), and completeness turns this into the lower bound $q \geq \lambda_{\max}(P_j G_j P_j)/c$, with P_j the projector onto the orthogonal complement of $\mathcal{R}_j$ (Lemma 6.4 (iv)). The bound depends on the basis in which the lattice family is declared and not on the lattice exponent: a finer lattice still has Gaussian-rational entries, so no refinement of the lattice lowers it.

An example at two photons. Take two orthogonal polarisation modes H and V and the two-photon block in the orthonormal occupation basis $|2,0\rangle, |1,1\rangle, |0,2\rangle$. With the diagonal mode created by $a_D^\dagger = (a_H^\dagger + a_V^\dagger)/\sqrt{2}$, the state with both photons in that mode is

$$|DD\rangle = \tfrac{1}{\sqrt{2}}\left(a_D^\dagger\right)^2 |0,0\rangle = \tfrac{1}{2}|2,0\rangle + \tfrac{1}{\sqrt{2}}|1,1\rangle + \tfrac{1}{2}|0,2\rangle,$$

since $(a_H^\dagger)^2 |0,0\rangle = \sqrt{2}\,|2,0\rangle$ and $a_H^\dagger a_V^\dagger |0,0\rangle = |1,1\rangle$. For an ideal, lossless analyser in the diagonal basis without dark counts, the element of the outcome in which only the detector of D clicks is $|DD\rangle\langle DD|$ on this block. Every non-zero vector on that ray has second coordinate $\sqrt{2}$ times its first, so none has Gaussian-rational coordinates, $\mathcal{R}_j = \{0\}$, and Lemma 6.4 (v) gives $q \geq \operatorname{tr}|DD\rangle\langle DD|/c = 1/c$: with $c = 1$ no lattice family declared in this basis is dominated by the model at a weight below one, whatever the lattice exponent. Such irrational directions are common in photon-number blocks of two or more photons, because the occupation basis carries factors $1/\sqrt{n}$; the diagonal basis already produces one.

A noise floor removes the obstruction. A physical detector has loss and dark counts. In a model that includes both in every port, no outcome is impossible on a photon-number block, and, under conditions that the module modelling the

detector states, every element is positive definite on every block. Supports are then no obstruction: let every element of F but one lie between zero and its reference element and let the remaining one, chosen positive definite in the reference, take what completeness leaves; the reference then dominates F at a weight of at most $\delta/(\lambda+\delta)$, where δ is the norm of the residue the remaining element absorbs and λ the least eigenvalue of the reference element at that outcome, not of the element of F, which the residue raises above it (Lemma 6.3 (iv)). When the other elements stay within a few lattice steps of their reference elements, δ is of the order of the lattice step times the number of outcomes and the dimension, and the weight is of the order of that over λ. Loss alone or dark counts alone need not suffice: what matters is that no element is rank-deficient on a block, and a model in which some outcome is still impossible on some block keeps the obstruction for that element.

Declaring the floor. A weight relates F to the model G and says nothing about whether G describes the device; that is the premise of the declaration's own model. A larger floor makes λ larger and the weight smaller, which is a temptation and not a licence: a floor above the noise the device has makes the model wrong, an error no weight can show. The floor to declare is the smallest loss and dark-count rates one is sure the device has.

Rank deficiency alone is no obstruction. A block whose elements are rank-deficient but have supports spanned by Gaussian-rational vectors may be dominated by its model at a weight of the order of the lattice step. For one block of dimension two with $c=1$ and $G_1=\operatorname{diag}(3/5,0)$, $G_2=\operatorname{diag}(2/5,0)$, $G_3=\operatorname{diag}(0,1)$, put $a \coloneqq \lfloor 2^k\cdot 3/5\rfloor\, 2^{-k}$ and $F_1=\operatorname{diag}(a,0)$, $F_2=\operatorname{diag}(1-a,0)$, $F_3=G_3$. Then $G_1-(1-q)F_1$ and $G_3-(1-q)F_3$ are positive semidefinite for every $q\in[0,1)$, and $G_2-(1-q)F_2\succeq 0$ exactly when $q\geq(3/5-a)/(1-a)$, which is below $\frac{5}{2}\,2^{-k}$ because $0\leq 3/5-a<2^{-k}$ and $1-a>2/5$, although no element is positive definite. Where the support of some element contains no non-zero Gaussian-rational vector, no lattice family is dominated below the bound of Lemma 6.4 (v) at any lattice exponent; there it is the noise floor, not a finer lattice, that makes a small weight possible.

What the obstruction does not say. Domination compares the reference with F element by element on the same block, and the channel it yields acts on each block as the identity followed by a branching into F and flags. A unitary applied to both families preserves domination; what fixes the basis is the lattice family, Gaussian-rational in the basis in which it is declared. Lemma 6.4 bounds the domination of such a family and nothing else. A reduction that first applies another channel can map some idealised models exactly onto a family with Gaussian-rational entries, with no direction to snap at all. An ideal analyser whose analysed polarisation $\cos\alpha\,|H\rangle+\sin\alpha\,|V\rangle$ has an irrational $\tan\alpha$, so that its one-photon element has no Gaussian-rational direction in its support, is the image, under one unitary applied to the input of every outcome, of the same analyser at an angle at which its elements are lattice values, such as $\alpha=0$. On the block of m photons in two modes, the isometry from that block into the m-fold tensor power of $\mathbb{C}^2$, which maps $|2,0\rangle$, $|1,1\rangle$ and $|0,2\rangle$ to $|HH\rangle$, $(|HV\rangle+|VH\rangle)/\sqrt{2}$ and $|VV\rangle$, carries $|DD\rangle$ of the example above to $\frac{1}{2}(|HH\rangle+|HV\rangle+|VH\rangle+|VV\rangle)$, which has rational coordinates, at dimension 2^m instead of $m+1$. Such reductions are not claims of this module; an idealised model that dominates no lattice family at a small weight may still be reducible exactly by them.

Uses Def. 5.3, Def. 6.1, Lem. 6.2, Lem. 6.3, Lem. 6.4 · *Used by* meat-core/rem:declared-measurement, receiver/detector-imperfections/rem:noise-floor-receiver · newly derived · accepted

7 The exact island, enclosures and the tiers

The exact island is where the engine computes on exact rationals, namely every decision that data satisfy the conditions their declaration states, a snap's decisions on its output and the decisions on a reference among them, a producer's construction of its emission, the exact establishment of the acceptance-side iterate and of the acceptance-side minimum over the acceptance set as its consumer reads it, and the certification of a lower bound on the feasible-side minimum. A least eigenvalue is bounded from below exactly from any approximate eigenbasis, by a Gershgorin bound corrected for the basis' defect from orthogonality (Lemma 7.2). The bound moves across a ball around an exact matrix, and an entrywise enclosure fixes the rank of every matrix it contains once its exactly zero rows are dropped and the rest is shown positive definite (Corollary 7.3), a factor enclosure fixes the rank and support of each of its members once the Gram matrix of its columns is shown positive definite throughout (Corollary 7.4), and powers of positive definite matrices and their divided-difference maps move by explicit constants under a perturbation (Lemma 7.5). A reported key rate carries one of three tiers, estimate, first-order or certified, and no other.

The exact island holds every decision that data meet their declared conditions, those on a reference among them, a producer's construction of its emission, and certificates of the optimiser's two linear minima. The tolerance ceiling, the rest of the optimiser and binary64 proposals are outside it.

A consumer replaces an emitted scalar bound only under the exception of Requirement 9.2, and the decision against the declared tolerance, which the definition places off the island, is the one that Requirement 10.1 specifies.

Definition 7.1 (The exact island)**.** The *exact island* is the following set of computations of the engine, the declarations of its instances included.

(i) *The admission boundary*: every decision that data satisfy a condition their declaration states of them — among them the conditions of Definition 4.3, a trace, a column sum, an identity between declared operators and a lattice condition — taken by a consumer of declared data it reads, or by a declarer, an instance or a producer (Definition 4.1), of data it is about to declare; the decisions by which a snap establishes the conditions of its output (Definition 5.5) are among the latter. So are, for a reference of an operator family that a declarer uses or hands beside its snap (Definition 5.3), the computation of the centres and radii of its entrywise enclosures, whose enclosure of the

reference, (5.1), the claims of the module that constructs the reference establish, the decisions on those centres and radii and, for an exact reference, on its positivity and completeness; and, for any snap of an operator family read as a measurement, the decisions by which the snap keeps a block, and the decisions on which entries are structurally zero. It contains, further, a producer's construction, block by block, of the operator data it emits, and, for each scalar bound that a consumer replaces as in Definition 4.1 (iii), the decision that the replacement lies on the weakening side of the bound emitted.

(ii) *The acceptance side of the optimisation*, the honest model and the completeness budget being declared as exact rational data: the construction of an acceptance set from them; the exact establishment that the optimiser's acceptance-side iterate lies in the acceptance set it reads, for its initial point and for each update as a convex combination, with exact rational weights, of points so established; and the exact certification of the minimum of a linear functional over that set, or of a lower bound on that minimum. The acceptance set the optimiser reads is the set so declared: the constructed set as its producer emits it, with each scalar bound that a consumer has replaced as in Definition 4.1 (iii) at its replaced value.

(iii) *The feasible side of the optimisation*: the certification of a lower bound on the minimum of a linear functional over the set the optimiser's linear oracle is asked to minimise over, formed from the functional, from the data of that set and from multipliers, each given to it as values, together with the bounds on least eigenvalues that the certification computes on the way.

The decision whether data an instance is about to declare lie within the declared tolerance (Definition 5.1) is not on the exact island. The optimiser's iterate apart from its acceptance side, the optimiser's oracle, the objective and its gradient lie outside the exact island, and so does a binary64 computation that only proposes a candidate, such as a point or a multiplier, for a decision or a certification of the island. A certification of (iii) holds of the functional and the data as they are given to it, and not of the quantities that a binary64 computation of them approximates.

Uses Def. 4.1, Def. 4.3, Def. 5.1, Def. 5.3, Def. 5.5 · *Used by* Req. 10.5, Req. 12.1, meat-core/frank-wolfe/req:verified-feasible-side, receiver/detector-imperfections/req:dark-composed-reference, reductions/req:reference-companion · newly derived · accepted

For a Hermitian matrix and any real V whose Gram matrix lies within $\theta < 1$ of the identity in Frobenius norm, the Gershgorin bound γ of the real embedding conjugated by V gives $\lambda_min \geq \gamma/(1+\theta)$, or $\gamma/(1-\theta)$ if $\gamma < 0$. On rational data every step is exact, and V decides tightness only.

An eigensolver in binary64 returns approximate eigenvectors, and nothing is established of them by the arithmetic that produced them. The lemma turns any such basis into a lower bound on the least eigenvalue that holds exactly: it asks of the basis only a bound on its defect from orthogonality, so the quality of the basis decides how close the bound lies to the least eigenvalue and never whether it is one. The real embedding lets the computation run over the reals, a Hermitian matrix having the Rayleigh quotients of its embedding, and on rational data every quantity is rational, so the bound is a certification of the exact island (Definition 7.1).

Lemma 7.2 (Least-eigenvalue lower bound from an approximate eigenbasis)**.** Let $n \geq 1$, let R be a Hermitian complex $n \times n$ matrix, and let

$$\widetilde{R} \coloneqq \begin{pmatrix} \operatorname{Re} R & -\operatorname{Im} R \\ \operatorname{Im} R & \operatorname{Re} R \end{pmatrix} \in \mathbb{R}^{2n \times 2n}, \tag{7.1}$$

the real and imaginary parts taken entrywise. Let V be any real $2n \times 2n$ matrix, put

$$B \coloneqq V^{\mathsf{T}} \widetilde{R} V, \qquad E \coloneqq V^{\mathsf{T}} V - \mathbb{1}_{2n}, \qquad \gamma \coloneqq \min_{1 \leq i \leq 2n} \Big(B_{ii} - \sum_{l \neq i} |B_{il}| \Big),$$

and let θ be a real number with $0 \leq \theta < 1$ and $\sum_{i,l} E_{il}^2 \leq \theta^2$.

(i) $\widetilde{R}$ is real symmetric, and $\lambda_{\min}(\widetilde{R}) = \lambda_{\min}(R)$.

(ii) V is invertible, and

$$\lambda_{\min}(R) \;\geq\; \begin{cases} \gamma/(1+\theta), & \gamma \geq 0, \\ \gamma/(1-\theta), & \gamma < 0. \end{cases} \tag{7.2}$$

(iii) If the entries of R lie in $\mathbb{Q}(i)$, the entries of V are rational and θ is rational, then $\widetilde{R}$, B and E have rational entries, γ is rational, the hypothesis on θ is a comparison of rationals, and the right-hand side of (7.2) is a rational number.

(iv) Applied to $-R$, (ii) bounds the largest eigenvalue of R, which is $-\lambda_{\min}(-R)$, from above.

Used by Cor. 7.3, meat-core/frank-wolfe/req:certified-evaluation, meat-core/frank-wolfe/req:verified-feasible-side, receiver/req:rigorous-weight, receiver/thm:exact-and-evaluated · newly derived · implemented · *Code* certiqs_key.foundations.dyadic:least_eigenvalue_bound

Proof. (i). Write $A \coloneqq \operatorname{Re} R$ and $C \coloneqq \operatorname{Im} R$. Since R is Hermitian, $\overline{R_{li}} = R_{il}$ for all i, l, so $A_{li} = A_{il}$ and $C_{li} = -C_{il}$: A is symmetric and C is antisymmetric. Hence $\widetilde{R}^{\mathsf{T}} = \begin{pmatrix} A^{\mathsf{T}} & C^{\mathsf{T}} \\ -C^{\mathsf{T}} & A^{\mathsf{T}} \end{pmatrix} = \begin{pmatrix} A & -C \\ C & A \end{pmatrix} = \widetilde{R}$. For $x = \begin{pmatrix} a \\ b \end{pmatrix} \in \mathbb{R}^{2n}$, with $a, b \in \mathbb{R}^n$, put $u \coloneqq a + ib \in \mathbb{C}^n$. Then $Ru = (Aa - Cb) + i(Ca + Ab)$ and

$$u^{\dagger} R u = a^{\mathsf{T}}(Aa - Cb) + b^{\mathsf{T}}(Ca + Ab) + i\big(a^{\mathsf{T}}(Ca + Ab) - b^{\mathsf{T}}(Aa - Cb)\big).$$

The number $u^{\dagger}Ru$ is real, R being Hermitian, so it equals its real part $a^{\mathsf{T}}Aa - a^{\mathsf{T}}Cb + b^{\mathsf{T}}Ca + b^{\mathsf{T}}Ab$, which is $x^{\mathsf{T}}\widetilde{R}x$ written out by blocks; and $u^{\dagger}u = a^{\mathsf{T}}a + b^{\mathsf{T}}b = x^{\mathsf{T}}x$. The map $x \mapsto u$ is a bijection of $\mathbb{R}^{2n}$ onto $\mathbb{C}^n$ that sends 0 to 0, so the set of Rayleigh quotients $u^{\dagger}Ru/u^{\dagger}u$, $u \neq 0$, equals the set of $x^{\mathsf{T}}\widetilde{R}x/x^{\mathsf{T}}x$, $x \neq 0$. By the spectral theorem the least eigenvalue of a Hermitian matrix is the minimum of its Rayleigh quotient over non-zero complex vectors, and the least eigenvalue of a real symmetric matrix, which has a real orthonormal eigenbasis, is the minimum of its Rayleigh quotient over non-zero real vectors. So $\lambda_{\min}(\widetilde{R}) = \lambda_{\min}(R)$.

(ii), the Gram matrix. For $y \in \mathbb{R}^{2n}$, $\|Vy\|^2 = y^{\mathsf{T}}V^{\mathsf{T}}Vy = \|y\|^2 + y^{\mathsf{T}}Ey$. By the Cauchy–Schwarz inequality applied to each row of E, $\|Ey\|^2 = \sum_i \big(\sum_l E_{il}y_l\big)^2 \le \sum_i \sum_l E_{il}^2 \, \|y\|^2 \le \theta^2\|y\|^2$, and again by the Cauchy–Schwarz inequality $|y^{\mathsf{T}}Ey| \le \|y\| \, \|Ey\| \le \theta\|y\|^2$. Therefore

$$(1-\theta)\|y\|^2 \;\le\; \|Vy\|^2 \;\le\; (1+\theta)\|y\|^2 \qquad \text{for every } y \in \mathbb{R}^{2n}. \tag{7.3}$$

As $\theta < 1$, $Vy = 0$ forces $y = 0$, so the square matrix V is injective and hence invertible.

(ii), Gershgorin's bound on the quadratic form. B is real symmetric, $B^{\mathsf{T}} = V^{\mathsf{T}}\widetilde{R}^{\mathsf{T}}V = B$ by (i). For $y \in \mathbb{R}^{2n}$, using $|y_i y_l| \le \frac{1}{2}(y_i^2 + y_l^2)$, so that $B_{il}y_i y_l \ge -|B_{il}| \, |y_i y_l| \ge -\frac{1}{2}|B_{il}|(y_i^2 + y_l^2)$ whatever the signs of B_{il}, y_i and y_l, and $|B_{il}| = |B_{li}|$,

$$y^{\mathsf{T}}By = \sum_i B_{ii}y_i^2 + \sum_{i \neq l} B_{il}y_i y_l \ge \sum_i B_{ii}y_i^2 - \tfrac{1}{2}\sum_{i \neq l} |B_{il}|\big(y_i^2 + y_l^2\big) = \sum_i y_i^2\Big(B_{ii} - \sum_{l \neq i} |B_{il}|\Big) \ge \gamma \, \|y\|^2,$$

the equality because the two halves of the double sum are equal after the indices are exchanged, $|B_{il}|$ being symmetric in i and l.

(ii), the bound. Let $x \in \mathbb{R}^{2n}$, $x \neq 0$, and put $y \coloneqq V^{-1}x \neq 0$. Then $x^{\mathsf{T}}\widetilde{R}x = y^{\mathsf{T}}V^{\mathsf{T}}\widetilde{R}Vy = y^{\mathsf{T}}By \ge \gamma\|y\|^2$, and $\|x\|^2 = \|Vy\|^2$ lies between $(1-\theta)\|y\|^2$ and $(1+\theta)\|y\|^2$ by (7.3). If $\gamma \ge 0$, then $\|y\|^2 \ge \|x\|^2/(1+\theta)$ gives $\gamma\|y\|^2 \ge \gamma\|x\|^2/(1+\theta)$. If $\gamma < 0$, then $\|y\|^2 \le \|x\|^2/(1-\theta)$, the denominator being positive, gives $\gamma\|y\|^2 \ge \gamma\|x\|^2/(1-\theta)$. In either case the Rayleigh quotient $x^{\mathsf{T}}\widetilde{R}x/x^{\mathsf{T}}x$ is at least the right-hand side of (7.2). Since x was arbitrary, so is $\lambda_{\min}(\widetilde{R})$, the minimum of that quotient, and $\lambda_{\min}(\widetilde{R}) = \lambda_{\min}(R)$ by (i).

(iii). The real and imaginary parts of an element of $\mathbb{Q}(i)$ are rational, so $\widetilde{R}$ is rational; B and E are sums of products of rationals; $|B_{il}|$, the sums and the minimum defining γ are rational; $\sum_{i,l} E_{il}^2 \le \theta^2$, $0 \le \theta$ and $\theta < 1$ compare rationals; and $1 \pm \theta > 0$ is rational, so the quotient is.

(iv). $-R$ is Hermitian, its real embedding is $-\widetilde{R}$, and its eigenvalues are those of R with the sign reversed, so the largest eigenvalue of R is $-\lambda_{\min}(-R)$, and a lower bound on $\lambda_{\min}(-R)$ from (ii) gives an upper bound on it. □

A Hermitian matrix within ρ of a Hermitian M in Frobenius norm has its spectrum within ρ of M's extreme eigenvalues. An entrywise enclosure whose exactly zero rows are dropped and whose rest certifies a positive least eigenvalue fixes the rank and support of every Hermitian matrix it contains.

Lemma 7.2 bounds the least eigenvalue of a matrix known exactly. An evaluation in ball arithmetic knows a matrix only through an enclosure: part (i) carries the bound across a ball in Frobenius norm around an exact matrix, which is Weyl's inequality at the two ends of the spectrum, and part (ii) reads an entrywise enclosure as such a ball. Parts (iii) and (iv) decide the rank of every matrix an enclosure contains at once: the rows the enclosure fixes at zero are removed, and the rest must be positive definite throughout the enclosure. Where the rest is not shown to be, and in particular where its centre vanishes, the enclosure decides no rank, and the corollary claims none.

Corollary 7.3 (Least eigenvalue and rank over an enclosure). Let $n \ge 1$, and for a complex $n \times n$ matrix X write $\|X\|_{\mathrm{F}} := \left(\sum_{i,l} |X_{il}|^2\right)^{1/2}$ and $\|X\| := \max\{\|Xx\| : x \in \mathbb{C}^n,\ \|x\| = 1\}$.

(i) *(A ball in Frobenius norm.)* Let R and M be Hermitian $n \times n$ matrices and $\rho \ge 0$ with $\|R - M\|_{\mathrm{F}} \le \rho$. Then $\|R - M\| \le \|R - M\|_{\mathrm{F}}$, and

$$\lambda_{\min}(R) \ \ge\ \lambda_{\min}(M) - \rho, \qquad \lambda_{\max}(R) \ \le\ \lambda_{\max}(M) + \rho, \tag{7.4}$$

so that every eigenvalue of R lies in $[\lambda_{\min}(M) - \rho, \lambda_{\max}(M) + \rho]$. In particular $\lambda_{\min}(R) \ge \ell - \rho$ for every lower bound ℓ on $\lambda_{\min}(M)$, the right-hand side of (7.2) for M among them.

(ii) *(An entrywise enclosure.)* If (C, r) is an entrywise enclosure of R (Definition 5.3) and ρ is a rational with $\rho \ge 0$ and $\rho^2 \ge \sum_{i,l} r_{il}^2$, then $\|R - C\|_{\mathrm{F}} \le \rho$ and (i) applies with $M = C$; whether ρ qualifies is a comparison of rationals.

(iii) *(A factor form fixes the rank.)* Let $1 \le k \le n$, let E be a $k \times n$ matrix of rank k and $\widetilde{X}$ a positive definite $k \times k$ matrix, and put $X := E^{\dagger} \widetilde{X} E$. Then $X \succeq 0$, $\operatorname{ran} X = \operatorname{ran} E^{\dagger}$ and $\operatorname{rank} X = k$.

(iv) *(The rank over an entrywise enclosure.)* Let C be a Hermitian $n \times n$ matrix with entries in $\mathbb{Q}(i)$ and $r_{il} = r_{li} \geq 0$ rationals, and say that (C, r) *contains* a Hermitian X when $|X_{il} - C_{il}| \leq r_{il}$ for all i, l. Let $Z \coloneqq \{i : C_{il} = 0 \text{ and } r_{il} = 0 \text{ for every } l\}$, the rows the pair fixes at zero, let K be its complement in $\{1, \dots, n\}$ and $k \coloneqq |K|$. If $k = 0$, the zero matrix is the only Hermitian matrix (C, r) contains. If $k \geq 1$, let E be the $k \times n$ matrix whose rows are the transposed standard basis vectors e_i^{T}, $i \in K$, in increasing order; then every Hermitian X that (C, r) contains satisfies $X = E^{\dagger}(EXE^{\dagger})E$. If moreover $\ell_K \leq \lambda_{\min}(ECE^{\dagger})$ and a rational $\rho_K \geq 0$ with $\rho_K^2 \geq \sum_{i,l \in K} r_{il}^2$ satisfy $\ell_K - \rho_K > 0$, then for every Hermitian X that (C, r) contains, $EXE^{\dagger} \succ 0$, and so, by (iii), $X \succeq 0$, $\operatorname{ran} X = \operatorname{span}\{e_i : i \in K\}$ and $\operatorname{rank} X = k$: the factor E, the support and the rank are the same for every such X.

Uses Def. 5.3, Lem. 7.2 · *Used by* Cor. 7.4, Lem. 7.5, meat-core/frank-wolfe/lem:certified-bracket, meat-core/frank-wolfe/lem:certified-bracket-enclosed, meat-core/frank-wolfe/rem:certified-scope, meat-core/frank-wolfe/req:certified-evaluation, receiver/req:rigorous-weight, receiver/thm:exact-and-evaluated · newly derived · implemented · *Code* certiqs_key.foundations.dyadic:enclosure_eigenvalue_bounds

Proof. (i). Let A be a complex $n \times n$ matrix and x a unit vector. By the Cauchy–Schwarz inequality applied to each row, $\|Ax\|^2 = \sum_i \left| \sum_l A_{il} x_l \right|^2 \leq \sum_i \left(\sum_l |A_{il}|^2 \right) \|x\|^2 = \|A\|_{\mathrm{F}}^2$, so $\|A\| \leq \|A\|_{\mathrm{F}}$, and $|x^{\dagger}Ax| \leq \|x\| \, \|Ax\| \leq \|A\|_{\mathrm{F}}$. With $A = R - M$, for every unit x, $x^{\dagger}Rx = x^{\dagger}Mx + x^{\dagger}(R - M)x$ lies between $x^{\dagger}Mx - \rho$ and $x^{\dagger}Mx + \rho$, hence between $\lambda_{\min}(M) - \rho$ and $\lambda_{\max}(M) + \rho$, since the Rayleigh quotient of the Hermitian M lies between its extreme eigenvalues. The least and the largest eigenvalue of the Hermitian R are the minimum and the maximum of $x^{\dagger}Rx$ over unit x, which gives (7.4), and every eigenvalue of R lies between them. The last sentence follows from $\lambda_{\min}(M) \geq \ell$.

(ii). By (5.1), $\|R - C\|_{\mathrm{F}}^2 = \sum_{i,l} |R_{il} - C_{il}|^2 \leq \sum_{i,l} r_{il}^2 \leq \rho^2$, and C is Hermitian by Definition 5.3. The sum of the squares of finitely many rationals is rational.

(iii). For every x, $x^{\dagger}Xx = (Ex)^{\dagger}\widetilde{X}(Ex) \geq 0$, so $X \succeq 0$, and $x^{\dagger}Xx = 0$ exactly when $Ex = 0$, $\widetilde{X}$ being positive definite. For a positive semidefinite X, $Xx = 0$ exactly when $x^{\dagger}Xx = 0$, since $x^{\dagger}Xx = \|X^{1/2}x\|^2$ and $Xx = X^{1/2}(X^{1/2}x)$. So $\ker X = \ker E$, and, X being Hermitian, $\operatorname{ran} X = (\ker X)^{\perp} = (\ker E)^{\perp} = \operatorname{ran} E^{\dagger}$, whose dimension is $\operatorname{rank} E = k$.

(iv). Let X be Hermitian and contained in (C, r). For $i \in Z$ and every l, $|X_{il}| = |X_{il} - C_{il}| \leq r_{il} = 0$, so row i of X vanishes, and so does column i, X being Hermitian. If $k = 0$, every row vanishes and $X = 0$, which (C, r) contains. If $k \geq 1$, $E^{\dagger}E$ is the diagonal matrix with entry 1 at the indices of K and 0 at those of Z, so $E^{\dagger}EXE^{\dagger}E$ is X with the rows and columns of Z set to zero, which is X. The matrix $EXE^{\dagger}$ is the principal

submatrix of X on K, Hermitian, and $|(EXE^\dagger)_{il} - (ECE^\dagger)_{il}| \le r_{il}$ for $i, l \in K$, so $\|EXE^\dagger - ECE^\dagger\|_{\mathrm{F}} \le \rho_K$ as in (ii), with $ECE^\dagger$ Hermitian. By (i), $\lambda_{\min}(EXE^\dagger) \ge \ell_K - \rho_K > 0$, so $EXE^\dagger$ is positive definite, and (iii) applies with $\widetilde{X} = EXE^\dagger$; E has rank k, its rows being distinct standard basis vectors, and $\operatorname{ran} E^\dagger = \operatorname{span}\{e_i : i \in K\}$. Neither E nor k depends on X. □

*A factor enclosure with no column holds only the zero matrix. One whose Gram matrix L*L has a positive floor over its box fixes, at every member, the rank, its number of columns, and the support, the span of that member's columns.*

Corollary 7.3 (iv) fixes the rank of the matrices an entrywise enclosure contains where they are positive definite on the rows it does not fix at zero. A factor enclosure fixes it where the columns of its factors are linearly independent throughout the box, which a positive floor under their Gram matrix shows: every member then has as many independent columns as the triple has, and its support is the span of its own columns. The support moves with the member, its dimension does not, and each member has a factor form whose factor lies in a box and whose core is the same exact matrix for all. Part (iii) gives one floor that is a comparison of rationals.

Corollary 7.4 (Rank over a factor enclosure). Let (L_c, t, W) be a triple of a matrix box (L_c, t) of complex $d \times k$ matrices and a diagonal $k \times k$ matrix W with positive rational diagonal entries, as in Definition 5.4, and call a matrix L that the box contains a *factor* and $LWL^\dagger$ a *member*.

(i) *(No column.)* If $k = 0$, every member is the zero $d \times d$ matrix.

(ii) *(A Gram floor fixes the rank.)* Let $k \ge 1$, let G be a Hermitian $k \times k$ matrix and $\rho \ge 0$ with $\|L^\dagger L - G\|_{\mathrm{F}} \le \rho$ for every factor L, the Frobenius norm of Corollary 7.3, and let $\ell \le \lambda_{\min}(G)$ with $\ell - \rho > 0$. Then for every factor L, $L^\dagger L \succ 0$, so that $k \le d$ and $\operatorname{rank} L = k$; and the member $X \coloneqq LWL^\dagger$ satisfies $X = E^\dagger W E$ with $E \coloneqq L^\dagger$ a $k \times d$ matrix of rank k, $X \succeq 0$, $\operatorname{ran} X = \operatorname{ran} L$, the span of the columns of L, and $\operatorname{rank} X = k$; in particular $X \ne 0$.

(iii) *(One such floor.)* Let $k \ge 1$, put $a_{im} \coloneqq |\operatorname{Re}(L_c)_{im}| + |\operatorname{Im}(L_c)_{im}|$ and

$$s_{ml} \coloneqq \sum_{i=1}^{d} \bigl(a_{im} t_{il} + t_{im} a_{il} + t_{im} t_{il}\bigr) \qquad (1 \le m, l \le k), \tag{7.5}$$

rationals. Then $G \coloneqq L_c^\dagger L_c$ is Hermitian with entries in $\mathbb{Q}(i)$, and every rational $\rho \ge 0$ with $\rho^2 \ge \sum_{m,l} s_{ml}^2$ satisfies $\|L^\dagger L - G\|_{\mathrm{F}} \le \rho$ for every factor L; whether ρ qualifies, and whether $\ell - \rho > 0$ for a rational ℓ, is a comparison of rationals.

Uses Def. 5.4, Cor. 7.3 · *Used by* meat-core/frank-wolfe/lem:certified-bracket-enclosed, meat-core/frank-wolfe/rem:certified-scope, meat-core/frank-wolfe/req:certified-evaluation, receiver/detector-imperfections/req:dark-composed-reference · newly derived · implemented · *Code* certiqs_key.foundations.reference:fixes_rank

Proof. (i). For $k = 0$ a factor L is $d \times 0$ and W is 0×0, so every entry $(LWL^\dagger)_{il} = \sum_{m=1}^{k} L_{im} W_{mm} \overline{L_{lm}}$ is an empty sum, 0.

(ii). Let L be a factor. $L^\dagger L$ is a Hermitian $k \times k$ matrix, and Corollary 7.3 (i) with $R := L^\dagger L$ and $M := G$ gives $\lambda_{\min}(L^\dagger L) \geq \lambda_{\min}(G) - \rho \geq \ell - \rho > 0$. So for $y \neq 0$, $\|Ly\|^2 = y^\dagger L^\dagger L y > 0$: L is injective, its rank is k, and $k \leq d$, the rank of a $d \times k$ matrix being at most d. The adjoint $E = L^\dagger$ has the same rank k, and $E^\dagger W E = LWL^\dagger = X$ with W positive definite, its diagonal entries being positive. Corollary 7.3 (iii), with E and $\widetilde{X} := W$, gives $X \succeq 0$, $\operatorname{ran} X = \operatorname{ran} E^\dagger = \operatorname{ran} L$ and $\operatorname{rank} X = k \geq 1$, so $X \neq 0$.

(iii). The entries $(L_c^\dagger L_c)_{ml} = \sum_i \overline{(L_c)_{im}} (L_c)_{il}$ are sums of products of Gaussian rationals, so they lie in $\mathbb{Q}(i)$, and $\overline{(L_c^\dagger L_c)_{lm}} = (L_c^\dagger L_c)_{ml}$. Let L be a factor and $\Delta := L - L_c$, so $|\Delta_{im}| \leq t_{im}$. Then

$$(L^\dagger L - L_c^\dagger L_c)_{ml} = \sum_{i=1}^{d} \Big(\overline{(L_c)_{im}}\, \Delta_{il} + \overline{\Delta_{im}}\, (L_c)_{il} + \overline{\Delta_{im}}\, \Delta_{il} \Big),$$

and $|(L_c)_{im}| \leq a_{im}$, since $|x + iy| \leq |x| + |y|$ for real x, y; by the triangle inequality the modulus of the left-hand side is at most s_{ml}. Hence $\|L^\dagger L - G\|_{\mathrm{F}}^2 = \sum_{m,l} |(L^\dagger L - G)_{ml}|^2 \leq \sum_{m,l} s_{ml}^2 \leq \rho^2$. The a_{im} and s_{ml} are sums and products of rationals, and so is $\sum_{m,l} s_{ml}^2$. □

On Hermitian matrices with spectra in an interval, a function with derivative bounded by L moves at most L times the perturbation in Frobenius norm, and the divided-difference map of x^p at most p(1−p)a^(p−2) times it above a floor a. Close arguments get a second-order bound.

The gradient of the pinched trace power is built from powers of positive definite matrices and from the divided-difference map of $x \mapsto x^p$ (Lemma "Divided-difference derivative of a matrix function" (meat-core/lem:dk), Theorem "Gradient of the pinched trace power on a face" (meat-core/thm:gradient)). An evaluation in ball arithmetic has such a matrix only as an exactly unitary U and an exact diagonal Λ with $U^\dagger N U = \Lambda + E$ and E small; the lemma says how far each quantity can move under E. Part (i) bounds the values and traces of a function, part (ii) moves them through U, and part (iii) bounds the divided-difference map by an integral representation of $x \mapsto x^p$, whose constant comes out exactly without being computed. Part (iv) encloses a divided difference of two close arguments without the cancellation of the quotient.

Lemma 7.5 (Perturbation bounds for matrix functions). Let $n \geq 1$, write $\|\cdot\|_{\mathrm{F}}$ and $\|\cdot\|$ as in Corollary 7.3, and let P, Q be Hermitian $n \times n$ matrices.

For a real function f defined on a set containing the spectrum of P, with $P = \sum_a \mu_a \Pi_a$ its spectral decomposition into distinct eigenvalues μ_a and spectral projectors Π_a, put $f(P) \coloneqq \sum_a f(\mu_a)\Pi_a$. For real $x \neq y$ put $f^{[1]}(x,y) \coloneqq (f(x) - f(y))/(x-y)$, and $f^{[1]}(x,x) \coloneqq f'(x)$ where f is differentiable at x.

(i) *(Values and traces.)* Let I be an interval containing the spectra of P and Q, let f be continuous on I and differentiable in its interior, and let $|f'| \leq L$ there. Then

$$\|f(P) - f(Q)\|_{\mathrm{F}} \leq L\,\|P - Q\|_{\mathrm{F}}, \qquad \big|\operatorname{tr} f(P) - \operatorname{tr} f(Q)\big| \leq \sqrt{n}\,L\,\|P - Q\|_{\mathrm{F}}, \tag{7.6}$$

and every entry of $f(P) - f(Q)$ has modulus at most $L\,\|P - Q\|_{\mathrm{F}}$. For $f(x) = x^s$ on $I = [a,b]$ with $0 < a \leq b$, one may take $L = s\,a^{s-1}$ if $0 < s \leq 1$ and $L = s\,b^{s-1}$ if $s \geq 1$.

(ii) *(Unitary covariance.)* For every unitary U, $f(UPU^\dagger) = U f(P) U^\dagger$, and $\|UXU^\dagger\|_{\mathrm{F}} = \|X\|_{\mathrm{F}}$ for every X.

(iii) *(The divided-difference map of a power.)* Let $0 < p < 1$ and $f(x) = x^p$ on $(0,\infty)$. For a positive definite $P = \sum_a \mu_a \Pi_a$ and every complex $n \times n$ matrix H put

$$\mathrm{D}_P[H] \coloneqq \sum_{a,b} f^{[1]}(\mu_a, \mu_b)\,\Pi_a H \Pi_b. \tag{7.7}$$

(a) $J_p \coloneqq \int_0^\infty u^{p-1}(1+u)^{-1}\,\mathrm{d}u$ is finite and positive, and with $c_p \coloneqq 1/J_p$,

$$\mathrm{D}_P[H] = c_p \int_0^\infty t^p\,(P+t)^{-1} H (P+t)^{-1}\,\mathrm{d}t, \tag{7.8}$$

the integral taken entry by entry, each entry's integrand absolutely integrable.

(b) If Q is positive definite as well and $0 < a \leq \min\{\lambda_{\min}(P), \lambda_{\min}(Q)\}$, then

$$\big\|\mathrm{D}_P[H] - \mathrm{D}_Q[H]\big\|_{\mathrm{F}} \;\leq\; p(1-p)\,a^{p-2}\;\|P - Q\|\;\|H\|_{\mathrm{F}} \;\leq\; p(1-p)\,a^{p-2}\,\|P - Q\|_{\mathrm{F}}\,\|H\|_{\mathrm{F}}. \tag{7.9}$$

(c) For every unitary U, $\mathrm{D}_{UPU^\dagger}[H] = U\,\mathrm{D}_P[U^\dagger H U]\,U^\dagger$; and if $P = \operatorname{diag}(d_1, \ldots, d_n)$, the $d_k > 0$ not necessarily distinct, then $\mathrm{D}_P[H]_{kl} = f^{[1]}(d_k, d_l)\,H_{kl}$ for all k, l.

(iv) *(Close arguments.)* Let $y \leq x$, let f be three times continuously differentiable on an open interval containing $[y,x]$, and put $m \coloneqq (x+y)/2$ and $d \coloneqq x - y$. Then

$$\big|f^{[1]}(x,y) - f'(m)\big| \;\leq\; \frac{d^2}{24} \max_{\xi \in [y,x]} |f'''(\xi)|. \tag{7.10}$$

Uses Cor. 7.3 · *Used by* meat-core/frank-wolfe/req:certified-evaluation, meat-core/lem:kraus-freedom · newly derived · implemented · *Code* certiqs_key.foundations.balls:deriv_power

Proof. (i). Let $\{u_i\}$ and $\{v_j\}$ be orthonormal eigenbases of P and Q, with $Pu_i = p_i u_i$ and $Qv_j = q_j v_j$, and let U and V be the unitary matrices with these columns. For every matrix A, $U^\dagger A V$ has the entries $u_i^\dagger A v_j$, and $\|U^\dagger A V\|_{\mathrm{F}}^2 = \operatorname{tr}\big[V^\dagger A^\dagger U U^\dagger A V\big] = \operatorname{tr}\big[A^\dagger A\big] = \|A\|_{\mathrm{F}}^2$, so $\|A\|_{\mathrm{F}}^2 = \sum_{i,j} |u_i^\dagger A v_j|^2$. Since $u_i^\dagger f(P) = f(p_i) u_i^\dagger$ and $f(Q) v_j = f(q_j) v_j$,

$$u_i^\dagger \big(f(P) - f(Q)\big) v_j = \big(f(p_i) - f(q_j)\big) u_i^\dagger v_j, \qquad u_i^\dagger (P - Q) v_j = (p_i - q_j)\, u_i^\dagger v_j.$$

Both p_i and q_j lie in the interval I, and so does the segment between them; if $p_i \neq q_j$, the mean value theorem on that segment gives $|f(p_i) - f(q_j)| \leq L|p_i - q_j|$, and if $p_i = q_j$ both sides vanish. Hence $|u_i^\dagger (f(P) - f(Q)) v_j| \leq L |u_i^\dagger (P - Q) v_j|$ for all i, j, and summing the squares gives the first inequality of (7.6). Every entry of a matrix A has modulus at most $\|A\|_{\mathrm{F}}$, and $|\operatorname{tr} A| = |\sum_k A_{kk}| \leq \sqrt{n}\,(\sum_k |A_{kk}|^2)^{1/2} \leq \sqrt{n}\,\|A\|_{\mathrm{F}}$ by the Cauchy–Schwarz inequality, which with $A = f(P) - f(Q)$ gives the rest. For $f(x) = x^s$, $f'(x) = s\,x^{s-1}$, which is non-increasing on $[a, b]$ if $s \leq 1$ and non-decreasing if $s \geq 1$, so its maximum there is $s\,a^{s-1}$, respectively $s\,b^{s-1}$.

(ii). The matrices $U \Pi_a U^\dagger$ are orthogonal projectors, mutually orthogonal and summing to the identity, and $UPU^\dagger = \sum_a \mu_a U \Pi_a U^\dagger$, so they are the spectral projectors of $UPU^\dagger$ and $f(UPU^\dagger) = \sum_a f(\mu_a) U \Pi_a U^\dagger = U f(P) U^\dagger$. And $\|UXU^\dagger\|_{\mathrm{F}}^2 = \operatorname{tr}\big[UX^\dagger U^\dagger U X U^\dagger\big] = \operatorname{tr}\big[X^\dagger X\big]$.

(iii)(a), Step 1 (the constant). The integrand of J_p is positive and continuous on $(0, \infty)$, at most u^{p-1} on $(0, 1]$, integrable there since $p > 0$, and at most u^{p-2} on $[1, \infty)$, integrable there since $p < 1$; so $0 < J_p < \infty$. For $x > 0$ the substitution $t = xu$ gives

$$\int_0^\infty t^{p-1} \frac{x}{x + t}\,\mathrm{d}t = \int_0^\infty x^{p-1} u^{p-1} \frac{x}{x(1+u)}\, x\,\mathrm{d}u = x^p J_p, \qquad \text{that is,} \qquad x^p = c_p \int_0^\infty t^{p-1} \frac{x}{x+t}\,\mathrm{d}t.$$

Step 2 (two integrals). Integration by parts on $[\varepsilon, R]$, $0 < \varepsilon < R$, gives

$$\int_\varepsilon^R \frac{u^{p-1}}{1+u}\,\mathrm{d}u = \Big[\frac{u^p}{p(1+u)}\Big]_\varepsilon^R + \frac{1}{p}\int_\varepsilon^R \frac{u^p}{(1+u)^2}\,\mathrm{d}u,$$
$$\int_\varepsilon^R \frac{u^p}{(1+u)^3}\,\mathrm{d}u = \Big[-\frac{u^p}{2(1+u)^2}\Big]_\varepsilon^R + \frac{p}{2}\int_\varepsilon^R \frac{u^{p-1}}{(1+u)^2}\,\mathrm{d}u.$$

The boundary terms tend to 0 as $\varepsilon \to 0$ and $R \to \infty$, since $0 < p < 1$, and the integrals on the right converge, their integrands being bounded by u^p or u^{p-1} near 0 and by u^{p-2} or u^{p-3} at infinity. So $\int_0^\infty u^p (1+u)^{-2}\,\mathrm{d}u = p J_p$. Since $u^{p-1}(1+u)^{-2} = u^{p-1}(1+u)^{-1} - u^p (1+u)^{-2}$, the numerator of the

difference being $u^{p-1}\big((1+u)-u\big)$, $\int_0^\infty u^{p-1}(1+u)^{-2}\,\mathrm{d}u = J_p - pJ_p = (1-p)J_p$, and so

$$\int_0^\infty \frac{u^p}{(1+u)^2}\,\mathrm{d}u = p\,J_p, \qquad \int_0^\infty \frac{u^p}{(1+u)^3}\,\mathrm{d}u = \frac{p(1-p)}{2}\,J_p.$$

Step 3 (divided differences). Let $x, y > 0$. If $x \neq y$, then $\frac{x}{x+t} - \frac{y}{y+t} = \frac{t(x-y)}{(x+t)(y+t)}$, and the two integrals of Step 1 converge separately, so $f^{[1]}(x,y) = c_p \int_0^\infty t^p\,(x+t)^{-1}(y+t)^{-1}\,\mathrm{d}t$. If $x = y$, the substitution $t = xu$ and Step 2 give $c_p \int_0^\infty t^p (x+t)^{-2}\,\mathrm{d}t = c_p\,x^{p-1}\int_0^\infty u^p(1+u)^{-2}\,\mathrm{d}u = p\,x^{p-1} = f'(x)$. So in either case

$$f^{[1]}(x,y) = c_p \int_0^\infty \frac{t^p}{(x+t)(y+t)}\,\mathrm{d}t.$$

Step 4 (the map). For $t \geq 0$, $P + t$ is positive definite with $(P+t)^{-1} = \sum_a (\mu_a + t)^{-1}\Pi_a$, so $t^p(P+t)^{-1}H(P+t)^{-1} = \sum_{a,b} t^p(\mu_a+t)^{-1}(\mu_b+t)^{-1}\Pi_a H \Pi_b$, a finite sum whose scalar coefficients are integrable by Step 3. Integrating term by term and using Step 3 gives (7.8).

(iii)(b). Put $A_t \coloneqq (P+t)^{-1}$ and $B_t \coloneqq (Q+t)^{-1}$ for $t \geq 0$. The eigenvalues of A_t are $(\mu+t)^{-1}$ for the eigenvalues $\mu \geq a$ of P, so $\|A_t\| \leq (a+t)^{-1}$, and likewise $\|B_t\| \leq (a+t)^{-1}$. From $A_t - B_t = A_t\big((Q+t)-(P+t)\big)B_t = A_t(Q-P)B_t$ and the submultiplicativity of the operator norm, $\|A_t - B_t\| \leq \|P-Q\|\,(a+t)^{-2}$. For matrices X, Y, W, $\|XYW\|_{\mathrm{F}} \leq \|X\|\;\|Y\|_{\mathrm{F}}\;\|W\|$: column by column, $\|XY\|_{\mathrm{F}}^2 = \sum_l \|Xy_l\|^2 \leq \|X\|^2\,\|Y\|_{\mathrm{F}}^2$ for the columns y_l of Y, and $\|YW\|_{\mathrm{F}} = \|W^\dagger Y^\dagger\|_{\mathrm{F}} \leq \|W\|\;\|Y\|_{\mathrm{F}}$, the operator norm being invariant under the adjoint. Since $A_tHA_t - B_tHB_t = (A_t - B_t)HA_t + B_tH(A_t - B_t)$,

$$\|A_tHA_t - B_tHB_t\|_{\mathrm{F}} \leq 2\;\|P-Q\|\;\|H\|_{\mathrm{F}}\,(a+t)^{-3}.$$

By (a), $\mathrm{D}_P[H] - \mathrm{D}_Q[H]$ is the entrywise integral of $\Phi(t) \coloneqq c_p t^p(A_tHA_t - B_tHB_t)$. For such an integral $\Delta = \int_0^\infty \Phi(t)\,\mathrm{d}t$ with $\int_0^\infty \|\Phi(t)\|_{\mathrm{F}}\,\mathrm{d}t < \infty$, $\|\Delta\|_{\mathrm{F}} \leq \int_0^\infty \|\Phi(t)\|_{\mathrm{F}}\,\mathrm{d}t$: if $\Delta \neq 0$, put $W \coloneqq \Delta/\|\Delta\|_{\mathrm{F}}$; then $\|\Delta\|_{\mathrm{F}} = \operatorname{Re}\operatorname{tr}\big[W^\dagger\Delta\big] = \int_0^\infty \operatorname{Re}\operatorname{tr}\big[W^\dagger\Phi(t)\big]\,\mathrm{d}t \leq \int_0^\infty \|\Phi(t)\|_{\mathrm{F}}\,\mathrm{d}t$, by linearity of the entrywise integral and the Cauchy–Schwarz inequality for $(X,Y) \mapsto \operatorname{tr}\big[X^\dagger Y\big]$. Hence, substituting $t = au$ and using Step 2,

$$\|\mathrm{D}_P[H] - \mathrm{D}_Q[H]\|_{\mathrm{F}} \leq 2c_p\;\|P-Q\|\;\|H\|_{\mathrm{F}} \int_0^\infty \frac{t^p}{(a+t)^3}\,\mathrm{d}t = 2c_p\,a^{p-2}\,\frac{p(1-p)}{2}\,J_p\;\|P-Q\|\;\|H\|_{\mathrm{F}},$$

which is the first inequality of (7.9), as $c_pJ_p = 1$. The second is $\|P-Q\| \leq \|P-Q\|_{\mathrm{F}}$ (Corollary 7.3 (i)).

(iii)(c). By the argument of (ii), the spectral projectors of $UPU^\dagger$ are the $U\Pi_aU^\dagger$, with the same eigenvalues, so $\mathrm{D}_{UPU^\dagger}[H] = \sum_{a,b} f^{[1]}(\mu_a,\mu_b)\,U\Pi_aU^\dagger HU\Pi_bU^\dagger = U\,\mathrm{D}_P[U^\dagger HU]\,U^\dagger$. If P is diagonal, $\Pi_a = \sum_{k:\,d_k=\mu_a} e_k e_k^\dagger$, so $(\Pi_aH\Pi_b)_{kl} = H_{kl}$

when $d_k = \mu_a$ and $d_l = \mu_b$ and 0 otherwise; summing over a and b leaves $f^{[1]}(d_k, d_l)H_{kl}$ at (k, l).

(iv). If $d = 0$, then $f^{[1]}(x, y) = f'(x) = f'(m)$. Otherwise put $h := d/2 > 0$. Taylor's theorem with the Lagrange remainder gives $\xi_+ \in (m, m+h)$ and $\xi_- \in (m-h, m)$ with

$$f(m \pm h) = f(m) \pm hf'(m) + \frac{h^2}{2}f''(m) \pm \frac{h^3}{6}f'''(\xi_\pm),$$

so $f(x) - f(y) = 2hf'(m) + \frac{h^3}{6}\big(f'''(\xi_+) + f'''(\xi_-)\big)$. Dividing by $d = 2h$, $f^{[1]}(x, y) - f'(m) = \frac{h^2}{12}\big(f'''(\xi_+) + f'''(\xi_-)\big)$, whose modulus is at most $\frac{h^2}{6}\max_{[y,x]}|f'''| = \frac{d^2}{24}\max_{[y,x]}|f'''|$. □

A reported key rate carries one of three tiers: estimate, a returned point's objective, claiming no bound; first-order, a lower bound on the minimum under named hypotheses; or certified, such a bound with every step from the data exact or rigorously enclosed, no hypothesis about arithmetic.

In this repository the estimate is the objective value of Lemma "Upper estimate of the value at any feasible point" (meat-core/lem:estimate), and the first-order number is the linearisation bound of the Frank–Wolfe solver, Theorem "The estimate and the first-order lower bound" (meat-core/frank-wolfe/thm:firstorder), whose hypotheses include lower bounds on the two linear minima it reads. A run evaluates that bound in binary64, so a first-order number rests, beside the hypotheses of the theorem, on arithmetic that no claim bounds. The certified number is the same bound formed again at a run's final points from enclosures of the gradient (Lemma "The certified bracket from enclosures of the gradient" (meat-core/frank-wolfe/lem:certified-bracket)), every step from the declaration's data exact or carried out in an arithmetic that bounds its own rounding, so that nothing is assumed of how it was computed.

Definition 7.6 (The tiers of a reported key rate)**.** A *tier* of a reported key-rate number is one of the following three labels.

(i) *estimate*: the number is the value of the objective at the point an optimiser returned. It is reported as an estimate of the minimum of the objective and is claimed as no bound.

(ii) *first-order*: the number is a lower bound on the minimum of the objective, established by a claim under hypotheses that claim names.

(iii) *certified*: the number is a lower bound on the minimum of the objective of the declaration it is formed on, established by a claim under hypotheses that claim names, and every arithmetic step between that declaration's data and the number is exact or *rigorously enclosed*: a computation is rigorously enclosed when each of its operations returns

a set of numbers, such as an interval or a ball, that contains the exact result of the operation at every value the sets of its inputs contain, so that the set it ends with contains the exact value, at the exact data, of the quantity it computes. None of those hypotheses is an assumption about how a number was computed, that is, none is an assumption of the category numerical-trust.

The tiers are ordered, estimate below first-order below certified, and a tier is *stronger* than the tiers below it. A certified number is a first-order number as well. There is no other tier.

Used by Req. 10.5, Req. 12.1, Req. 12.2, meat-core/frank-wolfe/rem:certified-scope, meat-core/frank-wolfe/req:certified-evaluation, meat-core/req:unreachable-acceptance-set, receiver/detector-imperfections/req:dark-composed-reference · newly derived · implemented · *Code* certiqs_key.foundations.assurance:Assurance

8 The snapped-measurement assumption

A moved block of a measurement snap is related to its model by no claim. Where the run reads it, because no reference reaches the evaluation, or the evaluation on the reference does not count, or the key length is formed from a number the run certified on the snapped family in its place, the analysis of the run is an analysis of the snapped family, the more so where a flagged last resort gave an element support the data deny. The one assumption of the module, Assumption 8.1, is the premise under which that analysis describes the device: on the blocks the record names, the device measures the snapped family as recorded, in place of the model there, and for a producer's snap of its own emission, what the run concludes from the emission holds of the snapped family. A block evaluated off its snap is never among them. A run with no such block, or a run that declares the strict option, does not rest on it.

On every moved block of a snap that the run does not evaluate off its snap, the device measures the snapped family; for a producer's snap of its own emission, what the run concludes from the emission holds of the snapped family there.

A snap moves a measurement family onto a lattice (Definition 5.5), and nothing established relates a moved block to the model it replaced. A run that holds a reference of the family and evaluates the numbers its key length rests on with that reference in place of the snap reads the snapped family only to find the points at which it evaluates them; on such a block, evaluated off its snap, no premise about the snap is needed, and this assumption does not cover it. Where the run does read a moved block, because no reference reaches the evaluation, the evaluation on the reference does not count, or the key length is formed from a number certified on the snapped family in its place, the analysis of the run is there an analysis of the snapped family,

a block whose data are an evaluation of a model not known exactly on it included, even where the snap leaves that evaluation as it stands. This assumption is the premise under which that analysis describes the device. Where the snap had to give an element support that the data deny, a block the record flags, the premise is stronger, since the snapped family may treat as possible an outcome that the model treats as impossible. On the blocks it covers it takes the place of the model as a description of the device, so that the two are never asserted of the same block; for a family that a producer snaps in its own emission, which is the target of a reduction and not the measurement of a device, it is the premise that what the run concludes from the emission holds of the snapped family. A run none of whose blocks it covers does not invoke it.

Assumption 8.1 [**A-SNAP**] (The device measures the snapped family where the run reads the snap)**.** Let F be a snap (Definition 5.5) of an operator family D read as a measurement, against a sector map with blocks $s \in \mathsf{S}$ (Definition 4.2), over an alphabet J, with D the data the snap is made from: the data as an instance states them, or, where the instance computes those data from a model of the family, that model, a reference of the family (Definition 5.3), or a producer's exact construction of what it emits (Definition 4.1). Let $B \subseteq \mathsf{S}$ be the set of blocks that the record of the run names as resting on this assumption: the moved blocks of F (Definition 5.5), those on which the record does not show, by an exact decision, that $F_j^{(s)} = D_j^{(s)}$ for every $j \in J$, that are not evaluated off its snap (Definition 5.7). A kept block and a block evaluated off its snap do not belong to B. Every other moved block belongs to B, whatever its cause; among them are the blocks on which the snap gives an element support outside the structural support of $D_j^{(s)}$ (Definition 5.5).

(i) Where D is the data an instance states as the measurement of a device, the model it computes them from, or a reference of it, then on every block $s \in B$ the block on s of the measurement the device performs, read against the sector map, is the family $\{F_j^{(s)}\}_{j \in J}$ with the entries recorded for the snap. On such a block, any statement of another assumption of the run that a model describes that measurement is not asserted, and this clause holds in its place.

(ii) Where D is a producer's exact construction of what it emits, every conclusion about the device that the claims of the run draw from the emitted family, under their hypothesis that it is the producer's exact construction, holds when the emitted family is $F^{(s)}$ on every block $s \in B$.

The engine does not verify this assumption; a run that rests on it records it as unverified. *Reading:* Applies only where the run record names blocks as resting on it, a premise about your device. The engine moves each measurement onto its numerical lattice. Where it

holds a reference of your model, it computes the key rate on the model at the points the moved measurement found; where the reported rate was formed there, the blocks carrying the reference do not rest on this. On the named blocks, the key rate is computed as if the device performed the moved measurement. It may treat as possible an outcome your model treats as impossible, except in rows and columns the record states to be exactly zero; on a block flagged support_not_preserved, not even there. The engine cannot check this. The record names the cause and a fix: a model with a reference, a run that uses enclosed references, the reasons the reference's bound was not counted or certified, or your device's loss and dark counts.

Uses Def. 4.1, Def. 4.2, Def. 5.3, Def. 5.5, Def. 5.7 · *Category* receiver · *Affects* secrecy, completeness · *Assumed by* Req. 10.5, Req. 11.3, reductions/cor:reference-companion, reductions/req:emission-record, reductions/thm:emit · *Machine key* `snapped_measurement` · newly derived · accepted

Part II
Engine requirements

What an implementation of this module must do, record or refuse; each requirement names the result that motivates it.

9 Admission

The section's main requirement, Requirement 9.1, is that declared operator data are exactly admissible at every module boundary. Every consumer decides its admission predicates exactly and refuses otherwise, and it alters nothing it reads except by rounding an emitted bound in a direction its producer licenses as weakening. A sum is decided in the order declared with its data.

Declared operator data cross every module boundary exactly admissible, so every sum a predicate reads is the exact sum. The declarer makes them so, or, for data an instance passes on unchanged, the consumer's exact decision on entry does, and a consumer that finds otherwise refuses them.

On exactly admissible data every sum a predicate reads rounds nowhere, and so is the exact sum (Lemma 3.4): each predicate a module decides is a predicate on the declared numbers and not on their rounding.

Requirement 9.1 (Exact admissibility at every module boundary). Declared operator data are exactly admissible at every module boundary (Definition 4.1). Precisely:

(i) every declared operator family that a consumer reads as a measurement, that is, as a positive operator-valued measure or as a family with a declared completeness constant, is exactly admissible with that constant for the sector map the consumer holds (Definition 4.3);

(ii) every other declared operator, and every declared matrix or vector whose entries are declared values, satisfies exactly, on those values, each condition its declaration states of it, such as Hermiticity, positivity, a trace, a column sum or an identity with another declared operator, and every sum such a condition reads rounds nowhere in its declared order (Definition 3.3).

This is established before the data cross the boundary. An instance establishes it by constructing or snapping what it declares (Requirement 10.4), or by passing on unchanged, from a supplier's output (Definition 4.1 (ii)), data whose conditions the consumer decides exactly on entry, that decision then establishing it. A producer establishes it by constructing exactly what it emits and by deciding exactly, before emitting, each condition that its

construction does not already establish, a sum that rounds nowhere among them; for a measurement family that decision is the one of Requirement 11.3. Data that fail it do not cross, because a consumer that finds one of its admission predicates violated refuses them (Requirement 9.2).

Motivated by Def. 3.3, Lem. 3.4, Def. 4.1, Def. 4.3, Req. 9.2, Req. 10.4, Req. 11.3 · accepted

Every consumer decides each admission predicate on declared data exactly, has no tolerance input, and refuses with a reason naming the failed predicate. It repairs nothing it was handed and alters nothing, except that it may round an emitted bound in a direction its producer licenses as weakening.

A consumer might be allowed to admit data that are only close to admissible data. Remark 4.4 shows that a computed sum close to the identity says nothing about the exact one, and Proposition "Instability of the rate under measurement perturbations" (meat-core/prop:optol-nobound) shows that closeness of two declared measurements carries no key-rate claim from one to the other. A bound rounded in the direction that weakens it is another matter: what its producer's claims establish survives the rounding, at a cost in rate. The acceptance set that the statistics module emits as exact rationals is read by meat-core in this way, each lower bound of the box rounded down and each upper bound up to a binary64 number, the direction that Lemma "Widening the box keeps its guarantees" (statistics/binomial-box/lem:widening) licenses.

Requirement 9.2 (Exact decision by every consumer). Every consumer (Definition 4.1) decides every admission predicate it states of declared data exactly, on the declared values, and none more loosely than exactly: no consumer has an input through which a predicate on declared data would be decided at a nonzero defect, and none admits data on the ground that they lie within a tolerance of data it would admit. A consumer that finds such a predicate violated refuses, with a machine-readable reason naming the predicate that failed. It repairs nothing it was handed, and it neither snaps nor otherwise alters declared data of another declarer, with one exception: it may replace a scalar bound that a producer emits by that bound rounded to a binary64 number in a direction that the producer's own claims license as weakening it. The direction in which each bound so replaced was rounded is recorded where the consumer's reading of the emission is specified, nothing else of the emission is altered, and every predicate on such a bound is decided exactly on its replaced value. The declared tolerance therefore widens nothing that any consumer admits; it bounds only how far an instance may move its own declaration (Requirement 10.1).

Motivated by Def. 4.1, Rem. 4.4, Req. 10.1, meat-core/prop:optol-nobound, statistics/binomial-box/lem:widening · accepted

Where a predicate on declared data is a sum, its order of accumulation is declared with the alphabet that indexes it and travels with the data. Every consumer, and every declarer, that decides the predicate decides it in that order.

Requirement 9.3 (Summation order declared with the data). Where a predicate on declared data is a sum, the order in which the sum is accumulated is part of the declared data: it is declared with the alphabet that indexes the sum, as that alphabet's declared order (Definition 3.3), or with an alphabet containing it, whose declared order it then carries by restriction; and it travels with the artefact that carries the data. Where a declared order arrives both with the alphabet and with an alphabet containing it, the two are one order: the first is the restriction of the second. A consumer that receives two such orders that differ takes the order of the containing alphabet, decides the predicate in its restriction, and refuses nothing for the difference. Every consumer that decides the predicate, and every declarer that decides it of data it is about to declare (Definition 4.1), decides it in that order, and no consumer substitutes an order of its own; a supplier deciding its own conditions at slacks declared to it is not bound by this. Only where no declared order arrives with the alphabet or with an alphabet containing it is the declaration's own listing of the alphabet taken as its declared order. Where such a predicate reads the trace of a declared matrix, the trace is accumulated in increasing row index.

Motivated by Def. 3.3, Def. 4.1 · accepted

10 The tolerance, norms and snapping

One declared tolerance per instance caps the defect an instance may repair in its own declaration, and it governs nothing else; a probability vector prepared from ordinary numerical inputs is admitted by the input preparation's own rule. A bound in one norm is never read as a bound in another, and only the declarer of data snaps them, on record. A block of a snap of a measurement family known exactly on the lattice is kept, and every other block is moved by one procedure that aims at fidelity; a moved block that the run does not evaluate off its snap is disclosed with its cause, one of four groups and a fix, or refused where the run declares the strict option (Requirement 10.5). No snap revives an element, row or column that the data state to be zero, save a flagged last resort where nothing else restores the conditions of a snap (Requirement 10.6).

One tolerance in [0, 1) per instance, with no default, caps the defect, measured in binary64, that an instance may repair in what it declares. A family snapped on an exact reference has no defect, supplier output is decided by its consumer, a prepared probability vector by its own rule.

The domain stops below one because at a tolerance of one or more a completeness test would admit the all-zero family, whose completeness residue against the identity is exactly one on every block. How an instance turns ordinary numerical inputs into declared values, the representation policy by which a binary64 input denotes a rounding cell included, is documented with the run tier (`runs/input-preparation.md`).

Requirement 10.1 (One declared tolerance per instance). Each instance declares exactly one declared tolerance τ_{num} (Definition 5.1) in its run configuration, and τ_{num} has no default: a run whose configuration does not declare it, or declares a value that is not a binary64 number in $[0, 1)$, is refused before anything is constructed. τ_{num} is the ceiling on the defect an instance may repair in what it declares. It binds the data an instance is about to declare other than numbers of a supplier's output that it passes on unchanged (Definition 4.1 (ii)); those it neither measures nor repairs, and their consumer decides their conditions exactly on entry (Requirement 9.1). Of the data it binds that have no exact reference on whose centres the snap runs (for those, see the next paragraph), the instance measures in binary64 the defects (Definition 5.1) and, for each block read as positive semidefinite, the least eigenvalue of its Hermitian part, each on the data as they stand before any step of the snap, and records those measured numbers. Data with a measured defect above τ_{num} or a measured least eigenvalue below $-\tau_{\text{num}}$ are refused as a modelling error; the others may be snapped (Definition 5.5), the snap being recorded with the measured defects it corrected (Requirement 10.4). Near τ_{num} this decision on measured numbers may differ from the predicate *within the declared tolerance* of Definition 5.1, whose sums are exact, and no bound on that difference is kept. This decision is not on the exact island; the exact decisions are those by which the snap establishes the conditions of its output (Definition 7.1 (i)).

An operator family that an instance snaps on the centres of an exact reference (Definition 5.3), as Requirement 10.5 (ii) states, stands before the snap as that reference, which is Hermitian, positive semidefinite and complete by the exact decision on its entries, and never by a binary64 evaluation. Its defects are therefore zero and no block of it falls below the positivity floor: τ_{num} decides nothing for it, and the instance records the defects as zero on that ground rather than on a binary64 measurement. τ_{num} keeps its role for the data it binds on which the snap runs as they stand, an evaluation of a model whose reference has a radius that is not zero included, since the snap does not run on such a reference.

A probability vector that an instance prepares from ordinary numerical

inputs before declaring it is decided instead by the rule of the input preparation. It is refused unless its normalisation defect, computed exactly on the inputs as given, is at most τ_{num}, or the sum of the rounding cells of its inputs contains 1: a binary64 input contributes the rounding cell it denotes under the preparation's documented representation policy, and an input given exactly, as a decimal or a fraction, contributes the one point that is its value. A value below 0 or above 1 is refused. The preparation records the defect, the cells and which of the two grounds admitted the vector. Its snap then runs at a *guard*, the larger of τ_{num} and the completeness residue the snap measures on the binary64 conversion of the inputs, which may exceed τ_{num}; the guard is recorded, and it admits no vector the rule has refused.

Motivated by Def. 4.1, Def. 5.1, Def. 5.3, Def. 5.5, Def. 7.1, Req. 9.1, Req. 10.4, Req. 10.5 · accepted

The declared tolerance governs only the repair of an instance's own data. Sets defined by declared data stay exact, a window on a computed object is another declared number, and a physical bound or a slack on a supplier's evaluation is never derived from the tolerance.

REQUIREMENT 10.2 (Scope of the declared tolerance). The declared tolerance governs the repair of an instance's own declared data (Requirement 10.1) and nothing else; in particular no consumer reads it (Requirement 9.2).

(i) A set defined by declared data is the exact set those data define, and the declared tolerance relaxes none of its defining conditions. A consumer that certifies a supplied point as a member of such a set certifies membership in the exact set, deciding it exactly on the declared values of the point. A check of a supplied point at a window of the module's own certifies no membership in such a set, and its window is a number of that module, as in (ii).

(ii) The declared tolerance does not govern a module's arithmetic on computed objects (Definition 4.1): a window at which a module checks a computed object, or within which it repairs one, is a number that module declares and records in its own right, and it is never the declared tolerance.

(iii) The declared tolerance is a numerical tolerance and not a device-uncertainty budget. A declared number that bounds a physical quantity, such as a tail budget, is declared separately and is never derived from the declared tolerance. A slack that an instance declares to a supplier (Definition 4.1 (ii)) for the supplier's own conditions is a declared number of its own. One that bounds the error of the supplier's floating-point evaluation of a quantity, such as a slack on a computed

eigenvalue (Requirement 12.1), is never derived from the declared tolerance; any other is derived from it, if at all, only as Requirement 10.3 says.

Motivated by Def. 4.1, Def. 5.1, Req. 9.2, Req. 10.1, Req. 10.3, Req. 12.1 · accepted

A bound stated in one norm is never read as a bound in another. A declared number derived from a bound in another norm carries the conversion factor between the two norms, written down and recorded by the party that declares it.

The operator norm and the entrywise maximum modulus differ by up to the dimension, and the factor is attained (Lemma 5.2); at dimensions in the thousands, reading one as the other would put that factor inside a security decision.

Requirement 10.3 (Explicit conversion between norms). No bound stated in one norm is read as a bound in another. The defects of Definition 5.1 are entrywise maximum moduli and the positivity floor is a bound on a least eigenvalue, and neither is converted, implicitly, into the other or into a bound in any other norm. A declared number that is derived from the declared tolerance, or from any bound stated in one norm, for use as a bound in another norm is derived explicitly by the party that declares it, with the conversion factor between the two norms concerned written down (for the operator norm and the entrywise maximum modulus, that of Lemma 5.2 (i)), and it is recorded together with that factor.

Motivated by Def. 4.1, Def. 5.1, Lem. 5.2 · accepted

Only the declarer of data snaps them, once and before any consumer reads them: an instance its own declaration, a producer its own emission. No party alters data it was handed, save a consumer rounding an emitted bound in a direction its producer licenses as weakening, and every snap is recorded.

A snap repairs what a party asserts, before anything consumes it. It is never a repair of what one module hands another, which a consumer decides and, where it fails, refuses.

Requirement 10.4 (Snapping by the declarer only). Declared data are snapped (Definition 5.5) only by their declarer, at most once, and before any consumer reads them: an instance snaps what it declares about its device and its protocol, on the admission that Requirement 10.1 states (for an operator family snapped on the centres of an exact reference, the decisions on the reference that Requirement 10.5 (i) lists; for a probability vector prepared from numerical inputs, the rule of the input preparation; otherwise within the declared tolerance), and a producer snaps only its own emission. No party snaps data it was handed, and no party otherwise alters them, with the one exception that Requirement 9.2 states and limits: a consumer may replace a scalar bound that a producer emits by that bound rounded in a direction that the producer's own claims license as weakening it, recording

the direction. Every snap is recorded: an instance's in the record of the run, with the defects it measured, or decided to be zero, before the snap (Definition 5.1), the displacement and the rule it applied, and a producer's in its emission, with the disclosures of Requirement 11.3. The record of a snap of an operator family read as a measurement carries, besides, the identity of every reference the declarer uses or hands beside the snap (the model, the exact parameters from which it is formed, and its entrywise enclosures where any radius is not zero, with the claims that establish them; Definition 5.3); the *reference kind* of the snap per block (*exact*, where the snap runs on the centres of an exact reference of the block, or *none*, where it runs on the data as they stand); whether the run declares that enclosed references are not used; the status of every block, kept or moved (Definition 5.5); the steps of the procedure that made each moved block (Requirement 10.6); and the flag `support_not_preserved` of every block snapped by the last resort (Requirement 10.6 (iv)). Where the declarer hands the reference beside the snap (Definition 5.7 (i)), the record names the handed blocks, with the kind of the reference handed on each (*exact*, every radius zero, or *enclosed*, with the largest radius of the block and the largest radius of the enclosures of irrational inputs (square roots, trigonometric values) from which the constructing module computed the centres exactly, as its claims state it), and, for every block on which it hands none, the cause of Requirement 10.5 (iii)(a). The record of the run names every moved block of the snap that is evaluated off its snap (Definition 5.7 (ii)), and every moved block that rests on Assumption 8.1, with its cause, group and fix (Requirement 10.5 (iii)); a kept block is named as kept and in neither list. Handing the reference alters neither the snap nor the reference. A producer that declares a lattice exponent re-establishes its lattice at each emission whose exact construction lies off it, by a snap of that emission or by refusing it (Requirements 11.1 and 11.3); a producer that declares no lattice exponent snaps nothing.

Motivated by Def. 4.1, Def. 5.1, Def. 5.3, Def. 5.5, Def. 5.7, Ass. 8.1, Req. 9.2, Req. 10.1, Req. 10.5, Req. 10.6, Req. 11.1, Req. 11.3 · *Used by* receiver/detector-imperfections/req:snap-fix · implemented (effective: accepted) · *Code* certiqs_key.foundations.snap:MeasurementSnapRecord

A measurement snap keeps the blocks known exactly on the lattice and moves the others by one fidelity-first, support-safe procedure. A moved block not evaluated off its snap rests on the snap premise, recorded with cause, group and fix; the strict option refuses it.

Closeness of two declared measurements does not bound the distance of their key-rate bounds (Proposition "Instability of the rate under measurement perturbations" (meat-core/prop:optol-nobound)), and a small move can change which outcomes a measurement treats as impossible. No claim relates a snapped family to the model it replaced, so the soundness of a run never rests on the snap where the run can avoid reading it. A run that holds a

reference of the family hands it beside the snap and evaluates the numbers its key length rests on with the reference in place of the snap (Requirement "The first-order bound evaluated on the reference companion" (meat-core/frank-wolfe/req:bracket-on-reference)): the snap then only steers the search, and every block so evaluated is evaluated off its snap (Definition 5.7) and rests on no premise about the snap. A producer that snaps its own emission holds its exact construction, a reference of the emission, and where its claims allow it forms beside the emission a companion built by that construction with no step of the snap, so that the classes it moved are evaluated off their snap in the same way. The snap is therefore made for the search and for the blocks where the run does read it, and on those it aims at fidelity: it keeps a block known exactly on the lattice, and otherwise stays as close to the model as the rule allows that nothing the data state to be zero becomes non-zero (Requirement 10.6), so that the premise departs from the model as little as possible where a run has to rest on it.

A block the run does not evaluate off its snap rests on the premise that the device measures the snapped family, Assumption 8.1, and the record and a report on the run disclose it. That happens where no reference reaches the evaluation, and where the evaluation on the reference does not count. A user acts on the group of such a block, of which there are four, each with its fix; the cause recorded with it names which change within the group's fix the block needs. A reference that reaches the evaluation can still be set aside after it: where the bound on the reference counts but no point of it can be certified, a number certified on the snapped declaration may carry the key length in its place, and the blocks the reference carried then rest on the premise after all. A run that must rest on no such premise declares the strict option of (iv), which refuses instead.

That a reference with non-zero radii encloses the model, and is positive semidefinite and complete, cannot be read off its centres and radii, so (i) asks that the module constructing it establish these properties by its claims and compute the enclosures exactly; a binary64 evaluation carries roundings of unknown size. The steps of the procedure are specified with the shared numerical code (`spec/architecture.md`) for an instance, and by its own claims for a producer.

REQUIREMENT 10.5 (Snaps of a measurement family: kept, or moved and disclosed). (i)

The reference. Before it snaps an operator family read as a measurement, an instance of what it declares or a producer of its own emission (Definition 4.1), the declarer decides exactly, of every reference of the family it uses or hands beside the snap (Definition 5.3), that every centre is Hermitian with entries in $\mathbb{Q}(i)$ and that the radii are non-negative rationals with $r_{il} = r_{li}$, of every factor enclosure it carries (Definition 5.4) that the centre of its matrix box has entries in $\mathbb{Q}(i)$, that its radii are non-negative rationals and that its core is diagonal

with positive rational entries, and, for an exact reference, that it is positive semidefinite and complete at its constant. For a reference with a radius that is not zero, three properties are not decided by the declarer: that its enclosures enclose it, (5.1), and that it is positive semidefinite and complete at its constant. They are established by the claims of the module that constructs the reference, which the record names, and its centres and radii are computed in exact rational arithmetic on the exact island (Definition 7.1 (i)); a binary64 evaluation establishes none of them. A reference of which any of them is not so established is neither used nor handed. Nor is it decided by the declarer, of any reference, that a block on which it carries a factor enclosure is a member of it, (5.3): that too is established by the claims of the module that constructs the reference, which the record names, and the centres and radii of the factor enclosure are computed in exact rational arithmetic or by a rigorously enclosed computation (Definition 7.6). A factor enclosure of which either is not so established is not carried, and the reference is read without it. A producer's exact construction, where it is exactly Hermitian, positive semidefinite and complete, is an exact reference of its emission. An instance that holds a reference so established hands it beside its snap (Definition 5.7 (i)) on every block, save that, in a run whose configuration declares that enclosed references are not used, by the option `reference_enclosure`, it hands only the blocks every radius of which is zero.

(ii) *The snap.* Every such snap is made block by block. A block is kept, $F^{(s)} = G^{(s)}$, where the declarer establishes an exact reference G of that block, against the sector map of the block alone, decided as (i) states, every element of which is a lattice value at the lattice exponent k (Definition 3.1 (iv)), and where exact admissibility at the constant c (Definition 4.3) and the lattice condition at k, decided last on the block, hold. For a producer the block kept is the one its own claims keep. Every other block is made by a procedure that meets Requirement 10.6 and aims at fidelity to the data it starts from rather than at any other property: an instance's is one procedure for every such block, the support-safe steps specified with the shared numerical code, run on the centres of an exact reference of the block where the declarer establishes one, and otherwise on the data as they stand, admitted within the declared tolerance; a producer's is the procedure its own claims state. Where that procedure yields on a block no data that satisfy the conditions of Definition 5.5, the block is snapped by the last resort of Requirement 10.6 (iv), which the record flags; where the last resort too yields no such data, the declarer refuses the data, naming the block and the condition that failed. Every block made by this procedure or the last resort is moved (Definition 5.5) unless the snap

is decided exactly to equal on it the data it is made from.

(iii) *Resting.* A moved block *rests on Assumption 8.1* when it is not evaluated off its snap (Definition 5.7); a kept block and a block evaluated off its snap do not. The record of the run names every block that rests on the assumption with its cause, its group and a fix. The group is *last resort* for a block snapped by the last resort, whose fix is a model that includes the noise the device has, so that every element of the block is positive definite, together with the fix of its cause; otherwise it is the group of its cause:

(a) *no usable reference*: no reference is handed on the block, because the declarer holds no reference of the family, because a reference fails the decisions of (i) or a property that (i) leaves to the constructing module is not established, or because the run declares that enclosed references are not used and a radius of the block is not zero; for a class of a producer's snap of its own emission, because the producer forms no companion that carries its exact construction past the snap, a *bypassing companion*. The fix is a model of the family with a reference that the run uses, and for the last of the instance's causes an exact reference, or a run that uses enclosed references; for a producer's class, declared numbers on which its exact construction stays on the lattice, so that the class is kept;

(b) *reference bracket withdrawn*: a reference is handed on the block, or, for a producer's class, a bypassing companion is formed, and a companion carrying the reference on the block, or bypassing the class, is evaluated, but the key length the run reports is formed from a number on the snapped declaration, for one of three causes. Either no evaluated point contributes a bound on the companion, so the run's first-order number is formed on the snapped declaration (Requirement "The first-order bound evaluated on the reference companion" (meat-core/frank-wolfe/req:bracket-on-reference)); or the certified evaluation finds the two enclosures of a block the companion carries inconsistent, the bracket on the companion being withdrawn and the run's first-order number formed on the snapped declaration, the cause `factor_enclosure_inconsistent` (Requirement "The certified evaluation of the first-order bound" (meat-core/frank-wolfe/req:certified-evaluation) (iv)); or a point contributes, but the certified evaluation on the companion certifies no point and the key length is formed from a number certified on the snapped declaration in its place, the cause `companion_not_certified` (Requirement "The certified evaluation of the first-order bound" (meat-core/frank-wolfe/req:certified-evaluation) (i)). The fix is

the one the reasons recorded at the evaluated points of the companion name: the withdrawal reasons for the first cause and the reasons of the certified evaluation for the third; for the second, no change of the declaration, an inconsistency being a defect of the engine in the modules that carry the enclosures, so the fix says that and that the defect be reported;

(c) *reference bracket not evaluated*: a reference is handed on the block, or, for a producer's class, a bypassing companion is formed, but no companion carrying the reference on the block, or bypassing the class, is evaluated, because none is formed, compiled or admitted. The fix is the one the reason recorded for the companion names.

(iv) *The strict option.* A run whose configuration declares the *strict option* `strict_snaps` refuses wherever a moved block would rest on Assumption 8.1, naming the block, its cause, its group and the fix, and reports no key length that rests on the assumption. It refuses as soon as the resting is decided: a declarer when it snaps a moved block on which it hands no reference, a producer when it snaps a moved class that no bypassing companion it forms carries past the snap, and the run, after its search and before any key length is reported, where a moved block or class is not evaluated off its snap. A strict run forms no certified number on the snapped declaration in place of a companion's (Requirement "The certified evaluation of the first-order bound" (meat-core/frank-wolfe/req:certified-evaluation) (i)), so the cause `companion_not_certified` of (iii)(b) does not arise in it, and it refuses on the other two. A strict run none of whose moved blocks rests on the assumption proceeds as one that does not declare the option. In a run that does not declare it, the run proceeds; its record carries, for every snap, the status of each block, kept or moved, the flag of every block snapped by the last resort, the blocks that rest on the assumption with their causes, groups and fixes, and Assumption 8.1 among the assumptions the run rests on, naming those blocks, so that a report on the key rate can flag that the rate rests on a snap. A run none of whose blocks rests on the assumption does not rest on it.

Motivated by Def. 3.1, Def. 4.1, Def. 4.3, Def. 5.3, Def. 5.4, Def. 5.5, Def. 5.7, Def. 7.1, Def. 7.6, Req. 10.6, meat-core/frank-wolfe/req:bracket-on-reference, meat-core/frank-wolfe/req:certified-evaluation, meat-core/prop:optol-nobound · *Assumes* A-SNAP · *Used by* Req. 11.3, meat-core/frank-wolfe/req:bracket-on-reference, meat-core/frank-wolfe/req:certified-evaluation, meat-core/rem:declared-measurement, receiver/detector-imperfections/req:dark-composed-reference, receiver/detector-imperfections/req:snap-fix, reductions/req:reference-companion · implemented (effective: accepted) · *Code* certiqs_key.foundations.snap:snap_measurement_family

No snap of a measurement family revives an element, row or column that the data state to be zero: a positivity lift and a completeness residue go only where the receiving element's structural support holds them. Where that fails, a flagged uniform lift is the last resort.

What a key rate is sensitive to is a change of which outcomes a measurement treats as impossible, more than a change of size (Proposition "Instability of the rate under measurement perturbations" (meat-core/prop:optol-nobound)). A snap carries no guarantee about the supports of its elements beyond what it is built to keep, so the requirement asks of every snap the part that can be read off the data: an element, a row or a column that the data state to be zero stays zero. A uniform positivity lift, which mixes every element with a multiple of the identity, violates it on every element with a structurally zero row or column, though not on a rank-deficient element whose structural support is full, such as the projector onto a direction that is not a coordinate; a lift confined to the coordinates of each element's own structural support, and a residue placed only where the receiving element already lives, do not. The rule is structural, not spectral: an element that is rank-deficient on a direction that is not a coordinate can still gain support on that direction. The zeros it protects are the elements, rows and columns the data state to be zero: for a reference, those all of whose entries are enclosed with centre and radius both zero; a zero row of the model enclosed with a positive radius is not protected. Where the support-safe steps of a procedure cannot restore the conditions of a snap on a block, positivity above all, a uniform lift is kept as a last resort, so that the run proceeds; the block is then flagged as one whose supports may not have been preserved (a block all of whose elements have full structural support carries the flag although no element has gained structural support). An instance's procedure is specified with the shared numerical code (`spec/architecture.md`), and which blocks a run rests on, last-resort ones among them, is Requirement 10.5.

Requirement 10.6 (Support-safe snaps of a measurement family). Every snap F at a lattice exponent k of an operator family D read as a measurement (Definition 5.5), against a sector map with blocks $s \in \mathsf{S}$ (Definition 4.2) and with constant c, by an instance of what it declares or by a producer of its own emission, with D the data as they stand or a reference of the family (Definition 5.3), satisfies (i) to (iii) on every block s and for every outcome j, save on a block snapped by the last resort of (iv).

(i) *No revival.* The structural support of $F_j^{(s)}$ is contained in that of $D_j^{(s)}$. In particular, if $D_j^{(s)}$ is structurally zero, then $F_j^{(s)} = 0$; and for every index i at which both row i and column i of $D_j^{(s)}$ are structurally zero, row i and column i of $F_j^{(s)}$ are zero, which for a reference is every index whose row is structurally zero.

(ii) *Lift per coordinate.* A step of the snap that changes the elements

in order to make them positive semidefinite does one of two things. Either it adds to $F_j^{(s)}$ only a matrix $\sum_i t_i e_i e_i^*$ with $t_i \geq 0$, e_i the i-th coordinate vector of the block, and $t_i > 0$ only for indices i in the structural support of $D_j^{(s)}$; or it replaces the family on the block by $(1-w)E + wT$. Here $w \in (0,1)$ is rational; E is the family on the block as the preceding steps of the snap left it or, where every entry of $D^{(s)}$ is a Gaussian rational, $D^{(s)}$ itself; and T is a family complete at c on the block whose element $T_j^{(s)}$ is, for every j, a diagonal matrix with non-negative entries that are non-zero only at indices in the structural support of $D_j^{(s)}$.

(iii) *Residue.* A step of the snap that adds to an element what the sum of the elements lacks of $c\,\mathbb{1}_{d_s}$, or a part of it, adds it to $F_j^{(s)}$ only where the structural support of $D_j^{(s)}$ contains every index i such that row i or column i of what it adds has a non-zero entry.

(iv) *Last resort.* Where a procedure meeting (i) to (iii) yields on a block s no family that satisfies the conditions of Definition 5.5, the snap may, on that block alone, replace the family by $(1-w)E + wU$ and then restore the lattice condition and completeness; (i) to (iii) do not bind on that block. Here $w \in (0,1)$ is rational; E is the family that procedure produced on the block or, where every entry of $D^{(s)}$ is a Gaussian rational, $D^{(s)}$ itself; and U is a family complete at c on the block whose element $U_j^{(s)}$ is, for every j, a positive rational multiple of $\mathbb{1}_{d_s}$, or zero where $D_j^{(s)}$ is structurally zero. The record of the snap flags the block `support_not_preserved`, since an element may then have support where $D_j^{(s)}$ is structurally zero.

Motivated by Def. 4.2, Def. 4.3, Def. 5.3, Def. 5.5, Req. 10.5, meat-core/prop:optol-nobound · *Used by* receiver/detector-imperfections/req:snap-fix · implemented (effective: accepted) · *Code* certiqs_key.foundations.snap:support_safe_snap_block

11 Emission

These are the three emission rules. The lattice condition binds data declared at a lattice exponent, emitted bounds are rounded in the direction that weakens them, and a producer establishes exact admissibility of what it emits before emitting it (Requirement 11.3), or refuses.

Every entry of operator data declared at a lattice exponent k is a lattice value at k, and so a binary64 number. The condition binds data an instance snaps or declares to a consumer that admits only at its lattice, and the emission of a producer that declares one, and no other data.

Requirement 11.1 (The declared lattice condition). (D-LAT) Let k be the one lattice exponent (Definition 3.1 (iv)) at which a declarer declares operator data. Every entry of every operator of those data is a lattice value at k: the real and the imaginary part of the entry are integer multiples of 2^{-k} and have modulus at most 1. Each entry is then exactly a binary64 number, by Lemma 3.2 (i).

Operator data are declared at a lattice exponent k, and (D-LAT) binds them, in these cases and no other: the data an instance snaps at k (Definition 5.5); the data an instance declares to a consumer that admits declared data only at a declared lattice exponent k; and every operator that a producer which declares a lattice exponent k emits (Definition 4.1). Where data of the second case are numbers of a supplier's output that the instance passes on unchanged (Definition 4.1 (ii)), the instance does not establish (D-LAT) of them: the consumer's exact decision on entry establishes it, and the consumer refuses data that fail it (Requirement 9.1). A producer that declares no lattice exponent has no obligation under (D-LAT).

Motivated by Def. 3.1, Lem. 3.2, Def. 4.1, Def. 5.5, Req. 9.1 · *Used by* Req. 11.3 · implemented (effective: accepted) · *Code* certiqs_key.foundations.snap:is_on_lattice

An emitted scalar bound is rounded onto the lattice in the direction that weakens the claim it carries, never to nearest, and the direction is recorded. A complementary quantity is recomputed from rounded weights and never rounded on its own.

A lower bound rounded down, or an upper bound rounded up, remains a bound of the same kind, so a directed rounding keeps what the bound certifies without a rounding-error budget.

Requirement 11.2 (Rounding direction of emitted bounds). (D-DIR) Every emitted scalar bound is rounded onto the lattice in the direction that weakens the claim it carries, never to nearest: a lower bound *down*, an upper bound *up*, a constraint constant in the direction that enlarges the feasible set for its declared relation. The direction is recorded per bound, and where a family's completeness depends on a rounded weight the complementary quantity is *recomputed* from the rounded weights, never rounded independently.

It binds every producer that declares a lattice exponent k (Definition 4.1), for each scalar bound it emits, the lattice being that of the lattice values at k (Definition 3.1 (iv)); the claim that states the emission names the direction of each such bound. A producer that declares no lattice exponent, and that constructs an emitted scalar bound exactly and then rounds it to a binary64

number, rounds it in the same direction, never to nearest.

Motivated by Def. 3.1, Def. 4.1 · *Used by* meat-core/frank-wolfe/req:certified-evaluation, receiver/req:rigorous-weight · accepted

A producer emits a measurement family only after establishing exactly, on the numbers it will emit, the sector map, Hermiticity, positivity, completeness, a declared-order sum that rounds nowhere and, where it declares a lattice, the lattice condition. Otherwise it refuses or emits a disclosed snap.

Requirement 11.3 (Exact verification before emission). (D-EMIT) Every producer (Definition 4.1) emits an operator family as a measurement only after establishing, of the numbers it will emit and by exact arithmetic on them: (i) the sector map is the consumer's, decided and never measured; (ii) exact Hermiticity; (iii) exact positive semidefiniteness, by a rational LDL^* with the zero-pivot rule or an equivalent exact decision, never by a floating-point eigenvalue; (iv) exact completeness per sector block, as an identity of the emitted numbers; (v) the completeness sum in the declared order rounds nowhere in binary64, and, where it emits Alice's measurement $\{M_x\}$ and her marginal τ_A of a protocol declaration, so do the completeness sum of the one and the trace of the other; (vi) where it declares a lattice exponent k, Requirement 11.1 of every entry at k. Conditions (i)–(v) of a family are its exact admissibility with constant 1 for the sector map the consumer holds (Definition 4.3), and the clause of (v) on $\{M_x\}$ and τ_A binds only a producer that emits them.

A producer that cannot establish the conditions that bind it *refuses, naming the predicate*, save in the one case below; it repairs nothing it was handed. A producer that declares a lattice exponent k may, where the family it constructed fails any of (ii)–(iv) or (vi), instead *declare* a nearby family established under all six: a snap at k of its own emission (Definition 5.5), made as Requirement 10.5 states. Its moved blocks are related to its exact construction by no claim; a run that consumes the emission evaluates the numbers its key length rests on with the exact construction in their place where it can, and otherwise rests, on the blocks that Requirement 10.5 (iii) names, on Assumption 8.1 (ii): what it concludes from the emitted family holds of the snapped family. A run that declares the strict option of Requirement 10.5 (iv) refuses there instead. The producer must disclose that the emitted numbers are a declaration and not the constructed ones, the measured per-block displacement, the rounding direction of every weight that it rounds in constructing the emission, and, separately, the status of every block, kept or moved, and the flag of every block snapped by the last resort of Requirement 10.6 (iv). A producer that declares no lattice exponent declares no nearby family.

Motivated by Def. 3.3, Def. 4.1, Def. 4.2, Def. 4.3, Def. 5.5, Req. 10.5, Req. 10.6, Req. 11.1 · *Assumes* A-SNAP · accepted

12 Arithmetic and tiers

Every decision, certification and construction of the exact island is computed exactly, a binary64 candidate entering only through one of them, and nothing outside the island carries an error budget, save a slack a supplier spends on its own floating-point evaluation under a named assumption; a rigorous enclosure, whose operations bound their own rounding, assumes none. Every run reports each tier of its key rate it reaches and names the strongest, which carries its claim.

Every decision, certification and construction of the exact island is computed exactly, a binary64 candidate entering only through one of them. Outside it no result rests on a floating-point error budget, save a supplier's slack under a named assumption; rigorously enclosed steps assume none.

Requirement 12.1 (Exact arithmetic on the island, unbudgeted rounding elsewhere). Every decision, certification and construction of the exact island (Definition 7.1) is computed in exact rational arithmetic on the values it is given, so that what it decides, certifies or constructs holds of those values exactly. A candidate proposed by binary64 arithmetic, such as a point or a multiplier, enters the exact island only as a value given to such a decision or certification, and nothing is established of it by the arithmetic that proposed it. Outside the exact island the engine may compute in binary64, and it neither bounds nor budgets the rounding of those computations: no module other than a supplier tracks a floating-point error budget, and no result of the engine rests on one except as follows. A supplier (Definition 4.1 (ii)) may allow for the error of one of its own floating-point evaluations by a slack that an instance declares to it, under an assumption of its own claims, which it does not verify, that the slack bounds that error on the side that would make its output unsound. A result that rests on such an evaluation rests on that assumption, and the supplier carries the assumption, with its slack, in its output. A computation outside the exact island that is rigorously enclosed (Definition 7.6) is neither: each of its operations bounds its own rounding, so what it establishes holds of the exact quantity and no allowance is assumed to bound its error. The engine forms every number it reports at the certified tier from such computations and from the exact island alone.

Motivated by Def. 3.1, Def. 4.1, Def. 7.1, Def. 7.6 · *Used by* meat-core/frank-wolfe/req:certified-evaluation, meat-core/frank-wolfe/req:verified-feasible-side · accepted

Every run that reports a key rate reports it at each tier it reaches and names the strongest, whose number is the headline and gives the key length. No module labels a number with a tier its checks do not support, and a request for an unknown tier is refused.

A run reaches the certified tier only where its certified evaluation succeeds, which is the business of the solver that forms the number (Requirement "The certified evaluation of the first-order bound" (meat-core/frank-wolfe/req:certified-evaluation)); the rule below only says how the tiers a run reaches are reported.

REQUIREMENT 12.2 (Reporting of the tiers). Every run that reports a key rate reports it at every tier of Definition 7.6 that it reaches: the estimate always, the first-order number wherever the hypotheses of the claim that establishes it held, and the certified number wherever the requirement of the solver that forms it reports it at that tier, which may withhold the tier from a number that meets Definition 7.6 (iii) but rests on an upstream premise; together with the field `reported_tier`, which names the strongest tier reached, the tier that carries the run's claim. Where the first-order number is reported, the number at that strongest tier is the headline key rate, and the key length the run reports is formed from it; a number at a weaker tier is recorded beside it. No module reports a number under a tier stronger than the checks that number has passed support, and a request naming a tier that Definition 7.6 does not list is refused as an unknown tier.

Motivated by Def. 7.6, meat-core/frank-wolfe/req:certified-evaluation · *Used by* meat-core/frank-wolfe/req:certified-evaluation · accepted

Claim index

Claim	Title and identifier	Page
Definition 1.1	Operator conventions and pinching foundations/def:pinching	11
Definition 1.2	Sandwiched Rényi divergence and conditional entropies foundations/def:renyi	12
Definition 1.3	Classical divergences and the quantum relative entropy foundations/def:kl	13
Lemma 1.4	Extended divergence kernel foundations/lem:kernel	13
Lemma 1.5	Pinching lowers the trace power foundations/lem:pinchmono	14
Fact 1.6	Pinched trace-power form of the conditional Rényi entropy foundations/fact:pinch-id	15
Fact 1.7	Concavity of the pinched trace-power functional foundations/fact:concavity	16
Definition 2.1	Round model, attack sequences and attack sets foundations/def:round-model	16
Definition 2.2	Round homogeneity of a per-round family foundations/def:round-homogeneity	18
Lemma 2.3	Compressing a register onto a finite-dimensional subspace foundations/lem:register-compression	19
Definition 3.1	Binary64 numbers, declared values and dyadic lattices foundations/def:binary64	23
Lemma 3.2	Binary64 numbers on dyadic lattices foundations/lem:binary64-lattice	24
Definition 3.3	Sums in a declared order and sums that round nowhere foundations/def:declared-order	25
Lemma 3.4	Exactness of a sum that rounds nowhere foundations/lem:exact-accumulation	26
Remark 3.5	Dependence of a computed sum on its order foundations/rem:summation-order	26
Definition 4.1	Declared data and the parties to a module boundary foundations/def:declared-data	27
Definition 4.2	Sector maps and block-list operators foundations/def:sector-map	28
Definition 4.3	Exact admissibility of a declared operator family foundations/def:exact-admissibility	30

Claim	Title and identifier	Page
Remark 4.4	A complete computed sum from an incomplete family foundations/rem:computed-completeness	30
Definition 5.1	The declared tolerance and the defects it bounds foundations/def:tolerance	31
Lemma 5.2	The positivity floor against entrywise closeness to the cone foundations/lem:floor-stricter	32
Definition 5.3	Reference of a measurement family, exact or enclosed foundations/def:exact-reference	33
Definition 5.4	Factor enclosure of a block foundations/def:factor-enclosure	34
Definition 5.5	Snapping declared data onto a lattice foundations/def:snap	35
Lemma 5.6	Zeroing the centres an enclosure cannot tell from zero foundations/lem:recentred-enclosure	37
Definition 5.7	A reference handed beside a snap; a block evaluated off its snap foundations/def:evaluated-off-snap	39
Definition 6.1	Domination of one operator family by another at block weights foundations/def:domination	40
Lemma 6.2	Elementary properties of domination foundations/lem:domination	41
Lemma 6.3	Domination of a family that absorbs its residue in one element foundations/lem:absorbing-domination	43
Lemma 6.4	Rationality obstruction to the domination of a lattice family foundations/lem:rational-obstruction	44
Lemma 6.5	Positive semidefiniteness from an entrywise enclosure foundations/lem:enclosure-psd	46
Remark 6.6	What domination controls, and why it needs a noise floor foundations/rem:noise-floor	47
Definition 7.1	The exact island foundations/def:exact-island	50
Lemma 7.2	Least-eigenvalue lower bound from an approximate eigenbasis foundations/lem:least-eigenvalue-bound	52
Corollary 7.3	Least eigenvalue and rank over an enclosure foundations/cor:least-eigenvalue-over-enclosure	54
Corollary 7.4	Rank over a factor enclosure foundations/cor:rank-over-factor-enclosure	56
Lemma 7.5	Perturbation bounds for matrix functions foundations/lem:matrix-function-perturbation	57

Claim	Title and identifier	Page
Definition 7.6	The tiers of a reported key rate foundations/def:tiers	61
Assumption 8.1	The device measures the snapped family where the run reads the snap foundations/ass:snapped-measurement	63
Requirement 9.1	Exact admissibility at every module boundary foundations/req:boundary-admission	65
Requirement 9.2	Exact decision by every consumer foundations/req:consumer-decides-exactly	66
Requirement 9.3	Summation order declared with the data foundations/req:declared-order	67
Requirement 10.1	One declared tolerance per instance foundations/req:declared-tolerance	68
Requirement 10.2	Scope of the declared tolerance foundations/req:tolerance-scope	69
Requirement 10.3	Explicit conversion between norms foundations/req:norm-conversion	70
Requirement 10.4	Snapping by the declarer only foundations/req:who-snaps	70
Requirement 10.5	Snaps of a measurement family: kept, or moved and disclosed foundations/req:snap-reference	72
Requirement 10.6	Support-safe snaps of a measurement family foundations/req:support-safe-snap	76
Requirement 11.1	The declared lattice condition foundations/req:declared-lattice	78
Requirement 11.2	Rounding direction of emitted bounds foundations/req:rounding-direction	78
Requirement 11.3	Exact verification before emission foundations/req:exact-emission	79
Requirement 12.1	Exact arithmetic on the island, unbudgeted rounding elsewhere foundations/req:exact-island	80
Requirement 12.2	Reporting of the tiers foundations/req:tiers	81

Index of defined terms

admissible attack sequence, 18
attack sequence, 17
attack set, 17

binary64 numbers, 23
binary64 sum, 23
block list, 28

certified, 61
computed object, 28
computed sequential sum, 25
consumer, 27

declared data, 27
declared order, 25
declared tolerance, 31
declared value, 24
declarer, 27
defects, 31
dominates, 40

entrywise enclosure, 33
estimate, 61
evaluated off its snap, 39
exact island, 50
exact reference, 34
exactly admissible, 30

factor enclosure, 34
first-order, 61

handed reference, 39

instance, 27

kept, 36

lattice exponent, 24
lattice value, 24

matrix box, 34
module boundary, 27
moved, 36

one-block map, 28

per-round family, 18
positivity floor, 32
producer, 27

reference, 33
residual family, 40
rigorously enclosed, 61
round model, 17
round-homogeneous, 18
rounds nowhere, 25

sector map, 28
sequential sum, 25
snap, 36
structural support, 36
structurally zero, 36
sum in the declared order, 25
supplier, 27

tier, 61

within the declared tolerance, 32

References

[1] Lars Kamin, John Burniston, and Ernest Y.-Z. Tan. Rényi security framework against coherent attacks applied to decoy-state QKD, 2025.

[2] Shlok Nahar, Devashish Tupkary, and Norbert Lütkenhaus. Imperfect detectors for adversarial tasks with applications to quantum key distribution. *Quantum 10, 2044 (2026)*, 2025.

[3] Marco Tomamichel. Quantum Information Processing with Finite Resources — Mathematical Foundations. *SpringerBriefs in Mathematical Physics, Vol. 5, Springer (2016)*, 2016.

[4] Devashish Tupkary, Shlok Nahar, Amir Arqand, Ernest Y.-Z. Tan, and Norbert Lütkenhaus. A rigorous and complete security proof of decoy-state BB84 quantum key distribution, 2026.